



รายงานความก้าวหน้า 240-401 โครงการวิศวกรรมคอมพิวเตอร์ 1 ครั้งที่ 1/2569

Srisurart Autopart POS: An Offline-First Multi-Tenant Point-of-Sale System
ระบบขายหน้าร้านอะไหล่รถยนต์แบบหลายร้านค้าที่ทำงานออฟไลน์ได้

ผู้ดำเนินโครงการ

.....

Chavatik Thorarit รหัสนักศึกษา 6610110066

.....

Pattarapon Kitcharoen รหัสนักศึกษา 66101xxxxx

.....

ชื่อ-สกุล (LomerAlloys) รหัสนักศึกษา 66101xxxxx

อาจารย์ที่ปรึกษาโครงการ

.....

(ตำแหน่งวิชาการ ชื่อ-สกุล อาจารย์ที่ปรึกษา)

รายงานความก้าวหน้าโครงการวิศวกรรมคอมพิวเตอร์นี้เป็นส่วนหนึ่งของการศึกษา
ตามหลักสูตรปริญญาวิศวกรรมศาสตรบัณฑิต สาขาวิชาวิศวกรรมคอมพิวเตอร์
คณะวิศวกรรมศาสตร์ มหาวิทยาลัยสงขลานครินทร์

1 ตุลาคม 2569

สารบัญ

สารบัญ	ก
รายการรูปภาพ	ฉ
รายการตาราง	ช
รายการคำย่อ	ช
บทที่ 1 บทนำ	1
1.1 ที่มาและความสำคัญ	1
1.1.1 ร้านอะไหล่รถยนต์และปัญหาของการทำงานด้วยมือ	1
1.1.2 ระบบเดิมและข้อจำกัด	2
1.1.3 การย้ายมาใช้ Flutter แบบ offline-first	3
1.1.4 เหตุผลที่ต้องมีระบบเซิร์ฟเวอร์แบบหลายร้านค้า	3
1.2 วัตถุประสงค์ของโครงการ	5
1.3 ขอบเขตของโครงการ	6
1.3.1 ขอบเขตที่อยู่ในโครงการ	6
1.3.2 ขอบเขตที่อยู่นอกโครงการ	7
1.3.3 สิ่งที่ยานงานนี้ยังไม่ถือว่าแล้วเสร็จ	8
1.4 แผนการดำเนินงาน	8
1.4.1 ลำดับเหตุการณ์ที่เกิดขึ้นจริง	8
1.4.2 ตารางแผนงานแบบแกนต์	8
1.4.3 โครงสร้างทีมและความรับผิดชอบ	9
1.5 ประโยชน์ที่คาดว่าจะได้รับ	11
1.6 ลำดับการเรียบเรียงรายงาน	11
บทที่ 2 ทฤษฎีและความรู้พื้นฐาน	12
2.1 บทนำ	12
2.2 แนวคิดของระบบขายหน้าร้านและโดเมนอะไหล่รถยนต์	12
2.2.1 การขายและค่าคงที่ของสต็อก	12
2.2.2 การคืนสินค้าและใบลดหนี้	12
2.2.3 การรับสินค้าเข้าและต้นทุนถัวเฉลี่ยถ่วงน้ำหนัก	13
2.2.4 กะ ลิ่นชักเงินสด และเครดิตช่วง	13
2.2.5 เลขที่เอกสาร	13
2.2.6 เหตุที่ไม่ใช้ระบบ POS สำเร็จรูป	13

2.3	สถาปัตยกรรมออฟไลน์เป็นหลัก (offline-first).....	14
2.3.1	แนวคิด local-first และ offline-first.....	14
2.3.2	เหตุที่ไม่ใช้ CRDT หรือการทำสำเนา (replicate) ฐานข้อมูลโดยตรง	14
2.3.3	รูปแบบกล่องขาออก (outbox pattern).....	14
2.3.4	ความสอดคล้องในที่สุด	15
2.3.5	ความล้มเหลวระหว่างทางและผลลัพธ์ที่ไม่ทราบแน่ชัด	15
2.4	เทคโนโลยีฝั่งไคลเอนต์.....	15
2.4.1	Flutter และ Dart	15
2.4.2	SQLite, Drift และการรันบนเว็บ	15
2.4.3	การจัดการสถานะด้วย BLoC และการนำทางด้วย go_router	16
2.5	การสื่อสารไคลเอนต์-เซิร์ฟเวอร์.....	16
2.6	ธุรกรรมฐานข้อมูลและการควบคุมภาวะพร้อมกัน	16
2.6.1	คุณสมบัติ ACID และระดับการแยกจากกัน.....	16
2.6.2	การล็อกระดับแถว เดดล็อก และลำดับการล็อก.....	17
2.6.3	ธุรกรรมกับกลุ่มการเชื่อมต่อ.....	17
2.7	การแยกข้อมูลผู้เช่าหลายราย (multi-tenancy)	17
2.8	การทำซ้ำคำขออย่างปลอดภัยและรหัสกำกับกับการเขียน (idempotency).....	20
2.9	การยืนยันตัวตนและการรักษาความปลอดภัย	20
2.9.1	JSON Web Token และอายุของโทเค็น	20
2.9.2	การแฮชรหัสผ่านด้วย Argon2	20
2.9.3	การผูกอุปกรณ์และการจำกัดอัตราคำขอ.....	20
2.10	เทคโนโลยีฝั่งเซิร์ฟเวอร์.....	21
2.11	คอนเทนเนอร์และการจัดวาง	21
2.12	การบูรณาการและการส่งมอบอย่างต่อเนื่อง (CI/CD)	21
2.13	การสังเกตการณ์ระบบ (observability).....	22
2.14	การบันทึกการตัดสินใจเชิงสถาปัตยกรรม (ADR)	22
2.15	สรุป.....	23
บทที่ 3	รายละเอียดการดำเนินงาน.....	24
3.1	ภาพรวมของโครงการ.....	24
3.1.1	หลักการของสถาปัตยกรรม	24
3.1.2	สถาปัตยกรรมโดยรวม	24
3.2	ส่วนประกอบของโครงการ	26

3.3	รายละเอียดฝั่งไคลเอนต์	29
3.3.1	โครงสร้างแบบสามชั้น	29
3.3.2	ฐานข้อมูลภายในเครื่อง (Drift)	30
3.3.3	Repository ที่พอร์ตมาจาก db.js และกติกาค้างไว้	31
3.3.4	Repository แบบ API (USE_API_WRITES)	33
3.3.5	หน้าจอ	33
3.3.6	การแยกข้อมูลแคชตามผู้เช่า	34
3.4	รายละเอียดฝั่งเซิร์ฟเวอร์	34
3.4.1	โมดูลของ NestJS	34
3.4.2	การแยกข้อมูลผู้เช่าด้วย RLS และ TenantService.runTx	35
3.4.3	Idempotency และลำดับการล็อก	37
3.4.4	ฐานข้อมูล PostgreSQL	38
3.4.5	จุดปลายทาง API หลัก	41
3.4.6	การยืนยันตัวตนและการผูกเครื่อง	43
3.4.7	ระบบผู้ดูแลแพลตฟอร์ม	43
3.4.8	การจำกัดอัตรา งานเบื้องหลัง และ etcd	44
3.4.9	การวัดผลและสังเกตการณ์	44
3.5	การซิงค์ข้อมูลเฟส 2	45
3.5.1	ภาพรวมของการทำงานออฟไลน์	45
3.5.2	Outbox ในเครื่อง	45
3.5.3	POST /sync/push	47
3.5.4	เลข RC และ CN ที่ออกบนเครื่อง	47
3.5.5	การดึงข้อมูลแบบ keyset	48
3.6	ความปลอดภัย	48
3.7	การนำระบบขึ้นใช้งาน (deploy)	50
3.7.1	โครงสร้างการ deploy และระดับของ CI/CD	50
3.7.2	Ansible และ self-hosted runner	52
3.8	เทคโนโลยีที่ใช้	52
3.9	วิธีการทำงานของทีม	54
3.9.1	การแบ่งงานและการติดตามงาน	54
3.9.2	ADR และลำดับความสำคัญของเอกสาร	54
3.9.3	ขั้นตอนการพัฒนาและการตรวจ	54

3.9.4	เครื่องมือช่วยพัฒนาด้วยปัญญาประดิษฐ์.....	55
บทที่ 4	ความก้าวหน้าการดำเนินงาน.....	56
4.1	ผลการย้ายระบบเดิมสู่ Flutter	56
4.1.1	ลำดับความก้าวหน้าของการย้ายระบบ ชั้น 0 ถึง ชั้น 6	56
4.1.2	การนำเข้าข้อมูลสำรองและฐานข้อมูลบนเว็บ	57
4.2	หน้าจอของระบบ	58
4.2.1	เมื่อนำทางและหน้าขายสินค้า	58
4.2.2	หน้าสินค้า ช่าง และใบเสนอราคา	61
4.2.3	หน้าคืนสินค้า ลีนชัก และรายงานปิดร้าน.....	63
4.2.4	หน้าจัดการเครื่อง การเข้าสู่ระบบ และสถานะการเชื่อมต่อ	66
4.2.5	ความเชื่อมโยงกับปัญหาของร้าน	68
4.3	การพัฒนาชั้นเขียนข้อมูลผ่าน API	69
4.4	ผลการพัฒนาระบบทำงานออฟไลน์ เฟส 2 ฝั่งอุปกรณ์	71
4.4.1	การแบ่งงานและตัวที่ปิดแล้ว.....	71
4.4.2	ผลเพิ่มเติมที่เกิดจากการแก้ไขบกพร่อง	73
4.4.3	งานที่ยังไม่เสร็จของฝั่งอุปกรณ์.....	74
4.5	ผลการทดสอบฝั่งไคลเอนต์.....	74
4.6	การพัฒนาฝั่งเซิร์ฟเวอร์.....	74
4.6.1	ภาพรวมของฝั่งเซิร์ฟเวอร์ เฟส 1.....	74
4.6.2	การแยกข้อมูลรายร้าน	75
4.6.3	จุดเชื่อมธุรกรรม (transaction seam) และ idempotency.....	76
4.6.4	ระนาบผู้ดูแลแพลตฟอร์ม.....	76
4.6.5	เซิร์ฟเวอร์ของเฟส 2.....	77
4.7	การพัฒนา CI/CD	77
4.7.1	ระดับที่ 1 ถึง 3	77
4.7.2	ระดับที่ 4: การ deploy ด้วย Ansible และ self-hosted runner	78
4.7.3	การติดตามผลด้วย Grafana	79
4.8	ผลการนำระบบขึ้นใช้งานบนเครื่อง mob04 และการติดตามระบบ	79
4.8.1	สแตกบนเครื่องสาริต.....	79
4.8.2	การสำรองข้อมูลและ etcd	80
4.9	ผลการทดสอบฝั่งเซิร์ฟเวอร์.....	80
4.10	ตรวจสอบเกณฑ์ตรวจรับ (DoD).....	81

4.11	สรุปผลงานที่มีความก้าวหน้า.....	83
บทที่ 5	ผลการดำเนินงาน	87
5.1	สรุปผลการดำเนินงาน	87
5.2	ปัญหาและอุปสรรค	90
5.2.1	พารามีอักษไทยทำให้ build_runner ล้มเหลว	90
5.2.2	ไฟร์วอลล์ของคณะปิดกั้นการดึงอิมเมจจาก ghcr.io	91
5.2.3	pool ฐานข้อมูลหมดเมื่อมีคำขอพร้อมกัน (#162).....	91
5.2.4	การทิ้งรายการในคิวย่นผลไม่ตรง (#473, #488).....	91
5.2.5	เลขที่เอกสารชำระระหว่างการออกเลขออนไลน์กับออฟไลน์ (#489)	92
5.2.6	การควบคุมการรวมโค้ดและหลักฐาน	92
5.2.7	การทดสอบที่ผูกกับเดือนปัจจุบัน (#525)	92
5.2.8	ค่า 0 เป็นเท็จในนิพจน์ของ GitHub Actions	93
5.3	งานที่จะดำเนินการต่อไป.....	93
บรรณานุกรม.....		95

รายการรูปภาพ

รูปที่ 1.1 วิวัฒนาการของระบบ POS ร้านศรีสุรัตน์จากฐานเว็บ localStorage สู่ระบบหลายร้านค้า.....	5
รูปที่ 2.1 แนวคิด Row-Level Security: คำขอของผู้เช่าหนึ่งเห็นเฉพาะแถวของผู้เช่านั้น	19
รูปที่ 3.1 สถาปัตยกรรมโดยรวมของระบบ Srisurart Autopart POS	25
รูปที่ 3.2 โครงสร้างชั้นของไคลเอนต์ Flutter และเส้นทางไปยังเซิร์ฟเวอร์	30
รูปที่ 3.3 ลำดับการประมวลผลคำขอเขียนหนึ่งคำขอผ่านชั้นการแยกผู้เช่าและ idempotency	37
รูปที่ 3.4 แผนภาพ ER ของตารางแกนกลางในเซิร์ฟเวอร์ และตาราง outbox_ops ที่อยู่ในเครื่อง	39
รูปที่ 3.5 การไหลของรูปแบบกล่องขาออกในโครงการ ตั้งแต่การขอยอฟไลน์จนถึงการเล่นซ้ำที่เซิร์ฟเวอร์	46
รูปที่ 3.6 ภาพรวมไปป์ไลน์ CI/CD ของโครงการ ตั้งแต่คำขอรวมถึงการติดตั้งบนเครื่องสาธิต	51
รูปที่ 4.1 เมื่อนำทางบนโทรศัพท์ เรียงครบ 12 หน้า.....	59
รูปที่ 4.2 หน้าขายสินค้า แสดงเมนู รายการสินค้า และตะกร้า	60
รูปที่ 4.3 การเลือกช่าง ราคาช่าง และการชำระด้วยเครดิตช่าง	60
รูปที่ 4.4 ใบเสร็จเงินสด แสดงเงินทอนและแต้มสะสมของลูกค้า	61
รูปที่ 4.5 หน้าสินค้า/สต็อก แท็บสต็อก.....	62
รูปที่ 4.6 หน้าช่าง แสดงยอดค้างและวงเงินเครดิตของช่างแต่ละราย.....	62
รูปที่ 4.7 ใบเสนอราคา A4 ส่วนหัวและรายการสินค้า (ถ่ายก่อนแก้ #479)	63
รูปที่ 4.8 หน้าคืนสินค้า	64
รูปที่ 4.9 แท็บรายการเงินและการ์ดสรุปเงินในลิ้นชัก.....	65
รูปที่ 4.10 รายงานปิดร้าน (Daily Closing Report, ถ่ายก่อนแก้ #461).....	65
รูปที่ 4.11 หน้าจัดการเครื่อง แยกเครื่องขาย (POS Terminal) และเครื่องจัดการทั่วไป (Backoffice)..	66
รูปที่ 4.12 ฟอรัมเข้าสู่ระบบด้วย PIN ออฟไลน์บนเครื่องขาย	67
รูปที่ 4.13 สถานะออฟไลน์ ป้ายขายสำรองและแถบเตือนสีเหลือง.....	68
รูปที่ 4.14 แผงควบคุม Grafana ระหว่างการซ่อมสาธิต (บันทึกเมื่อ 21 กันยายน พ.ศ. 2569)	79

รายการตาราง

ตารางที่ 1.1 ปัญหาของร้านก่อนมีระบบและผลที่ตามมา	2
ตารางที่ 1.2 ขอบเขตของโครงการจำแนกตามเฟส	7
ตารางที่ 1.3 แผนการดำเนินงานของโครงการ (■ = ดำเนินการแล้ว, □ = แผนในอนาคต)	9
ตารางที่ 1.4 ความรับผิดชอบของสมาชิกแต่ละคนตามแผน	10
ตารางที่ 2.1 เปรียบเทียบแบบจำลองการแยกผู้เช่าในฐานข้อมูล	18
ตารางที่ 3.1 เปรียบเทียบเฟส 1 (Architecture A) กับเฟส 2 (offline shell)	26
ตารางที่ 3.2 ส่วนประกอบหลักของที่เก็บโค้ด	27
ตารางที่ 3.3 คอนเทนเนอร์หลักและเพดานหน่วยความจำ	28
ตารางที่ 3.4 กติกาธุรกิจของ repository ที่ทำธุรกรรม	32
ตารางที่ 3.5 หน้าจอหลักของไคลเอนต์	34
ตารางที่ 3.6 โมดูลหลักของเซิร์ฟเวอร์	35
ตารางที่ 3.7 ตารางแกนกลางและข้อกำหนดสำคัญ	40
ตารางที่ 3.8 กลุ่มจุดปลายทางหลักของ API	41
ตารางที่ 3.9 รายการ op ที่เข้าคิวและ endpoint ออนไลน์ที่เป็นคู่กัน	45
ตารางที่ 3.10 มาตรการด้านความปลอดภัยและที่มา	49
ตารางที่ 3.11 เทคโนโลยีที่ใช้ในโครงการ	53
ตารางที่ 4.1 ลำดับการย้ายระบบเดิมสู่ Flutter (ตรวจสอบจากประวัติ git)	57
ตารางที่ 4.2 ปัญหาของร้านกับฟังก์ชันและหลักฐานในรายงาน	69
ตารางที่ 4.3 งานขึ้นเชื่อมต่อ API ฟังก์ชันไคลเอนต์ (fe.0 ถึง fe.3)	70
ตารางที่ 4.4 ตัวเฟส 2 ฟังก์ชันที่ปิดแล้ว	71
ตารางที่ 4.5 สรุปองค์ประกอบสำคัญของฟังก์ชันเซิร์ฟเวอร์และหลักฐานการพิสูจน์	75
ตารางที่ 4.6 หลักฐานการ deploy บนเครื่อง mob04 (30 กันยายน พ.ศ. 2569)	78
ตารางที่ 4.7 จำนวนไฟล์และกรณีทดสอบของเซิร์ฟเวอร์และไคลเอนต์ (นับเมื่อ 1 ตุลาคม พ.ศ. 2569)	80
ตารางที่ 4.8 สถานะเกณฑ์ตรวจรับ (DoD) ของเฟส 1 (17 ข้อ)	81
ตารางที่ 4.9 สถิติภาพรวมของโครงการ (วัดเมื่อ 1 ตุลาคม พ.ศ. 2569)	84
ตารางที่ 4.10 ผลงานของสมาชิกแต่ละคน (นับเมื่อ 1 ตุลาคม พ.ศ. 2569)	85
ตารางที่ 5.1 ผลการดำเนินงานเทียบกับวัตถุประสงค์	88

รายการคำย่อ

ACID	Atomicity, Consistency, Isolation, Durability
ADR	Architecture Decision Record
AOF	Append-Only File
AOT	Ahead-Of-Time (compilation)
API	Application Programming Interface
ASCII	American Standard Code for Information Interchange
BLoC	Business Logic Component
CDN	Content Delivery Network
CI/CD	Continuous Integration / Continuous Delivery
CN	Credit Note (ใบลดหนี้)
CORS	Cross-Origin Resource Sharing
CP	Credit Payment (ใบรับชำระเครดิตต่าง)
CRDT	Conflict-free Replicated Data Type
DDL	Data Definition Language
DoD	Definition of Done
e2e	End-to-End
ER	Entity-Relationship
GHCR	GitHub Container Registry
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IP	Internet Protocol
JSON	JavaScript Object Notation
JWT	JSON Web Token
LRU	Least Recently Used

MVCC	Multi-Version Concurrency Control
NAS	Network-Attached Storage
NFC	Normalization Form C (Unicode)
OPFS	Origin Private File System
ORM	Object-Relational Mapping
PDF	Portable Document Format
PIN	Personal Identification Number
PO	Purchase Order
POS	Point of Sale
PR	Pull Request
PWA	Progressive Web App
QT	Quotation (ใบเสนอราคา)
RC	Receipt (เลขที่ใบเสร็จ)
REST	Representational State Transfer
RLS	Row-Level Security
SaaS	Software as a Service
SAN	Subject Alternative Name
SFTP	SSH File Transfer Protocol
SHA	Secure Hash Algorithm
SLI	Service Level Indicator
SMB	Server Message Block
SQL	Structured Query Language
SSH	Secure Shell
TLS	Transport Layer Security
UI	User Interface
VM	Virtual Machine

XSS	Cross-Site Scripting
YAML	YAML Ain't Markup Language

บทที่ 1

บทนำ

1.1 ที่มาและความสำคัญ

1.1.1 ร้านอะไหล่รถยนต์และปัญหาของการทำงานด้วยมือ

โครงการนี้เริ่มจากความต้องการใช้งานจริงของร้านจำหน่ายอะไหล่รถยนต์ขนาดเล็กแห่งหนึ่งในประเทศไทย คือ ร้านศรีสุรัตน์อะไหล่ยนต์ ร้านประเภทนี้มีลักษณะงานที่ต่างจากร้านค้าปลีกทั่วไปหลายประการ กล่าวคือ สินค้ามีจำนวนรายการมากและแต่ละรายการมีรหัสอะไหล่เฉพาะ ลูกค้าส่วนหนึ่งเป็นช่างซ่อมรถที่ซื้อซ้ำเป็นประจำและขอซื้อเชื่อ (เครดิตช่าง) สินค้าชนิดเดียวกันอาจเข้าร้านหลายล็อตในต้นทุนที่ต่างกัน และมีทั้งงานเสนอราคา งานรับคืนสินค้า และงานปิดกะเก็บเงินสดในสิ้นซัปดาห์

ตามที่สรุปไว้ในเอกสารประกอบโครงการ [1] ร้านประสบปัญหาหลัก 8 ประการก่อนมีระบบดังสรุปในตารางที่ 1.1 ตัวเลขเชิงปริมาณของปัญหา เช่น จำนวนบิลต่อวัน หรือยอดหนี้ค้างรวม ยังไม่มีข้อมูลจากร้านที่ยืนยันได้ คณะผู้จัดทำจึงไม่ระบุตัวเลขเหล่านี้ในรายงานฉบับนี้

ตารางที่ 1.1 ปัญหาของร้านก่อนมีระบบและผลที่ตามมา

รหัส	ปัญหา	ผลที่ตามมา
P1	เขียนบิลด้วยมือเป็นส่วนใหญ่	ช้า ลายมืออ่านยาก บิลสูญหาย ไม่มีการรวมยอดอัตโนมัติ
P2	ไม่แน่ใจว่าสินค้าในสต็อกเหลือหรือไม่	รับปากลูกค้าแล้วสินค้าหมด หรือสั่งซื้อซ้ำทั้งที่ยังมีของ
P3	ไม่แน่ใจราคา และสินค้าที่เข้าต่างล็อตมีต้นทุนต่างกัน	ขายขาดทุนโดยไม่รู้ตัว หรือตั้งราคาสูงเกินไป
P4	ไม่ทราบยอดรับ-จ่ายประจำวัน	เงินในลิ้นชักไม่ตรงกับบิลเมื่อปิดร้าน
P5	ไม่ทราบยอดเครดิตที่ค้างใช้ไป	หนี้ค้างสะสม ทวงถามไม่ตรงตัว ไม่รู้ว่าควรหยุดให้เครดิตเมื่อใด
P6	ไม่ทราบกำไร	ขายได้มากแต่ไม่ทราบว่ากิจการได้กำไรหรือขาดทุน
P7	ไม่ทราบยอดขาย	ตัดสินใจเชิงธุรกิจไม่ได้ เช่น การจ้างงานหรือขยายร้าน
P8	ไม่ทราบว่าสินค้าใดขายดี	สั่งสินค้าผิดตัว เงินทุนจมกับสินค้าที่ไม่หมุนเวียน

ปัญหา P1–P3 เป็นปัญหาที่พนักงานหน้าร้านสัมผัสโดยตรง ส่วน P4–P8 เป็นปัญหา "ข้อมูลที่เจ้าของร้านมองไม่เห็น" การแยกกลุ่มผู้ใช้เช่นนี้มีผลต่อการออกแบบ เพราะพนักงานต้องการขั้นตอนที่รวดเร็ว ขณะที่เจ้าของร้านต้องการตัวเลขที่ถูกต้องและตรวจย้อนหลังได้ ซึ่งบางครั้งขัดกัน เช่น การบังคับให้ยืนยันก่อนขายเชื่อเกินวงเงินทำให้ช้าลงหนึ่งขั้นตอน แต่ป้องกันหนี้ค้างได้

1.1.2 ระบบเดิมและข้อจำกัด

ก่อนโครงการนี้ ร้านมีแอปพลิเคชันจุดขาย (Point of Sale: POS) รุ่นแรกที่พัฒนาด้วย React ทำงานในเว็บเบราว์เซอร์ และเก็บข้อมูลทั้งหมดใน `localStorage` ของเบราว์เซอร์ [2] กติกาทางธุรกิจทั้งหมด เช่น การตัดสต็อก การคิดต้นทุนเฉลี่ย การคืนสินค้า และการคิดเครดิตค้างอยู่ในไฟล์ `pos/db.js` ของที่เก็บโค้ดเดิม (tag `v1.0-js-localstorage`) ระบบรุ่นนี้ไม่ต้องมีเซิร์ฟเวอร์ และเป็นต้นแบบเชิงพฤติกรรมที่โครงการนี้ยึดเป็นแหล่งความจริง (source of truth) ของกติกาทางธุรกิจ อย่างไรก็ตาม ระบบรุ่นแรกมีข้อจำกัดสำคัญ [3] ดังนี้

- ข้อมูลผูกอยู่กับเบราว์เซอร์และเครื่องเดียว หากล้างข้อมูลของเบราว์เซอร์หรือเครื่องเสีย ข้อมูลหายพร้อมกัน
- ไม่มีธุรกรรม (transaction) ที่แท้จริงในระดับฐานข้อมูล การทำหลายขั้นตอนให้สำเร็จทั้งหมดหรือไม่ทำเลยต้องอาศัยการเก็บภาพถ่าย (snapshot) และย้อนกลับ (rollback) ที่เขียนเองในโค้ด JavaScript
- ไม่สามารถใช้ข้อมูลร่วมกันระหว่างหลายเครื่อง และที่เก็บข้อมูลของเบราว์เซอร์มีข้อจำกัดด้านขนาดและเงื่อนไขการลบที่เบราว์เซอร์เป็นผู้กำหนด [2]

1.1.3 การย้ายมาใช้ Flutter แบบ offline-first

ขั้นถัดมา โครงการย้ายแอปไปยัง Flutter [4] เพราะเฟรมเวิร์กนี้สร้างแอปสำหรับ Android, iOS และเว็บได้จากการรหัสต้นฉบับชุดเดียว และเปลี่ยนที่เก็บข้อมูลเป็น Drift ซึ่งเป็นไลบรารีที่ห่อ SQLite [5][6] ทำให้ได้ธุรกรรมของฐานข้อมูลจริงแทนกลไก snapshot/rollback เดิม สถาปัตยกรรมนี้เรียกว่า offline-first กล่าวคือ ฐานข้อมูลในเครื่องเป็นที่เก็บข้อมูลตัวจริง ร้านจึงขายสินค้าได้ต่อเนื่องแม้อินเทอร์เน็ตขัดข้อง

การย้ายมา Flutter เริ่มเมื่อวันที่ 23 มิถุนายน พ.ศ. 2569 โดยในวันเดียวกันมีทั้งโครงสร้างโครงการ ขึ้นข้อมูลพร้อมการทดสอบ และหน้าจอ 11 หน้าที่มีในขณะนั้น ต่อมา มีการปรับการแสดงผลบนอุปกรณ์ขนาดต่าง ๆ และย้ายการจัดการสถานะจาก Riverpod ไปเป็น flutter_bloc เมื่อวันที่ 14 กรกฎาคม พ.ศ. 2569 ผลลัพธ์คือแอป Flutter แบบ offline-first ที่ร้านใช้งานจริงอยู่ และถูกเก็บรักษาไว้เป็นสแนปช็อตอ้างอิงบนสาขา `POC_sample_offline_first` สแนปช็อตข้อมูลจริงของร้านถูกนำเข้าร้านสาธิตบนเชิร์ฟเวอร์เมื่อ 17 กันยายน พ.ศ. 2569 (#185) แต่ยังไม่มีการเก็บความเห็นหรือประเมินผลจากผู้ใช้อย่างเป็นทางการ

1.1.4 เหตุผลที่ต้องมีระบบเชิร์ฟเวอร์แบบหลายร้านค้า

แม้แอป offline-first จะแก้ปัญหการทำงานหน้าร้านได้ แต่โครงการในรายวิชา 240-401 ถูกกำหนดโจทย์ให้ขยายเป็นระบบที่มีเชิร์ฟเวอร์กลางและรองรับร้านอะไหล่หลายร้านที่เป็นเจ้าของคนละราย (multi-tenant) ซึ่งเจ้าของโครงการยืนยันโมเดลนี้เมื่อวันที่ 3 กันยายน พ.ศ. 2569 [7] ในรายงานนี้ **เจ้าของโครงการ** (product owner) หมายถึง Chavatik Thorarit (NuimanLP) ในฐานะหัวหน้าทีม — โปรดยืนยันชื่อและบทบาท ผู้ตัดสินใจขอบเขต ลำดับงาน และเกณฑ์ตรวจรับ ข้อความภาษาไทยบนหน้าจอรับรองโดย **เจ้าของโครงการหรือเจ้าของร้าน** — โปรดยืนยัน และคำว่า **เจ้าของร้าน** หมายถึงเจ้าของร้านศรีสุรัตน์อะไหล่ยนต์ ส่วนร้านที่ใช้ระบบเรียกว่า "ผู้เช่า" (tenant) ในเชิง

เทคนิค และ "ร้านค้า" เมื่อกล่าวถึงงานของร้าน ปัจจุบันยังไม่มีร้านที่สองที่ใช้งานจริง การแยกผู้เช่าจึงพิสูจน์ด้วยร้านสาธิตหลายร้านในชุดทดสอบ ([cross-tenant-read.e2e-spec.ts](#))

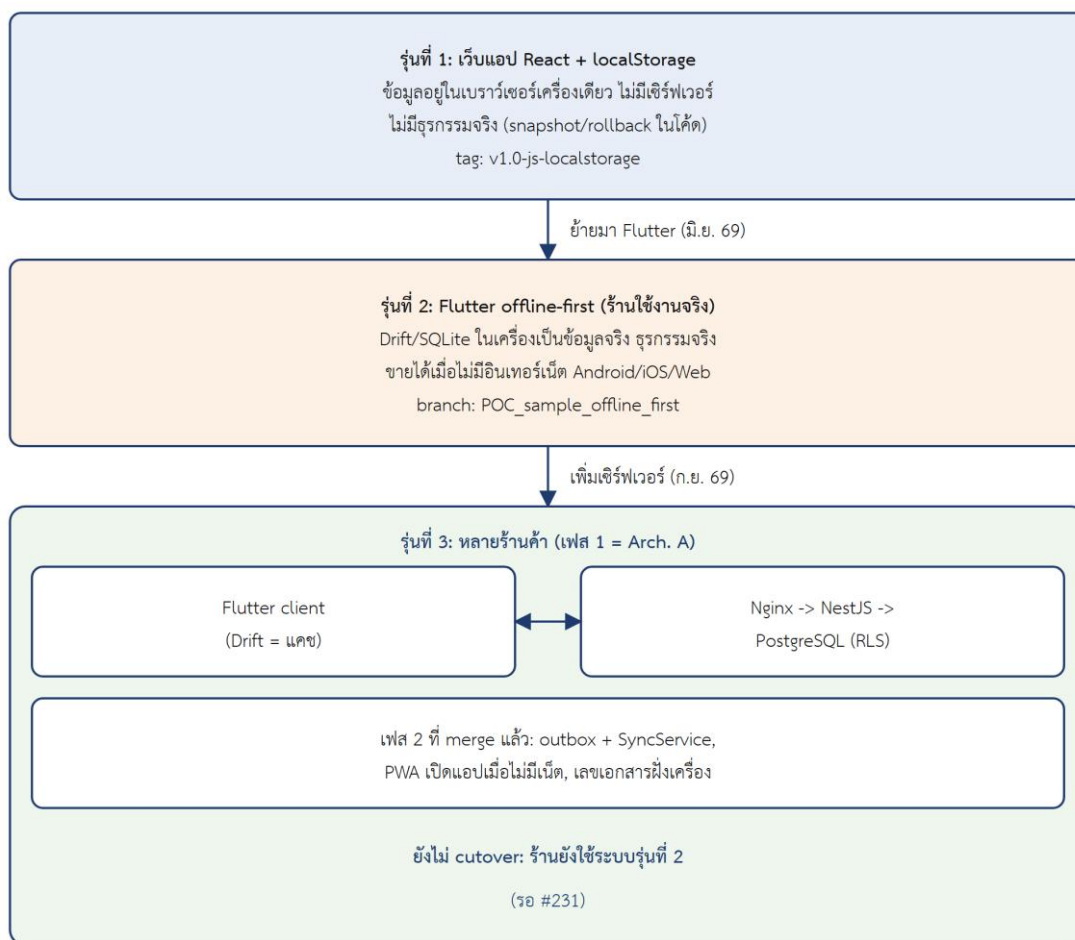
แรงผลักดันของการมีเซิร์ฟเวอร์มี 4 ประการ [3]

1. โจทย์เปลี่ยนเป็นบริการแบบ SaaS (Software as a Service) ที่ให้บริการหลายร้านที่ไม่เกี่ยวข้องกัน ข้อมูลจึงต้องอยู่ศูนย์กลางและต้องแยกขาดจากกันอย่างเด็ดขาด
2. รายวิชากำหนดชุดเทคโนโลยีไว้ ได้แก่ NestJS, PostgreSQL, Redis, BullMQ และ Nginx [3][7]
3. ข้อมูลที่อยู่ในเครื่องเดียวสูญหายพร้อมเครื่อง จึงต้องมีที่เก็บกลางที่สำรองข้อมูลได้
4. ร้านหนึ่งอาจมีหลายเครื่อง เช่น เครื่องขายหน้าร้าน ([pos](#)) และเครื่องหลังร้าน ([backoffice](#)) ซึ่งต้องเห็นสต็อกชุดเดียวกัน (บันทึกการตัดสินใจเชิงสถาปัตยกรรม (Architecture Decision Record: ADR) หมายเลข 0004 [8])

การเลือกสถาปัตยกรรมเริ่มจากคำถามกันว่า "สต็อกตัวจริงอยู่ที่เซิร์ฟเวอร์หรือที่เครื่อง" เอกสารออกแบบเปรียบเทียบสามทางเลือก คือ A (online-first: ทุกการเขียนต้องผ่านเซิร์ฟเวอร์) B (offline-first: เครื่องเป็นที่เก็บหลักแล้วซิงก์ภายหลัง) และ C (ไฮบริด: เริ่มแบบ A แล้วเพิ่มการทำงานออฟไลน์แบบจำกัดภายหลัง) ผลการตัดสินใจคือเลือก C โดยกำหนดให้เฟส 1 ของ C เท่ากับ A ทุกประการ [7] เหตุผลคือเฟส 1 ตรงกับสิ่งที่รายวิชากำหนดและไม่ต้องรอกลไกซิงก์ ขณะที่ความสามารถ "ขายได้เมื่ออินเทอร์เน็ตขัดข้อง" ไม่ถูกทิ้ง แต่กลับมาในเฟส 2 ด้วยกลไกคิวคำสั่งในเครื่องที่เรียกว่ากล่องขาออก (outbox) ราคาที่ต้องจ่ายคือต้องปรับชั้น repository ของ Flutter ให้เรียกส่วนต่อประสานโปรแกรมประยุกต์ (Application Programming Interface: API) และต้องดูแลเส้นทางโค้ดทั้งออนไลน์และออฟไลน์ ทั้งนี้ ADR-0010 กำหนดให้ Drift ยังคงอยู่เป็นแคชแบบ write-through แทนที่จะถูกลบทิ้ง [9]

สำหรับการแยกข้อมูลของแต่ละร้าน โครงการเลือกใช้ฐานข้อมูลและสคีมาเดียวกัน โดยทุกตารางมีคอลัมน์ `tenant_id` และเปิดใช้การรักษาความปลอดภัยระดับแถว (Row-Level Security: RLS) ของ PostgreSQL [10] เพราะต้นทุนต่อร้านค้าต่ำที่สุด ส่วนข้อเสนอให้เปลี่ยนฐานข้อมูลเป็น CouchDB ถูกปฏิเสธเมื่อวันที่ 8 กันยายน พ.ศ. 2569 (ADR-0012 [11]) เหตุผลของทั้งสองเรื่องอธิบายไว้ในบทที่ 2 และกลไกที่สร้างจริงอยู่ในบทที่ 3

ข้อควรย้าคือ การมีเซิร์ฟเวอร์ไม่ได้หมายความว่าร้านจริงย้ายไปใช้ระบบใหม่แล้ว เฟส 1 กำหนดชัดว่า "ไม่มี cutover" ร้านยังคงใช้แอป Drift เดิม ส่วนเซิร์ฟเวอร์พัฒนาและทดสอบกับผู้เช่าสาธิต (demo tenant) [7] ดังสรุปในรูปที่ 1.1



รูปที่ 1.1 วิวัฒนาการของระบบ POS ร้านศรีสุรัตน์จากรุ่นเว็บ localStorage สู่ระบบหลายร้านค้า

1.2 วัตถุประสงค์ของโครงการ

โครงการมีวัตถุประสงค์ดังต่อไปนี้

1. เพื่อพัฒนาแอปพลิเคชัน POS สำหรับร้านอะไหล่รถยนต์ด้วย Flutter ที่ทำงานแบบ offline-first บนเว็บเป็นหลัก (โค้ดชุดเดียวรองรับ Android และ iOS ตามความสามารถของเฟรมเวิร์ก แต่ยังไม่ได้สร้างและทดสอบบนอุปกรณ์จริง) โดยคงกติกาทางธุรกิจของระบบเดิมไว้ครบถ้วน ได้แก่ การขาย การคืนสินค้า การรับสินค้าเข้าและคิดต้นทุนเฉลี่ย ถ่วงน้ำหนัก ใบเสนอราคา เครดิตช่าง การเปิด-ปิดกะ และรายงานยอดขาย กำไร และสินค้าขายดี
2. เพื่อพัฒนาระบบเซิร์ฟเวอร์ด้วย NestJS, PostgreSQL, Redis, BullMQ และ Nginx ที่รองรับหลายร้านค้า โดยให้ฐานข้อมูล PostgreSQL เป็นแหล่งความจริง และย้ายกติกาธุรกรรมที่เกี่ยวกับเงินและสต็อกไปบังคับใช้ฝั่งเซิร์ฟเวอร์

3. เพื่อออกแบบและพิสูจน์การแยกข้อมูลระหว่างร้านค้าด้วย `tenant_id` และ RLS พร้อมการทดสอบที่แสดงว่าการอ่านข้ามร้านได้ผลลัพธ์ศูนย์แถว
4. เพื่อให้การเขียนข้อมูลที่เกี่ยวข้องกับเงินและสต็อกปลอดภัยเมื่อมีคำขอซ้ำหรือพร้อมกัน ด้วย กลไก idempotency key และการล็อกแถวตามลำดับที่กำหนด
5. เพื่อพัฒนากลไกการทำงานเมื่อไม่มีอินเทอร์เน็ต (เฟส 2) ได้แก่ คิวคำสั่งในเครื่อง การ ชิงก์ การออกเลขเอกสารฝังเครื่อง และการผูกอุปกรณ์กับร้าน ตามสเปกของเฟส 2 [12]
6. เพื่อสร้างกระบวนการ CI/CD (Continuous Integration / Continuous Delivery) ที่ ครอบคลุมการทดสอบอัตโนมัติ การสแกนความปลอดภัย การสร้างอิมเมจ การนำระบบ ขึ้นใช้งาน (deploy) บนเครื่องเสมือน (Virtual Machine: VM) ของคณะ และการเฝ้า สังเกตระบบ [13]
7. เพื่อบันทึกการตัดสินใจเชิงสถาปัตยกรรมเป็น ADR พร้อมเหตุผล เพื่อให้ผู้ร่วมงานและผู้ ตรวจสอบตามย้อนได้ว่าเหตุใดจึงเลือกแต่ละแนวทาง

1.3 ขอบเขตของโครงการ

1.3.1 ขอบเขตที่อยู่ในโครงการ

ขอบเขตของโครงการแบ่งตามเฟสของสถาปัตยกรรมแบบไฮบริด ดังสรุปในตารางที่ 1.2

ตารางที่ 1.2 ขอบเขตของโครงการจำแนกตามเฟส

เฟส	สิ่งที่อยู่ในขอบเขต	สถานะ ณ วันที่ 1 ตุลาคม พ.ศ. 2569
ฐาน Flutter	แอป offline-first (Drift/SQLite) หน้าจอ 12 หน้าในแถบนำทางหลัก กติกาธุรกรรมที่พอร์ตจาก db.js	แล้วเสร็จ ร้านใช้งานจริงบนเว็บ (ยังไม่ทดสอบบน Android/iOS)
เฟส 1	เซิร์ฟเวอร์ NestJS, PostgreSQL พร้อม RLS, ชั้นธุรกรรมและ idempotency, การจัดการร้านค้า โดยผู้ดูแลแพลตฟอร์ม, ขึ้นเขียน ข้อมูลผ่าน API ของ Flutter, CI/CD ระดับ 1-3	แล้วเสร็จ เว้นการวัดภาระด้วย k6
เฟส 2	เว็บแอปแบบก้าวหน้า (Progressive Web App: PWA), outbox, SyncService , เลข ใบเสร็จ (RC) และใบลดหนี้ (CN) ผังเครื่อง, รหัสส่วนตัว (Personal Identification Number: PIN) แบบออฟไลน์, การดึงข้อมูล, หลาย กะต่อวัน	ดำเนินงานส่วนใหญ่ผ่านการรวมโค้ด แล้ว ตัวงานหลักของลำดับเริ่มต้น เฟส 2 ปิดแล้ว 7 จาก 8 ตัว (เหลือ #231)
การปฏิบัติการ	นำระบบขึ้นใช้งาน (deploy) บน VM mob04 ด้วย self-hosted runner และ Ansible, Prometheus และ Grafana, etcd	deploy จริงครั้งแรกเมื่อ 30 กันยายน พ.ศ. 2569 การสาธิต ครบวงจรยังไม่ได้เริ่ม

1.3.2 ขอบเขตที่อยู่นอกโครงการ

- การย้ายร้านจริงไปใช้ระบบใหม่ (cutover) ถูกเลื่อนไปเฟสถัดไปตามการตัดสินใจของเจ้าของโครงการเมื่อวันที่ 15 กันยายน พ.ศ. 2569 (#231 ยังเปิดอยู่)
- ใบกำกับภาษีเต็มรูป ไม่อยู่ในขอบเขตของเวอร์ชันแรก
- ฮาร์ดแวร์เฉพาะทาง ได้แก่ เครื่องพิมพ์ใบเสร็จความร้อน ลินซ์เก็บเงิน และการสแกน บาร์โค้ด ซึ่งต้องทดสอบที่ร้านจริง ปัจจุบันการสแกนใช้การป้อนด้วยมือ

- การสำรองข้อมูลออกนอก VM (#363, #288) ถูกพักไว้จนกว่าจะเสร็จการสาธิตตามการตัดสินใจเมื่อวันที่ 22 กันยายน พ.ศ. 2569 ดังนั้นในขณะนี้ยังไม่มีข้อมูลสำรองที่ออกจาก VM
- การให้ร้านสมัครใช้เองแบบ self-service ไม่ทำในเฟส 1 ผู้ดูแลแพลตฟอร์มเป็นผู้สร้างร้าน (ADR-0001 [14])
- การย้ายขึ้นผู้ให้บริการคลาวด์ภายนอก ถูกปฏิเสธโดยเจ้าของโครงการเมื่อวันที่ 15 กันยายน พ.ศ. 2569

1.3.3 สิ่งที่ยังไม่ถือว่าแล้วเสร็จ

งานที่ยังเปิดอยู่ ได้แก่ การวัดภาระด้วย k6 (#380) การสาธิตครบวงจรบน mob04 (#344) และประเด็นการจัดการอุปกรณ์ (#476) ในเกณฑ์ตรวจรับของเฟส 1 (Definition of Done: DoD) 17 ข้อ ผ่านแล้ว 16 ข้อ เหลือข้อ k6 หนึ่งข้อ [7] รายละเอียดอยู่ในตารางที่ 4.8 ของบทที่ 4

1.4 แผนการดำเนินงาน

1.4.1 ลำดับเหตุการณ์ที่เกิดขึ้นจริง

ลำดับการดำเนินงานจริงสรุปจากประวัติ `git log` และบันทึกส่งมอบงานของโครงการ ปริมาณงานกระจุกตัวในเดือนกันยายน พ.ศ. 2569 กล่าวคือ จากคอมมิตทั้งหมด 1,058 รายการ มี 962 รายการเกิดในเดือนกันยายน เทียบกับ 18 รายการในเดือนมิถุนายน 17 รายการในเดือนกรกฎาคม 2 รายการในเดือนสิงหาคม และ 59 รายการในวันที่ 1 ตุลาคม ช่วงแรก (มิถุนายน–กรกฎาคม) เป็นการพัฒนาแอป Flutter ช่วงปลายเดือนสิงหาคมเป็นการจัดทำเอกสารออกแบบเซิร์ฟเวอร์ ส่วนเดือนกันยายนเป็นการพัฒนาเซิร์ฟเวอร์ CI/CD และกลไกเฟส 2 แบบคู่ขนานโดยสมาชิกสามคน เหตุการณ์สำคัญได้แก่ การบันทึก ADR-0001 ถึง 0009 และกำหนดให้สาขา `main` เป็นแนวพัฒนาหลายร้านค้า (4 กันยายน) การปิดงานเฟส 1 ส่วนใหญ่และเขียนสเปกเฟส 2 (15–17 กันยายน) และการ deploy ขึ้น VM mob04 ครั้งแรก (30 กันยายน)

1.4.2 ตารางแผนงานแบบแกนต์

ตารางที่ 1.3 แสดงงานตามช่วงเดือนของปีการศึกษา สัญลักษณ์ ■ หมายถึงช่วงที่ดำเนินการแล้วตามหลักฐาน `git log` ส่วน □ หมายถึงแผนในอนาคต ซึ่งเป็นแผนที่คณะผู้จัดทำเสนอให้อาจารย์ที่ปรึกษาพิจารณา แผนของเฟส 1 จัดลำดับตามความพึงพาของงาน [7]

ตารางที่ 1.3 แผนการดำเนินงานของโครงการ (■ = ดำเนินการแล้ว, □ = แผนในอนาคต)

งาน	มิ.ย. 69	ก.ค. 69	ส.ค. 69	ก.ย. 69	ต.ค. 69	พ.ย. 69	ธ.ค. 69	ม.ค. 70	ก.พ. 70	มี.ค. 70
1. พอร์ตแอป Flutter offline-first (ขั้น 0-6) และปรับหน้าจอ	■	■								
2. ปรับปรุงโค้ดและย้ายเป็น flutter_bloc	■	■								
3. ออกแบบ backend และบันทึก ADR			■	■						
4. เฟส 1: เซิร์ฟเวอร์ ฐานข้อมูล RLS ธุรกรรม				■						
5. ขึ้นเขียนข้อมูลผ่าน API ของ Flutter				■						
6. CI/CD ระดับ 1-3				■						
7. เฟส 2: outbox, SyncService, PWA, เลขเอกสาร				■	□					
8. deploy ขึ้น VM mob04 และการเฝ้าสังเกตระบบ				■	□					
9. สาธิตครบวงจร (#344)					□					
10. วัดภาระ k6 สามเครื่อง (#380)					□	□				
11. สำรองข้อมูลออกนอก VM (#363, #288)						□	□			
12. นำร่องกับร้านจริงและ cutover (#231)							□	□		
13. ฮาร์ดแวร์เฉพาะร้าน ทดสอบบน Android/iOS และเอกสารภาษี								□	□	
14. ทดสอบ ปรับปรุง และจัดทำรายงานฉบับสมบูรณ์									□	□

แถวที่ 7 และ 8 มีทั้ง ■ และ □ เพราะงานเริ่มแล้วแต่ยังไม่ปิด แถวที่ 9-11 เรียงตามที่เจ้าของโครงการกำหนดว่าการสาธิตมาก่อน และงานสำรองข้อมูลออกนอก VM ต้องรอหลังการสาธิต [15] ส่วนแถว 12-14 ยังไม่ระบุวันที่แน่นอนเพราะต้องรอผลการสาธิตและการเข้าถึงร้านจริง

1.4.3 โครงสร้างทีมและความรับผิดชอบ

ทีมประกอบด้วยสมาชิกสามคน ตามกติกาของรายวิชาที่ประกาศเมื่อวันที่ 5 กันยายน พ.ศ. 2569 สมาชิกทุกคนต้องมีส่วนทั้งในส่วน frontend, backend และ CI/CD โดยแต่ละคนรับผิดชอบเลน (lane) หนึ่งเลนเป็นหลัก ดังตารางที่ 1.4 ในเฟส 2 เจ้าของโครงการอนุมัติการแบ่งใหม่เมื่อวันที่ 16 กันยายน พ.ศ. 2569 เป็น 35 ตัวงาน (issue บน GitHub) แบ่งเป็นเลน A 3 ตัว เลน B 15 ตัว และเลน C 17 ตัว [15] หลักการแบ่งเลนอธิบายไว้ในหัวข้อวิธีการทำงานของทีมในบทที่ 3 ตารางที่ 1.4 แสดงการแบ่งงานตามแผน ส่วนผลงานจริงของสมาชิกแต่ละคนนับจาก PR ที่รวมแล้วในตารางที่ 4.10 ของบทที่ 4

ตารางที่ 1.4 ความรับผิดชอบของสมาชิกแต่ละเลนตามแผน

เลน	สมาชิก (GitHub)	ความรับผิดชอบในเฟส 1	ความรับผิดชอบในเฟส 2
A (team/1)	Chavatik Thorarit (NuimanLP)	เส้นทางเงิน: Docker Compose, idempotency, เลขเอกสาร, การขาย การคืน และการยกเลิกบิล, เครดิตช่วง, กะและ รายงานปิดร้าน; CI: สร้าง อิมเมจ (#40); ฝั่งแอป: หน้าขาย คืน ลีนซัก (เขียนผ่าน API)	ข้อตกลงระหว่างสองฝั่ง (SyncFacade และไฟล์ ตัวอย่าง /sync/ push), ข้อความ ภาษาไทยของเฟส 2, รายการอนุญาตผู้ดูแล แพลตฟอร์ม (3 ตัว)
B (team/2)	LomerAlloys	สคีมาและ migration, การสร้างร้านค้า, สินค้า ลูกค้าและช่วง, ใบสั่งซื้อ, ใบเสนอราคา, รายงานฝั่ง เซิร์ฟเวอร์; CI: path filter (#39); ฝั่งแอป: หน้าอ่านข้อมูลและ Drift schema v3	เครื่องยนต์ฝั่งเครื่อง ทั้งหมด: PWA, outbox, SyncService, เลข RC/CN, PIN ออฟไลน์, การดึงข้อมูล และการ เปลี่ยนสคีมา Drift (15 ตัว)
C (team/3)	Pattarapon Kitcharoen (PattaraponKitcharoen)	ยืนยันตัวตนด้วย JSON Web Token (JWT), บทบาทอุปกรณ์, แคช Redis, การจำกัดอัตรา, คิว BullMQ, การส่งออก ข้อมูลร้าน, k6, CI ฝั่ง เซิร์ฟเวอร์และด้านความปลอดภัย; ฝั่งแอป: เข้าสู่ ระบบและ device token	เซิร์ฟเวอร์ หน้าจอใหม่ที่ไม่ผูกกับเครื่องยนต์ซิงก์ และงานปฏิบัติการ ได้แก่ VM, runner, การเฝ้าสังเกตระบบ (17 ตัว)

1.5 ประโยชน์ที่คาดว่าจะได้รับ

1. **ร้านค้า** ได้ระบบที่ตัดสต็อกจริง คิดต้นทุนเฉลี่ยถ่วงน้ำหนักเมื่อรับสินค้าแต่ละล็อต คุมเครดิตช่วง และออกรายงานยอดขายและกำไร ซึ่งออกแบบเพื่อตอบปัญหา P1-P8 (ยังไม่ได้ประเมินผลกับผู้ใช้จริง ดูตารางที่ 4.2 ในบทที่ 4) โดยยังขายได้เมื่ออินเทอร์เน็ตขัดข้อง
2. **เจ้าของร้านหลายราย** ได้ข้อมูลที่แยกขาดจากกันด้วย RLS ข้อมูลอยู่ที่ศูนย์กลางและสำรองรายวันบนเครื่องเซิร์ฟเวอร์ (การสำรองออกนอกเครื่องยังเป็นงานค้าง #363)
3. **คณะผู้จัดทำ** ได้ประสบการณ์การออกแบบระบบกระจายตัวที่มีธุรกรรมเกี่ยวกับเงิน การจัดการสิทธิ์หลายผู้เช่า การซิงค์ข้อมูลแบบออฟไลน์ และการสร้างไปป์ไลน์ CI/CD ครบวงจร
4. **ผู้ที่สนใจต่อยอด** ได้ชุดเอกสารและบันทึกการตัดสินใจ (ADR) ที่ตามย้อนเหตุผลของการออกแบบได้ และได้ชุดทดสอบที่ใช้เป็นหลักฐานของพฤติกรรมของระบบ

1.6 ลำดับการเรียบเรียงรายงาน

บทที่ 1 อธิบายที่มา วัตถุประสงค์ ขอบเขต และแผนการดำเนินงาน บทที่ 2 อธิบายทฤษฎีและเทคโนโลยีที่เกี่ยวข้อง บทที่ 3 อธิบายการออกแบบและวิธีสร้างระบบ บทที่ 4 รายงานผลที่ได้พร้อมหลักฐาน และบทที่ 5 สรุปผลการดำเนินงาน ปัญหา และงานต่อไป ทุกตัวเลขและสถานะงานในรายงานตรวจสอบกับที่เก็บโค้ด ประวัติ [git](#) และบันทึกส่งมอบงานของโครงการ หากรายการใดยังไม่แล้วเสร็จจะระบุไว้อย่างชัดเจน

บทที่ 2

ทฤษฎีและความรู้พื้นฐาน

2.1 บทนำ

บทนี้รวบรวมทฤษฎี แนวคิด และเทคโนโลยีที่โครงการอาศัยเป็นฐาน โดยจัดลำดับจากโดเมนธุรกิจ (ระบบขายหน้าร้าน) ไปสู่สถาปัตยกรรมฝั่งไคลเอนต์ (offline-first, Flutter, Drift) ต่อด้วยฝั่งเซิร์ฟเวอร์ (ธุรกรรม การแยกข้อมูลผู้เช่า การยืนยันตัวตน) และปิดท้ายด้วยโครงสร้างพื้นฐานและกระบวนการส่งมอบ บทนี้ไม่ทบทวนเทคโนโลยีทั้งหมดอย่างครอบคลุม แต่เลือกเฉพาะแนวคิดที่มีผลต่อการตัดสินใจเชิงออกแบบ ส่วนวิธีที่โครงการนำแนวคิดเหล่านี้ไปสร้างจริงอธิบายไว้ในบทที่ 3

2.2 แนวคิดของระบบขายหน้าร้านและโดเมนอะไหล่รถยนต์

ระบบขายหน้าร้าน (Point of Sale: POS) คือระบบที่บันทึกการขายสินค้า ณ จุดชำระเงิน พร้อมปรับปรุงสต็อก ยอดลูกค้า และสมุดบัญชีเงินสดของร้านให้สอดคล้องกัน ร้านอะไหล่รถยนต์มีลักษณะเฉพาะ เช่น สินค้าจำนวนมากที่ค้นด้วยเลขอะไหล่ (part number) ลูกค้าประจำที่เป็นช่างซึ่งซื้อเป็นเครดิต และการเสนอราคาก่อนขาย แนวคิดหลักที่ระบบต้องรักษาให้ถูกต้องมีดังนี้

2.2.1 การขายและค่าคงที่ของสต็อก

การขายหนึ่งครั้งประกอบด้วยหวั้บิลและรายการสินค้า และต้องทำหลายอย่างพร้อมกันหรือไม่ทำเลย ได้แก่ ตรวจสอบว่าสต็อกพอ ตัดสต็อก บันทึกบิลและเลขที่ใบเสร็จ สะสมคะแนนสะสม (แต้ม) ของลูกค้า และปรับยอดของช่างเมื่อชำระด้วยเครดิต ค่าคงที่ (invariant) ที่สำคัญที่สุดคือสต็อกห้ามติดลบจากการขาย หากสต็อกไม่พอต้องปฏิเสธบิลทั้งใบ ลักษณะ "ทำทั้งหมดหรือไม่ทำเลย" นี้ตรงกับคุณสมบัติความเป็นอะตอม (atomicity) ของธุรกรรมฐานข้อมูลที่กล่าวในหัวข้อการควบคุมภาวะพร้อมกัน

2.2.2 การคืนสินค้าและใบลดหนี้

การคืนสินค้าสร้างเอกสารใบลดหนี้ (credit note: CN) ที่อ้างอิงบิลเดิม หลักสำคัญคือห้ามคืนเกินจำนวนที่ขายหักด้วยจำนวนที่เคยคืนไปแล้ว และต้องย้อนส่วนลด แต้ม และยอดสะสมของช่างตามสัดส่วนของสินค้าที่คืน

2.2.3 การรับสินค้าเข้าและต้นทุนถัวเฉลี่ยถ่วงน้ำหนัก

ใบสั่งซื้อ (Purchase Order: PO) เมื่อรับสินค้าจะเพิ่มสต็อกและปรับต้นทุนต่อหน่วยด้วยวิธีถัวเฉลี่ยถ่วงน้ำหนัก (weighted-average cost) ซึ่งคำนวณจากสูตร

ต้นทุนใหม่ = (จำนวนเดิม × ต้นทุนเดิม + จำนวนที่รับ × ต้นทุนที่รับ) / (จำนวนเดิม + จำนวนที่รับ)

วิธีนี้ทำให้ต้นทุนสะท้อนราคาที่ซื้อในหลายรอบโดยไม่ต้องติดตามล็อตของสินค้าที่ละชิ้น อย่างไรก็ตาม หากรายงานกำไรคำนวณจากต้นทุนปัจจุบัน กำไรของปีเก่าจะเปลี่ยนไปเมื่อต้นทุนเปลี่ยน ระบบจึงควรบันทึกต้นทุน ณ เวลาขาย (cost at sale) ไว้กับบรรทัดบิล

2.2.4 กะ ลีนชักเงินสด และเครดิตข้าง

กะ (shift) คือช่วงเวลาที่พนักงานรับผิดชอบลีนชักเงินสดชุดหนึ่ง เริ่มด้วยเงินสดตั้งต้น มีรายการนำเงินเข้า-ออก และปิดกะด้วยการนับเงินเทียบกับยอดที่ระบบคาดหวัง ซึ่งต้องคำนวณด้วยวิธีเดียวทุกจุด มิฉะนั้นหน้าจอสองแห่งจะแสดงยอดคาดหวังไม่ตรงกัน ส่วนเครดิตข้างคือบัญชีลูกหนี้ของข้างแต่ละราย ประกอบด้วยยอดค้าง วงเงิน และประวัติการชำระ การขายเกินวงเงินควรต้องได้รับการยืนยันจากผู้ใช้อย่างชัดเจน

2.2.5 เลขที่เอกสาร

เลขที่ใบเสร็จ (RC) และใบลดหนี้ (CN) ต้องไม่ซ้ำและเรียงต่อเนื่อง เพื่อให้ตรวจสอบทางบัญชีได้ เมื่อหลายเครื่องออกเลขได้ รูปแบบเลขต้องรวมหมายเลขเครื่องและช่วงเวลา เช่น **RC01-2569-08-0042** ตาม ADR-0007 ของโครงการ การให้เครื่องที่ออฟไลน์ออกเลขเองโดยไม่ชนกับเซิร์ฟเวอร์จึงเป็นโจทย์เชิงระบบกระจาย

2.2.6 เหตุที่ไม่ใช้ระบบ POS สำเร็จรูป

คณะผู้จัดทำยังไม่ได้สำรวจเปรียบเทียบผลิตภัณฑ์ POS สำเร็จรูปอย่างเป็นระบบ จึงไม่อาจคุณสมบัติของผลิตภัณฑ์รายใด เหตุผลที่พัฒนาเองมีสองข้อ ข้อแรก กติกาเฉพาะของร้าน เช่น เครดิตข้างพร้อมวงเงิน ราคาข้าง การย่อนส่วนลดและแต้มตามสัดส่วนเมื่อคืนสินค้า และข้อความภาษาไทยที่พนักงานคุ้นเคย อยู่ในไฟล์ **pos/db.js** ซึ่งโครงการต้องคงพฤติกรรมไว้ทุกข้อ ข้อที่สอง รายวิชากำหนดให้สร้างเซิร์ฟเวอร์หลายผู้เช่าด้วยชุดเทคโนโลยีที่กำหนด ซึ่งการซื้อระบบสำเร็จรูปไม่ตอบโจทย์นี้ การเปรียบเทียบกับผลิตภัณฑ์ในตลาดด้านการขายออฟไลน์ เครดิตข้าง ใบเสร็จภาษาไทย และค่าใช้จ่าย เป็นงานที่ควรเพิ่มในรายงานฉบับสมบูรณ์

2.3 สถาปัตยกรรมออฟไลน์เป็นหลัก (offline-first)

2.3.1 แนวคิด local-first และ offline-first

ระบบ offline-first ออกแบบให้แอปทำงานได้เต็มที่กับข้อมูลในเครื่องเป็นหลัก แล้วจึงโครโนซ์กับเซิร์ฟเวอร์เมื่อมีเครือข่าย ต่างจากแอปแบบ online-first ที่ทุกการกระทำต้องรอเซิร์ฟเวอร์ Kleppmann และคณะ [16] เสนอแนวคิด local-first software พร้อมอุดมคติเจ็ดข้อ เช่น การตอบสนองเร็วเพราะไม่ต้องรอเครือข่าย การทำงานได้ขณะออฟไลน์ และผู้ใช้เป็นเจ้าของข้อมูลของตน บทความชี้ว่าความท้าทายหลักคือการรวมการแก้ไขจากหลายอุปกรณ์ ซึ่งงานของคณะผู้เขียนแก้ด้วยโครงสร้างข้อมูลแบบ CRDT (Conflict-free Replicated Data Type) สำหรับร้านค้าปลีก เหตุผลเชิงปฏิบัติคืออินเทอร์เน็ตของร้านอาจขัดข้อง แต่แคชเตอร์ต้องขายต่อได้

2.3.2 เหตุที่ไม่ใช้ CRDT หรือการทำสำเนา (replicate) ฐานข้อมูลโดยตรง

CRDT เหมาะกับข้อมูลที่ผสมกันได้โดยไม่ขัดแย้ง เช่น เอกสารร่วมแก้ไข แต่ข้อมูลการเงินและสต็อกมีค่าคงที่ที่ต้องรักษา เช่น สต็อกห้ามติดลบและเลขที่ใบเสร็จห้ามซ้ำ การผสมอัตโนมัติอาจทำให้ขายเกินสต็อกโดยไม่มีใครถูกปฏิเสธ ADR-0012 ของโครงการพิจารณาข้อเสนอให้ใช้ CouchDB ที่ replicate ได้ในตัวแทน PostgreSQL แล้วมีมติไม่ใช้ เพราะการสลับฐานข้อมูลเพียงอย่างเดียวทำให้เสียธุรกรรมข้ามตาราง การล็อกแถว คีย์นอก และ RLS โดยไม่ได้สั่งทดแทน และการป้องกันการขายเกินสต็อกที่แท้จริงมาจากการจำกัดให้มีเครื่องที่เขียนออฟไลน์ได้ (writer) เพียงเครื่องเดียวต่อร้าน ไม่ใช่จากชนิดของฐานข้อมูล ผลลัพธ์คือสถาปัตยกรรมที่เซิร์ฟเวอร์เป็นแหล่งความจริง ส่วนฐานข้อมูลในเครื่องเป็นแคชและเปลือกลำหรับใช้งานออฟไลน์ (offline shell) ที่บันทึกคำสั่งไว้ส่งทีหลัง

2.3.3 รูปแบบกล่องขาออก (outbox pattern)

เมื่อแอปเขียนข้อมูลขณะออฟไลน์ ต้องบันทึกทั้งผลลัพธ์ในเครื่องและ "ความตั้งใจที่จะส่งเซิร์ฟเวอร์" ในธุรกรรมเดียวกันของฐานข้อมูลในเครื่อง แล้วให้ตัวส่งอ่านคิวนั้นไปส่งภายหลัง รูปแบบนี้เรียกว่า outbox pattern ข้อสำคัญคือการบันทึกผลกับการบันทึกคำสั่งในคิวต้องอะตอมิกด้วยกัน มิฉะนั้นอาจเกิดกรณีที่สต็อกถูกตัดแล้วแต่ไม่มีคำสั่งให้ส่ง หรือมีคำสั่งแต่ผลในเครื่องไม่ตรงกัน แต่ละแถวของคิวมีสถานะ เช่น รอส่ง ติดขัด หรือถูกปฏิเสธ และฝั่งรับต้องรองรับการได้รับคำสั่งเดิมซ้ำ ซึ่งเชื่อมโยงกับแนวคิด idempotency ในหัวข้อถัดไป

2.3.4 ความสอดคล้องในที่สุด

ระบบกระจายที่ยอมให้สำเนาต่างกันชั่วคราวเรียกว่ามีความสอดคล้องในที่สุด (eventual consistency) คือหากไม่มีการเขียนใหม่ สำเนาทั้งหมดจะลู่เข้าสู่ค่าเดียวกัน [17] ในระบบแบบนี้ ยอดสต็อกและยอดลูกค้าในเครื่องขณะออฟไลน์เป็นค่าประมาณจากข้อมูลล่าสุดที่ดึงมาบวกด้วยผลของคำสั่งที่ค้างในคิว เมื่อซิงค์สำเร็จจึงถูกแทนด้วยค่าจากเซิร์ฟเวอร์ ข้อควรระวังคือการดึงข้อมูลจากเซิร์ฟเวอร์ต้องไม่ทับค่าในเครื่องที่ยังมีคำสั่งค้างอ้างอิงอยู่ มิฉะนั้นยอดจะกระโดดกลับก่อนคำสั่งถูกส่ง

2.3.5 ความล้มเหลวระหว่างทางและผลลัพธ์ที่ไม่ทราบแน่ชัด

การเรียกผ่านเครือข่ายล้มได้สามรูปแบบที่ต่างกัน คือ ส่งไม่ถึง ส่งถึงแต่คำตอบไม่กลับ และเซิร์ฟเวอร์ตอบข้อผิดพลาด รูปแบบที่สองอันตรายที่สุด เพราะผู้ส่งไม่ทราบว่าการเขียนสำเร็จหรือไม่ หากผู้ส่งตีความว่าล้มเหลวแล้วเขียนซ้ำหรือเขียนลงเครื่องแทน อาจติดสต็อกสองครั้ง ไคลเอนต์จึงต้องแยกคำตอบที่เป็น "คำตอบสิ้น" ของเซิร์ฟเวอร์ (เช่น รหัส 4xx ที่บอกว่าคำขอผิด) ออกจากคำตอบที่บอกเพียงว่าเซิร์ฟเวอร์มีปัญหา (5xx) หรือไม่มีคำตอบเลย

2.4 เทคโนโลยีฝั่งไคลเอนต์

2.4.1 Flutter และ Dart

Flutter เป็นชุดเครื่องมือสร้างส่วนติดต่อผู้ใช้ของ Google ที่คอมไพล์รหัสเดียวไปยังแอป Android, iOS และเว็บ [4] เขียนด้วยภาษา Dart ซึ่งเป็นภาษาแบบมีชนิดข้อมูล รองรับการทำงานแบบอะซิงโครนัสด้วย **Future** และ **Stream** ส่วนติดต่อผู้ใช้ประกอบจากวิดเจ็ต (widget) ที่ไม่เปลี่ยนค่า และเฟรมเวิร์กสร้างต้นไม้วิดเจ็ตใหม่เมื่อสถานะเปลี่ยน โครงงานเลือก Flutter เพราะต้องการฐานรหัสเดียวสำหรับแคนเดอไรต์ที่อาจเป็นแท็บเล็ตหรือเว็บเบราว์เซอร์

2.4.2 SQLite, Drift และการรันบนเว็บ

SQLite เป็นฐานข้อมูลเชิงสัมพันธ์แบบฝังตัวในไฟล์เดียว รองรับธุรกรรมแบบ ACID (ความเป็นอะตอม ความสอดคล้อง การแยกจากกัน และความคงทน ดูหัวข้อคุณสมบัติ ACID ในบทนี้) [6] Drift เป็นไลบรารีแมปวัตถุกับฐานข้อมูล (Object-Relational Mapping: ORM) แบบมีชนิดข้อมูลสำหรับ Dart ที่สร้างรหัสจากการประกาศตาราง และเรียกใช้ SQLite ได้ทั้งบนอุปกรณ์เคลื่อนที่และเว็บ [5] บนเว็บ Drift ใช้ SQLite ที่คอมไพล์เป็น WebAssembly ทำงานร่วมกับ web worker และเลือกที่เก็บตามความสามารถของเบราว์เซอร์ เช่น Origin Private File System (OPFS) หรือ IndexedDB

2.4.3 การจัดการสถานะด้วย BLoC และการนำทางด้วย go_router

การจัดการสถานะ (state management) คือการกำหนดว่าข้อมูลที่หน้าจอใช้เก็บที่ใดและเปลี่ยนแปลงอย่างไร รูปแบบ BLoC (Business Logic Component) แยกตรรกะธุรกิจออกจากวิดเจ็ต โดยรับเหตุการณ์และปล่อยสถานะเป็นสตรีม Cubit เป็นรูปแบบย่อยที่เรียกเมธอดแล้วปล่อยสถานะโดยตรง ไลบรารี **flutter_bloc** มี **BlocProvider** และ **RepositoryProvider** สำหรับฉีดการพึ่งพา (dependency injection) ลงต้นไม้วิดเจ็ต [18] การฉีด repository ผ่าน provider ทำให้สลับ implementation ได้โดยหน้าจอไม่ต้องแก้ ส่วน **go_router** เป็นไลบรารีการนำทางแบบประกาศ (declarative routing) ที่แมป URL ไปยังหน้าจอ และมี **ShellRoute** ที่ห่อหน้าจอย่อยด้วยโครงสร้างร่วม [19]

2.5 การสื่อสารไคลเอนต์-เซิร์ฟเวอร์

REST (Representational State Transfer) เป็นรูปแบบสถาปัตยกรรมที่ใช้ทรัพยากรที่ระบุด้วย URL และเมธอด HTTP เช่น GET, POST, PATCH ตามความหมายของแต่ละเมธอด โดยเซิร์ฟเวอร์ไม่เก็บสถานะของเซสชันระหว่างคำขอ ทำให้เพิ่มจำนวนอินสแตนซ์ของเซิร์ฟเวอร์ได้ตรงไปตรงมา การห่อผลลัพธ์ด้วยโครงสร้างคงที่ (envelope) ที่มีรหัสข้อผิดพลาดแบบข้อความ ช่วยให้ไคลเอนต์ตัดสินใจว่าข้อผิดพลาดใดเป็นคำตัดสินและแปลงเป็นข้อความสำหรับผู้ใช้ได้ ส่วนจำนวนเงินควรส่งเป็นสตริงทศนิยมเพื่อเลี่ยงความคลาดเคลื่อนของเลขทศนิยมแบบลอยตัว

แคชแบบเขียนผ่าน (write-through cache) คือรูปแบบที่การเขียนทุกครั้งไปยังแหล่งความจริงก่อน แล้วจึงนำผลที่แหล่งความจริงตอบกลับมาปรับแคช แคชจึงไม่มีข้อมูลที่แหล่งความจริงไม่ได้ยอมรับ รูปแบบนี้คือแนวทางที่ ADR-0010 ของโครงการใช้กับฐานข้อมูล Drift ในเครื่อง

2.6 ธุรกรรมฐานข้อมูลและการควบคุมภาวะพร้อมกัน

2.6.1 คุณสมบัติ ACID และระดับการแยกจากกัน

ธุรกรรม (transaction) คือหน่วยงานที่ฐานข้อมูลรับประกันคุณสมบัติ ACID ได้แก่ ความเป็นอะตอม (Atomicity) คือทำครบหรือไม่ทำเลย ความสอดคล้อง (Consistency) คือข้อจำกัดของข้อมูลยังคงอยู่หลังธุรกรรม การแยกจากกัน (Isolation) คือธุรกรรมที่ทำพร้อมกันไม่เห็นผลครึ่งทางของกันและกัน และความคงทน (Durability) คือเมื่อยืนยัน (commit) แล้วข้อมูลไม่สูญหาย [20] มาตรฐาน SQL นิยามระดับการแยกจากกันตามความผิดปกติที่ยอมให้เกิด เช่น dirty read, non-repeatable read และ phantom read PostgreSQL ใช้ระดับ Read Committed เป็นค่าเริ่มต้น ซึ่งแต่ละคำสั่งเห็นข้อมูลที่ยืนยันแล้วในขณะเริ่มคำสั่งนั้น [20] ในระดับนี้ การอ่านค่าแล้วคำนวณแล้วเขียนกลับ

(read-modify-write) เช่นอ่านสต็อกแล้วลบแล้วเขียน อาจเสียการปรับปรุงของธุรกรรมอื่นที่แทรกกลาง (lost update) เว้นแต่จะล็อกแถวไว้ก่อน

2.6.2 การล็อกระดับแถว เดดล็อก และลำดับการล็อก

PostgreSQL ใช้ MVCC (Multi-Version Concurrency Control) ทำให้ผู้อ่านไม่บล็อกผู้เขียน แต่ผู้เขียนที่แก้ไขแถวเดียวกันต้องรอกัน เมื่อต้องการกันธุรกรรมอื่นแก้ไขระหว่างที่ตัวเองตรวจเงื่อนไขอยู่ ใช้ **SELECT ... FOR UPDATE** เพื่อล็อกแถวแบบเขียน หรือ **FOR SHARE** เพื่อล็อกแบบแบ่งปัน [20] เดดล็อก (deadlock) เกิดเมื่อธุรกรรมสองรายการถือล็อกที่อีกฝ่ายต้องการและรอกันเป็นวงกลม PostgreSQL ตรวจพบแล้วยกเลิกธุรกรรมหนึ่งรายการ [20] วิธีป้องกันที่เชื่อถือได้คือกำหนดลำดับการล็อกให้ตรงกันทุกเส้นทางโค้ด ถ้าทุกธุรกรรมล็อกทรัพยากรตามลำดับเดียวกัน วงกลมจะไม่เกิด

2.6.3 ธุรกรรมกับกลุ่มการเชื่อมต่อ

การเชื่อมต่อฐานข้อมูลเป็นทรัพยากรจำกัดที่จัดสรรผ่านกลุ่มการเชื่อมต่อ (connection pool) หากคำขอหนึ่งถือการเชื่อมต่อแรกไว้แล้วขอการเชื่อมต่อที่สองจาก pool เดียวกัน เมื่อจำนวนคำขอพร้อมกันเท่ากับขนาด pool ทุกคำขอจะรอกันเองโดยไม่มีใครได้ไปต่อ (pool starvation) นอกจากนี้ธุรกรรมที่ยาวเกินจำเป็นจะถือทั้งการเชื่อมต่อและล็อกไว้นาน งานที่ซ้ำแต่ไม่ต้องอยู่ในธุรกรรม เช่น การแฮชรหัสผ่าน จึงควรทำนอกธุรกรรม

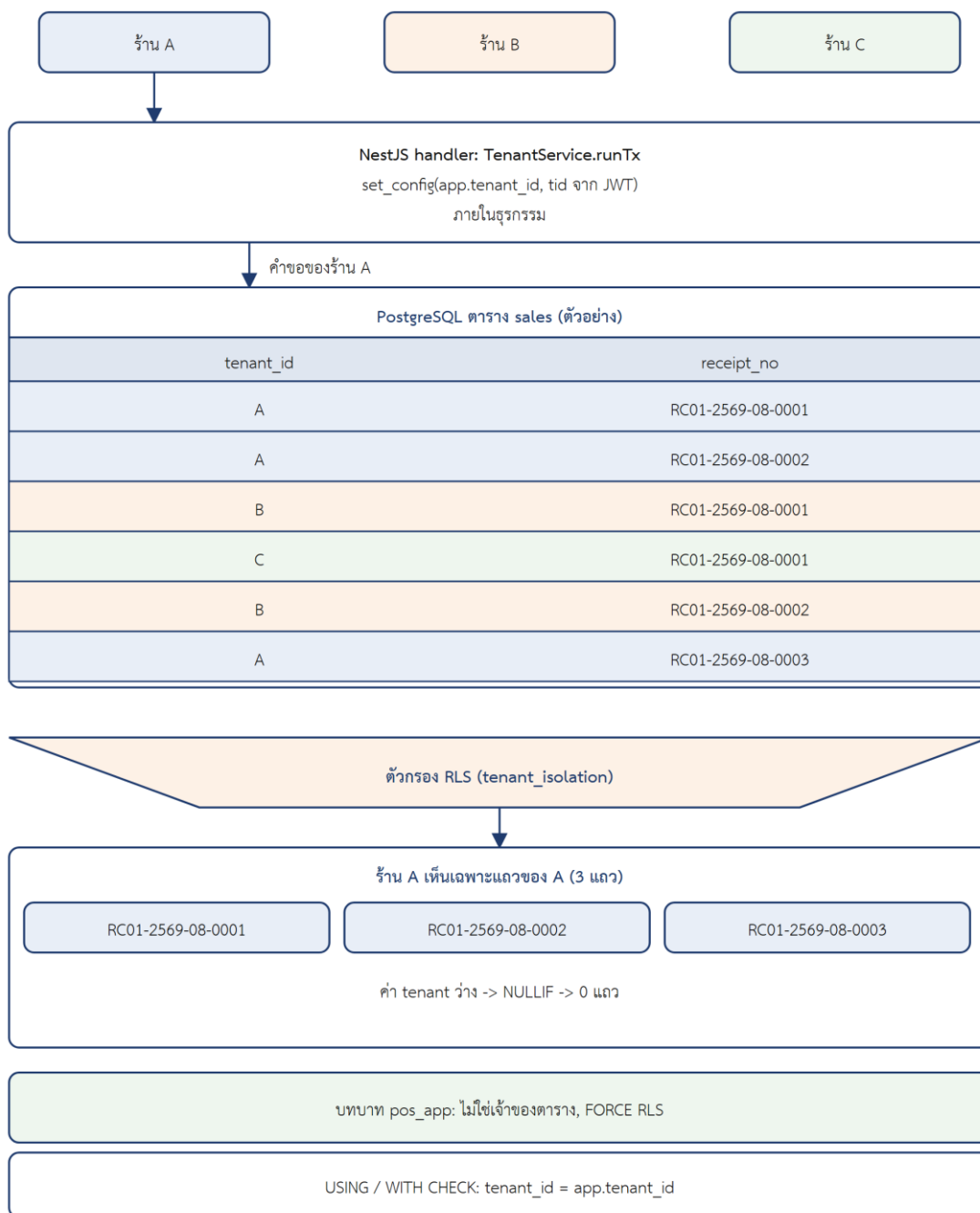
2.7 การแยกข้อมูลผู้เช่าหลายราย (multi-tenancy)

ระบบหลายผู้เช่า (multi-tenant) ให้องค์กรหลายราย (ผู้เช่า) ใช้ระบบเดียวกันโดยข้อมูลต้องไม่ปะปนกัน มีแนวทางหลักสามแบบที่ต่างกันที่ระดับการแยกและต้นทุนการดูแล ดังตารางที่ 2.1

ตารางที่ 2.1 เปรียบเทียบแบบจำลองการแยกผู้เช่าในฐานข้อมูล

แบบจำลอง	การแยกข้อมูล	ต้นทุนการดูแล	ข้อสังเกตต่อโครงการ
ฐานข้อมูลแยกต่อผู้เช่า	แยกสูงสุดระดับฐานข้อมูล	สูง: ต้องจัดการหลายฐานข้อมูล ย้าย schema หลายชุด	ไม่เหมาะกับทรัพยากรของเครื่องสาคิตเครื่องเดียว
Schema แยกต่อผู้เช่า	แยกระดับ schema ในฐานข้อมูลเดียว	ปานกลาง: migration ต้องทำซ้ำทุก schema	จำนวนออบเจ็กต์โตตามจำนวนผู้เช่า
ตารางร่วมกับคอลัมน์ tenant_id	แยกระดับแถว ต้องบังคับด้วยกติกา	ต่ำ: schema เดียว migration ครั้งเดียว	โครงการเลือกแบบนี้ร่วมกับ Row-Level Security

ความเสี่ยงของแบบที่สามคือ หากโค้ดลืมใส่เงื่อนไข **tenant_id** ในคำสั่งใดคำสั่งหนึ่ง ข้อมูลของผู้เช่าอื่นจะรั่วไหล จึงต้องมีกลไกที่ไม่พึ่งความระมัดระวังของนักพัฒนา PostgreSQL มีคุณสมบัติ Row-Level Security (RLS) ที่ให้กำหนดนโยบาย (policy) ต่อตาราง ซึ่งฐานข้อมูลจะเติมเป็นเงื่อนไขกรองแถวให้ทุกคำสั่งโดยอัตโนมัติ [10] นโยบายมีสองส่วน คือ **USING** ที่กรองแถวที่มองเห็นหรือแก้ไขได้ และ **WITH CHECK** ที่ตรวจแถวที่กำลังจะเขียน โดยทั่วไปนโยบายเทียบ **tenant_id** กับตัวแปรเซสชันที่แอปตั้งไว้ในธุรกรรม ดังรูปที่ 2.1 ข้อควรระวังคือเจ้าของตารางและผู้ใช้ระดับ superuser ได้รับการยกเว้นจาก RLS โดยปริยาย แอปจึงต้องเชื่อมต่อด้วยบทบาทที่ไม่ใช่เจ้าของตาราง หรือบังคับด้วย **FORCE ROW LEVEL SECURITY**



รูปที่ 2.1 แนวคิด Row-Level Security: คำขอของผู้เช่าหนึ่งเห็นเฉพาะแถวของผู้เช่านั้น

ผู้ดูแลแพลตฟอร์มที่สร้างและดูแลผู้เช่าหลายรายต้องข้ามขอบเขตผู้เช่าได้ แนวทางที่ปลอดภัยคือแยก "ระนาบผู้ดูแล" (admin plane) ออกจากระนาบผู้เช่าทั้งในระดับบัญชี โทเค็น และเส้นทางเครือข่าย เพื่อไม่ให้ความผิดพลาดของฝั่งหนึ่งเปิดทางเข้าอีกฝั่ง

2.8 การทำซ้ำคำขออย่างปลอดภัยและรหัสกำกับกับการเขียน (idempotency)

การดำเนินการเป็น idempotent เมื่อทำซ้ำกี่ครั้งก็ให้ผลต่อสถานะเท่ากับทำครั้งเดียว ใน HTTP เมธอด GET, HEAD, PUT, DELETE ถูกนิยามให้ idempotent แต่ POST ซึ่งใช้สร้างบิลไม่เป็น [21] เมื่อไคลเอนต์ส่ง POST แล้วเครือข่ายหมดเวลา ไคลเอนต์ไม่ทราบว่าเซิร์ฟเวอร์ทำสำเร็จหรือไม่ การส่งซ้ำอาจทำให้ขายซ้ำสองบิล แนวทางที่กำลังถูกกำหนดเป็นมาตรฐานคือให้ไคลเอนต์สร้างรหัสสุ่มหนึ่งค่าต่อหนึ่งเจตนา แนบในส่วนหัว **Idempotency-Key** [22] เซิร์ฟเวอร์บันทึกคีย์พร้อมค่าสรุปของคำขอ (request fingerprint) และผลลัพธ์ หากได้รับรหัสเดิมกับคำขอเดิมจะตอบผลเดิมโดยไม่ทำซ้ำ หากรหัสเดิมมากับคำขอที่ต่างไปจะปฏิเสธ กลไกนี้ได้ผลก็ต่อเมื่อไคลเอนต์ใช้รหัสเดิมทุกครั้งทีลองส่งเจตนาเดิมซ้ำ และการจัดรหัสอยู่ในธุรกรรมเดียวกับการเขียน

2.9 การยืนยันตัวตนและการรักษาความปลอดภัย

2.9.1 JSON Web Token และอายุของโทเค็น

JWT ตามมาตรฐาน RFC 7519 [23] เป็นรูปแบบโทเค็นแบบกะทัดรัดที่ประกอบด้วยส่วนหัว เพย์โหลดที่มี claim เช่น **iss**, **sub**, **exp**, **aud** และลายเซ็น ผู้รับตรวจสอบลายเซ็นด้วยกุญแจโดยไม่ต้องถามฐานข้อมูล การเซ็นด้วยกุญแจสมมาตร เช่น RS256 ทำให้ผู้ตรวจถือเพียงกุญแจสาธารณะ ระบบทั่วไปแยกโทเค็นเข้าถึง (access token) อายุสั้นเพื่อจำกัดความเสียหายเมื่อรั่ว ออกจากโทเค็นต่ออายุ (refresh token) ที่อายุยาวกว่าและใช้ขอ access token ใหม่ การเพิกถอนทำได้ที่จังหวัดต่ออายุ ซึ่งตรวจสอบฐานข้อมูลอีกครั้ง บนเว็บ โทเค็นที่เก็บใน **localStorage** อ่านได้จากสคริปต์ใด ๆ ในหน้า จึงเสี่ยงต่อการโจมตีแบบ Cross-Site Scripting (XSS)

2.9.2 การแฮชรหัสผ่านด้วย Argon2

รหัสผ่านต้องไม่เก็บเป็นข้อความตรง แต่เก็บค่าแฮชจากฟังก์ชันที่ออกแบบให้ช้าและใช้หน่วยความจำมาก เพื่อต้านการเดาแบบเหมารวม Argon2 ผู้ชนะการแข่งขัน Password Hashing Competition ถูกกำหนดใน RFC 9106 [24] มีพารามิเตอร์จำนวนหน่วยความจำ จำนวนรอบ และระดับขนาน ข้อควรระวังสำหรับข้อความภาษาไทยคือสตริงเดียวกันอาจเข้ารหัสยูนิโคดได้หลายรูป การทำให้เป็นรูปมาตรฐาน (normalization) ก่อนแฮชจึงมีผลต่อการตรวจรหัสผ่าน

2.9.3 การผูกอุปกรณ์และการจำกัดอัตราคำขอ

ระบบที่ต้องจำกัดสิทธิ์ตามเครื่อง เช่น ให้มีเครื่องขายเพียงเครื่องเดียวต่อร้าน มักใช้รหัสลงทะเบียนแบบใช้ครั้งเดียวที่ผู้มีสิทธิ์ออกให้ แล้วนำไปแลกเป็นโทเค็นของอุปกรณ์ ทำให้ตัวตนของ

เครื่องมาจากเซิร์ฟเวอร์ ไม่ใช่จากค่าที่ไคลเอนต์ส่งเอง ส่วนการจำกัดอัตราค่าขอ (rate limiting) ช่วยลดการเดารหัสผ่านและการใช้ทรัพยากรเกินควร มักทำหลายชั้น เช่น ต่อ IP ที่พรีอ็อกซีหน้าระบบ และต่อผู้เช่าในแอปพลิเคชัน

2.10 เทคโนโลยีฝั่งเซิร์ฟเวอร์

NestJS เป็นเฟรมเวิร์กฝั่งเซิร์ฟเวอร์บน Node.js ที่เขียนด้วย TypeScript จัดโครงสร้างเป็นโมดูล คอนโทรลเลอร์ และผู้ให้บริการ (provider) พร้อมระบบฉีดการพึ่งพา (dependency injection) และกลไก guard, interceptor และ middleware [25] โครงสร้างที่บังคับให้แยกชั้นชัดเจนเหมาะกับทีมสามคนที่แบ่งงานตามโดเมน PostgreSQL เป็นระบบจัดการฐานข้อมูลเชิงสัมพันธ์ โอเพนซอร์สที่รองรับ ACID, MVCC, RLS, ข้อจำกัดเชิงซ้อน และดัชนีบางส่วน [20]

Redis เป็นที่เก็บข้อมูลแบบคีย์-ค่า (key-value store) ในหน่วยความจำ ใช้เป็นแคช ตัวนับ และคิว [26] BullMQ เป็นไลบรารีคิวงานบน Redis ที่มีการลองช้าและตารางเวลา [27] เหมาะกับงานเบื้องหลังที่ไม่ควรทำในคำขอ HTTP เช่น การส่งออกข้อมูล ข้อควรสังเกตคือแคชกับคิวต้องการนโยบายหน่วยความจำที่ตรงข้ามกัน แคชทิ้งข้อมูลได้เมื่อเต็ม แต่คิวห้ามทิ้งงานที่ยังไม่ทำ

พรีอ็อกซีย้อนกลับ (reverse proxy) รับคำขอจากภายนอกแล้วส่งต่อไปยังเซิร์ฟเวอร์ภายใน ทำหน้าที่ยุติ Transport Layer Security (TLS) จำกัดอัตรา และซ่อนโครงสร้างภายใน Nginx ทำหน้าที่นี้ได้ด้วยคำสั่ง `proxy_pass` และ `limit_req_zone` [28] เมื่อมีพรีอ็อกซี แอปจะอ่าน IP ผู้เรียกจากส่วนหัว `X-Forwarded-For` จำนวนชั้นของพรีอ็อกซีจึงต้องแน่นอน หากมีพรีอ็อกซีหรือ Content Delivery Network (CDN) ซ่อนเพิ่ม ผู้ใช้ทุกคนอาจถูกมองเป็น IP เดียว

2.11 คอนเทนเนอร์และการจัดวาง

คอนเทนเนอร์เป็นการแยกโปรเซสด้วยพีเจอรของเคอร์เนล (namespace และ cgroup) ที่บรรจุแอปพร้อมไลบรารีที่ต้องใช้เป็นอิมเมจ ทำให้รันได้เหมือนกันทุกเครื่อง [29] Docker Compose ประกาศชุดบริการหลายตัว เครือข่าย และเพดานทรัพยากรในไฟล์เดียว [30] การตรึงอิมเมจด้วย digest แทนป้ายชื่อทำให้สร้างสภาพแวดล้อมเดิมซ้ำได้ เพราะป้ายชื่ออย่าง `latest` อาจชี้ไปยังอิมเมจคนละตัวในเวลาต่างกัน

2.12 การบูรณาการและการส่งมอบอย่างต่อเนื่อง (CI/CD)

การบูรณาการต่อเนื่อง (Continuous Integration: CI) คือการรวมโค้ดเข้าสาขาหลักบ่อยครั้งพร้อมสร้างและทดสอบอัตโนมัติ เพื่อพบข้อผิดพลาดแต่เนิ่น การส่งมอบต่อเนื่อง

(Continuous Delivery: CD) ต่อยอดโดยทำให้ผลงานที่ผ่านการทดสอบพร้อมนำขึ้นใช้งาน (deploy) ได้ตลอดเวลา ไปป์ไลน์ที่ดีให้ผลลัพธ์ที่ทำซ้ำได้และมีประตูละเอียดคุณภาพก่อนถึงสภาพแวดล้อมจริง

GitHub Actions เป็นบริการรันเวิร์กโฟลว์ที่นิยมเป็น YAML ประกอบด้วยงาน (job) ที่ทำงานบนตัวรัน (runner) ทั้งแบบที่ GitHub จัดให้และแบบติดตั้งเอง (self-hosted) [31] สาขาหลักสามารถบังคับให้ต้องผ่านการตรวจสอบสถานะ (status check) ที่กำหนดก่อนรวมโค้ด ซึ่งมีเงื่อนไขว่าการตรวจสอบนั้นต้องรายงานผลทุกครั้ง GitHub Container Registry (GHCR) เป็นที่เก็บอิมเมจคอนเทนเนอร์ [32] Trivy เป็นเครื่องมือสแกนช่องโหว่ของอิมเมจและการพึ่งพา [33] และ gitleaks ตรวจสอบหาความลับที่เผลอคอมมิต เช่น คีย์ API [34] เมื่อพบความลับจริงในประวัติ วิธีที่ถูกต้องคือหมุนเปลี่ยนความลับนั้น ไม่ใช่ใส่ไว้ในรายการยักเว้น

Ansible เป็นเครื่องมือจัดการคอนฟิกที่สั่งงานเครื่องปลายทางผ่าน Secure Shell (SSH) ด้วย playbook และโมดูลส่วนใหญ่ออกแบบให้ idempotent [35] เมื่อเครื่องปลายทางอยู่ในเครือข่ายปิดที่ตัวรันของ GitHub เข้าไม่ถึง ต้องใช้ตัวรันแบบติดตั้งเองภายในเครือข่ายนั้น และควรมีประตุนุมัติโดยผู้ตรวจที่กำหนดก่อนงานติดตั้งเริ่ม

2.13 การสังเกตการณ์ระบบ (observability)

การสังเกตการณ์คือความสามารถในการอนุมานสถานะภายในของระบบจากข้อมูลที่ระบบปล่อยออกมา โดยทั่วไปประกอบด้วยตัวชี้วัด (metrics) บันทึก (logs) และร่องรอย (traces) Prometheus เก็บอนุกรมเวลาด้วยการดึง (scrape) จาก endpoint ที่แสดงตัวชี้วัดในรูปแบบข้อความ และมีภาษาสืบค้น PromQL [36] Grafana แสดงผลเป็นแดชบอร์ด [37] ป้ายกำกับ (label) ของตัวชี้วัดควรมีค่าที่เป็นไปได้จำนวนจำกัด เช่น ใช้รูปแบบเส้นทางแทนเส้นทางจริง มิฉะนั้นจำนวนอนุกรม (cardinality) จะพุ่งสูง

ตัวชี้วัดระดับบริการ (Service Level Indicator: SLI) คือการวัดเชิงปริมาณของพฤติกรรมบริการ เช่น สัดส่วนคำขอที่สำเร็จ และเปอร์เซ็นต์ไทล์ที่ 95 (p95) ของเวลาตอบ การเลือกว่าคำขอใดถูกนับมีความสำคัญเท่ากับการนับ เพราะคำขอที่ระบบสร้างเอง เช่น การตรวจสอบสุขภาพ อาจทำให้อัตราสำเร็จดูดีเกินจริง ส่วนการทดสอบโหลดใช้เครื่องมือเช่น k6 ที่เขียนสถานการณ์เป็นสคริปต์ JavaScript เพื่อวัดพฤติกรรมของระบบภายใต้ปริมาณคำขอที่กำหนด

2.14 การบันทึกการตัดสินใจเชิงสถาปัตยกรรม (ADR)

Nygard [38] เสนอให้บันทึกการตัดสินใจทางสถาปัตยกรรมเป็นเอกสารสั้นหนึ่งฉบับต่อการตัดสินใจ ประกอบด้วยบริบท การตัดสินใจ สถานะ และผลที่ตามมา เหตุผลคือโค้ดบอกว่าระบบทำอะไร แต่ไม่บอกว่าทำไมจึงเลือกทางนั้นและทางใดถูกปฏิเสธ เมื่อสมาชิกเปลี่ยนหรือเวลาผ่านไป

ความรู้นี้จะหายถ้าไม่จด ADR ที่ปฏิเสธข้อเสนอพร้อมเหตุผลมีประโยชน์เท่ากับ ADR ที่ยอมรับ เพราะป้องกันการถกเถียงเดิมซ้ำ

2.15 สรุป

บทนี้วางแนวคิดที่ใช้ในบทถัดไป ได้แก่ กติกาโดเมน POS ที่ต้องอะตอมิก offline-first ด้วยกล่องขาออกและเครื่องเขียนเครื่องเดียว ลำดับการล็อก RLS รหัสกำกับกับการเขียน การยืนยันตัวตน และ CI/CD พร้อมการสังเกตการณ์

บทที่ 3

รายละเอียดการดำเนินงาน

3.1 ภาพรวมของโครงการ

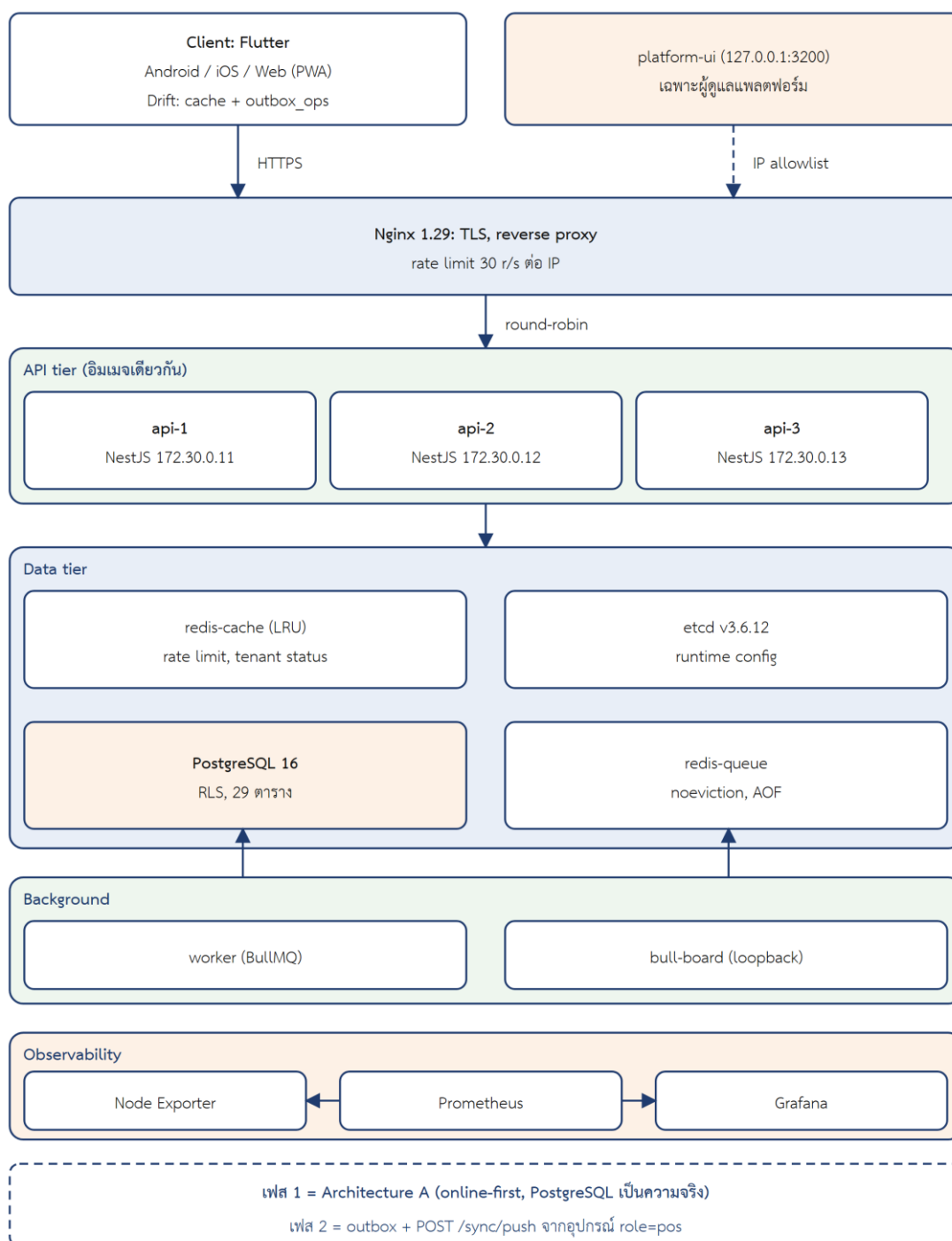
บทนี้อธิบายการออกแบบและการสร้างระบบ Srisurart Autopart POS ตามสถานะของโค้ดบนสาขา **main** ณ วันที่ 1 ตุลาคม พ.ศ. 2569 ได้แก่ สถาปัตยกรรม ส่วนประกอบ ฝั่งไคลเอนต์ ฝั่งเซิร์ฟเวอร์ การซิงค์ข้อมูลในเฟส 2 ความปลอดภัย การนำระบบขึ้นใช้งาน (deploy) เทคโนโลยีที่ใช้ และวิธีการทำงานของทีม การตัดสินใจที่สำคัญอ้างอิงบันทึกการตัดสินใจเชิงสถาปัตยกรรม (Architecture Decision Record: ADR) ในโฟลเดอร์ **docs/Backend_design/adr/** เพื่อบอกเหตุผลของการเลือก ส่วนผลที่ได้และหลักฐานรายงานไว้ในบทที่ 4

3.1.1 หลักการของสถาปัตยกรรม

ตามที่กล่าวในบทที่ 1 โครงการเลือก Architecture C (ไฮบริด) ร่วมกับโมเดลผู้เช่าแบบ T1 (ฐานข้อมูลและสคีมเดียว คอลัมน์ **tenant_id** และ Row-Level Security) โดยเฟส 1 คือ Architecture A หรือ online-first modular monolith ที่ให้ PostgreSQL เป็นแหล่งความจริงเพียงแหล่งเดียว และเฟส 2 เพิ่มความสามารถทำงานออฟไลน์กลับเข้ามา เอกสาร **03_ARCHITECTURE.md** ระบุว่า หากเวลาไม่พอ ให้ทำ A กับ T1 ให้สมบูรณ์ดีกว่าทำส่วนเสริมไม่ครบ หลักการนี้กำหนดลำดับงานของทีม คือเฟส 1 ต้องทำงานได้จริงและทดสอบได้ก่อน ทั้งไคลเอนต์เซิร์ฟเวอร์ และไปป์ไลน์อยู่ในที่เก็บโค้ดเดียว (monorepo) ตาม ADR-0011 เพื่อให้การแก้ API หนึ่งครั้งแก้ได้ทั้งสองฝั่งในคอมมิตเดียว

3.1.2 สถาปัตยกรรมโดยรวม

ไคลเอนต์ Flutter (ปัจจุบันสร้างและทดสอบเฉพาะบนเว็บ) สื่อสารกับเซิร์ฟเวอร์ผ่าน HTTPS ทุกคำขอเข้าสู่ Nginx ซึ่งเป็น reverse proxy ตัวเดียวหน้าระบบ แล้วกระจายไปยังอินสแตนซ์ NestJS [25] สามตัว (**api-1** ถึง **api-3**) ที่ใช้อิมเมจเดียวกัน ข้อมูลธุรกิจเก็บใน PostgreSQL ส่วน Redis แบ่งเป็นสองอินสแตนซ์ตามหน้าที่ งานเบื้องหลังประมวลผลโดยโปรเซส **worker** แยกต่างหาก ค่าตั้งที่ปรับขณะรันได้เก็บใน etcd และการเฝ้าสังเกตระบบใช้ Prometheus กับ Grafana ดังรูปที่ 3.1 ตารางที่ 3.1 สรุปความต่างระหว่างเฟส 1 และเฟส 2 ในมุมที่กระทบการออกแบบ ตามเอกสาร **03_ARCHITECTURE.md** และ **08_PHASE2_SPEC.md**



รูปที่ 3.1 สถาปัตยกรรมโดยรวมของระบบ Srisurart Autopart POS

ตารางที่ 3.1 เปรียบเทียบเฟส 1 (Architecture A) กับเฟส 2 (offline shell)

หัวข้อ	เฟส 1	เฟส 2
แหล่งความจริงของข้อมูล	PostgreSQL	PostgreSQL (ไม่เปลี่ยน)
บทบาทของ Drift ในเครื่อง	แคชแบบ write-through ของคำตอบจากเซิร์ฟเวอร์	แคช และเป็นที่เก็บคิว outbox_ops ตอนออฟไลน์
การเขียนข้อมูลเมื่อไม่มีเครื่องช่วย	ไม่รองรับ ยกเว้นคิวรับชำระเครดิตช้า (#24) และบิลด์ offline-first เดิม	เครื่อง pos เข้าคิวได้ตามรายการคำสั่ง (operation: op) ที่กำหนด
การออกเลขใบเสร็จ RC และใบลดหนี้ CN	เซิร์ฟเวอร์ออกให้	เครื่อง pos ออกเองทั้งออนไลน์และออฟไลน์
การส่งข้อมูลกลับ	เรียก endpoint ออนไลน์โดยตรง	POST /sync/push ด้วย device token
สถานะของไคลเอนต์	ออนไลน์ล้วน	Online / Degraded / Syncing

3.2 ส่วนประกอบของโครงการ

ที่เก็บโค้ดแบ่งเป็นส่วนหลักดังตารางที่ 3.2 นอกจากนี้ **docs/** เก็บบันทึกการส่งมอบงาน (**handoff_log**) และคู่มือผู้ใช้ภาษาไทย

ตารางที่ 3.2 ส่วนประกอบหลักของที่เก็บโค้ด

ส่วนประกอบ	ตำแหน่ง	หน้าที่
ไคลเอนต์ Flutter	frontend/	แอปหน้าร้าน (สร้างและทดสอบบนเว็บ; Android/iOS ยังไม่ทดสอบ) ประกอบด้วยชั้น data, domain และ presentation
เซิร์ฟเวอร์ NestJS	server/	API, worker, Bull Board, migrations และเครื่องมือบรรทัดคำสั่งของแพลตฟอร์ม
ชุด deploy	deploy/	Ansible playbook, ไฟล์ Compose ส่วนเสริมสำหรับ VM, การตั้งค่า Prometheus และ Grafana, สคริปต์สำรองข้อมูลและ runner
ไปป์ไลน์	.github/workflows/	flutter.yml , server.yml และ deploy.yml
เอกสารออกแบบ	docs/Backend_design/	เอกสารหมายเลข 00 ถึง 09 และ ADR ที่เป็นบันทึกการตัดสินใจที่มีผลผูกพัน

บนเครื่องเสมือน ระบบถูกจัดเป็นคอนเทนเนอร์ภายใต้ Docker Compose [30] ตามตารางที่ 3.3 ค่าเพดานหน่วยความจำ ([mem_limit](#)) มาจากไฟล์ [server/docker-compose.yml](#) ซึ่งตั้งไว้เพื่อให้ระบบทั้งชุดอยู่ในทรัพยากรของเครื่องเสมือนระดับเล็กได้

ตารางที่ 3.3 คอนเทนเนอร์หลักและเพดานหน่วยความจำ

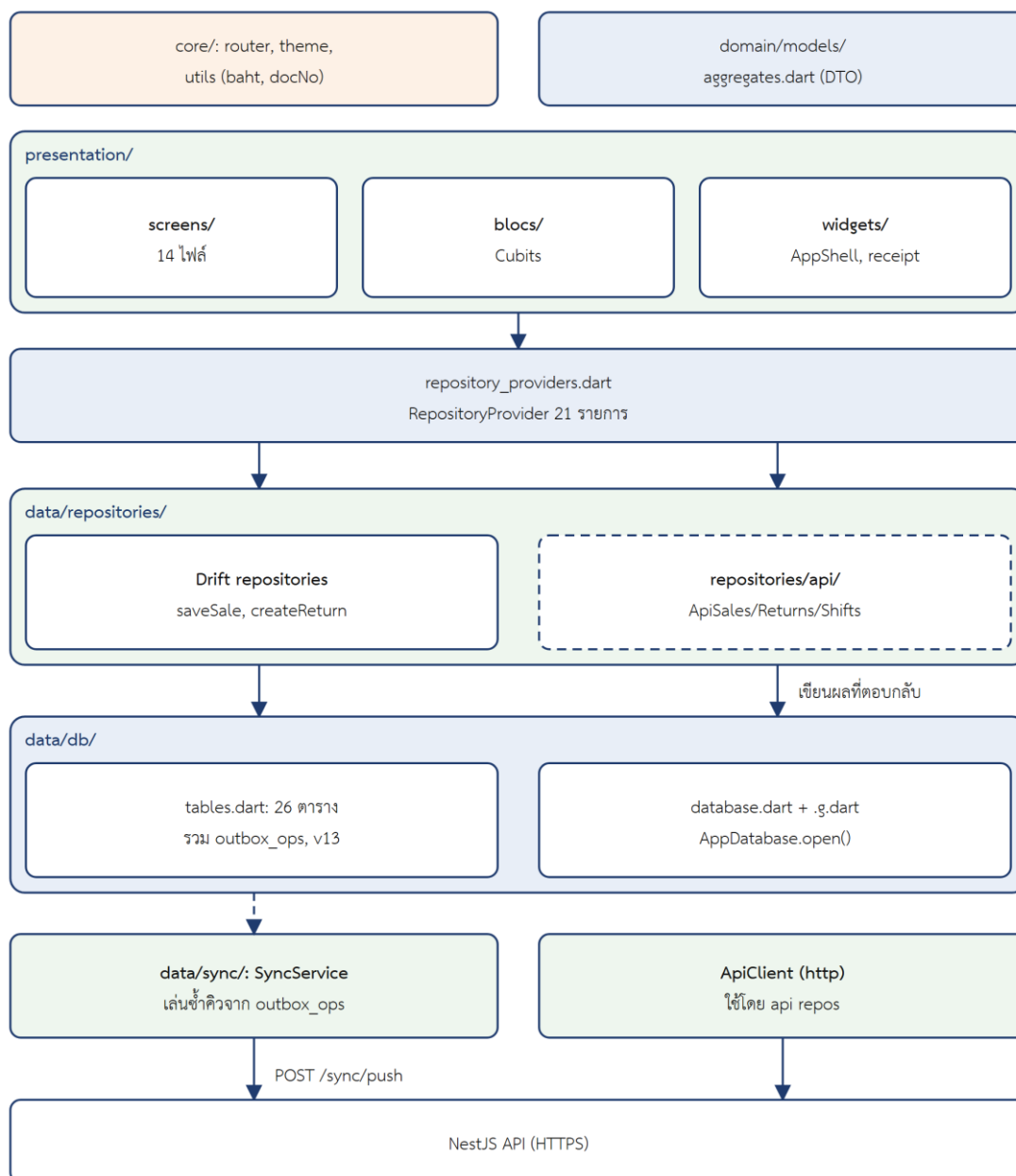
บริการ	อิมเมจ	เพดาน RAM	บทบาท
nginx	nginx:1.29-alpine	64 MB	reverse proxy, TLS, rate limit ต่อ IP, ปิดเส้นทาง <code>/metrics</code> ต่อภายนอก
api-1, api-2, api-3	อิมเมจของ server	384 MB ต่อตัว	NestJS API หลัง Nginx
worker	อิมเมจของ server	256 MB	ประมวลผลงานเบื้องหลัง เช่น export ข้อมูลรายวัน
bull-board	อิมเมจของ server	128 MB	หน้าจอดูคิวงาน ผูกกับ loopback และต้องใช้ basic auth
migrate	อิมเมจของ server	128 MB	งานครั้งเดียวที่รัน migrations ก่อนที่ api ใด ๆ จะเริ่ม
postgres	postgres:16-alpine	1024 MB	ฐานข้อมูลหลัก ตั้ง <code>max_connections=100</code>
redis-cache	redis:7-alpine	256 MB (Redis <code>maxmemory</code> 192 MB)	<code>allkeys-lru</code> , ไม่เขียนดิสก์
redis-queue	redis:7-alpine	256 MB (Redis <code>maxmemory</code> 192 MB)	<code>noeviction, appendonly yes</code> สำหรับ BullMQ
etcd	etcd v3.6.12	256 MB	ค่าตั้งแบบไดนามิก (เช่น <code>log_level</code>) ไม่ใช่ข้อมูลธุรกิจ
platform-ui	nginx:1.29-alpine	32 MB	หน้าจอผู้ดูแลแพลตฟอร์ม เปิดบน <code>127.0.0.1:3200</code> เท่านั้น
Prometheus, Grafana, Node Exporter	v2.55.1, 11.2.0, v1.8.2	512, 256 และ 64 MB (ไฟล์ <code>monitoring.yml</code>)	เก็บและแสดงตัวชี้วัด

เหตุผลที่แยก Redis เป็นสองอินสแตนซ์คือนโยบายหน่วยความจำต้องตรงกันข้าม [39] ข้อมูลใน **redis-cache** ที่tingได้เมื่อเต็ม (ตัวนับ rate limit หายแล้วนับใหม่ได้) แต่ข้อมูลใน **redis-queue** คือคิวงานที่ยังไม่ทำ จึงใช้ **noeviction** พร้อมเขียนล็อกลงดิสก์

3.3 รายละเอียดฝั่งไคลเอนต์

3.3.1 โครงสร้างแบบสามชั้น

ไคลเอนต์ Flutter [4] อยู่ในโฟลเดอร์ **frontend/lib/** และแบ่งตามหลัก data → domain → presentation ดังรูปที่ 3.2 ชั้น **core/** เก็บสิ่งที่ใช้ร่วมกัน ได้แก่ตัวกำหนดเส้นทาง **go_router** ธีมพร้อมฟอนต์ Sarabun และ Barlow และฟังก์ชันช่วยเรื่องเงินและเลขเอกสาร (**baht**, **round2**, **newId**, **docNo**, **csvSafe**) กติกาที่ใช้ทั้งโปรเจกต์คือหน้าจอต้งเรียกใช้ repository ผ่าน provider เท่านั้น ห้ามเข้าถึงฐานข้อมูลโดยตรง การจัดการสถานะใช้ **flutter_bloc** โดยมี Cubit สำหรับโหมดธีม ขนาดฟอนต์ ใบเสนอราคาที่รอ ตะกร้าสินค้า และสถานะการเข้าสู่ระบบ ส่วน repository และบริการทั้งหมดลงทะเบียนผ่านต้นไม้ **RepositoryProvider** 21 รายการ (repository 17 ตัว และบริการ **ApiClient**, **BootstrapService**, **DocCounterSeeder**, **SyncFacade**)



รูปที่ 3.2 โครงสร้างชั้นของโคลเอนต์ Flutter และเส้นทางไปยังเซิร์ฟเวอร์

3.3.2 ฐานข้อมูลภายในเครื่อง (Drift)

ฐานข้อมูลภายในเครื่องใช้ Drift [5] กำหนดไว้ใน **data/db/tables.dart** มี 26 ตาราง และ **schemaVersion** เท่ากับ 13 แบ่งเป็นสามกลุ่ม กลุ่มแรก 20 ตารางพอร์ตมาจากที่เก็บข้อมูล **sa_*** ของแอปเดิม เช่น **Products, Customers, Mechanics, Sales, SaleItems, Returns, Shifts** และ **DocCounters** กลุ่มที่สองเป็นคิวและตารางสนับสนุนการชิ่งได้แก่ **PendingCreditPayments** (#24) และตารางของเฟส 2

`DocCounterSeeds`, `OutboxOps`, `SyncCursors` และ `OpEffects` กลุ่มที่สามคือ `AppMeta` สำหรับค่าเมตาดาต้า เช่น รหัสผู้เช่าที่แคชนี้เป็นเจ้าของ การยกระดับสคีม่าที่เกี่ยวข้องกับรายงานนี้ ได้แก่ เวอร์ชัน 7 ที่ลบคอลัมน์ `offlineOk` เวอร์ชัน 12 ที่เพิ่มดัชนี และเวอร์ชัน 13 ที่เพิ่มตาราง `op_effects` เพื่อเก็บผลต่างที่คำสั่งออฟไลน์ใช้จริง

ไฟล์ `database.g.dart` ที่ Drift สร้างอัตโนมัติถูกคอมมิตไว้ในที่เก็บโค้ด เพราะเครื่องมือ `build_runner` ทำงานบนพารามิเตอร์ที่มีอักขระนอกชุด ASCII ไม่ได้ (รายละเอียดในบทที่ 5) และ CI มีงาน `codegen-check` ตรวจสอบว่าไฟล์ที่สร้างตรงกับของที่คอมมิตไว้ สำหรับเว็บ ไฟล์ `web/sqlite3.wasm` และ `web/drift_worker.js` ต้องมีเวอร์ชันตรงกับแพ็คเกจ `sqlite3` และ `drift` ที่ล็อกไว้ CI จึงทำให้บิลด์ล้มเหลวเมื่อเวอร์ชันไม่ตรง เพราะความไม่ตรงกันเปลี่ยนพฤติกรรมของแกน SQLite หรือการยกเลิกงานของ worker

3.3.3 Repository ที่พอร์ตมาจาก db.js และกติกาคงไว้

ชั้น `data/repositories/` มี repository หนึ่งตัวต่อโดเมน กฎธุรกิจถอดมาจากไฟล์ `pos/db.js` ของแอปเดิม แต่เปลี่ยนกลไกจาก `snapshot/rollback` ใน JavaScript มาเป็น `transaction(() async {...})` ของ Drift ซึ่งหากเกิดข้อผิดพลาดกลางทางจะยกเลิกทั้งชุด ตารางที่ 3.4 สรุปกติกาสำคัญ

ตารางที่ 3.4 กติการธุรกิจของ repository ที่ทำธุรกรรม

กระบวนการ	กติกาที่คงไว้
<code>saveSale</code>	ตรวจสอบสต็อกล่วงหน้า (ข้อความผิดพลาดภาษาไทย สต็อกไม่พอ...); ตัดสต็อกแบบเข้มงวด ไม่ปัดค่าติดลบเป็น ศูนย์; แด้มที่ได้ = $\text{floor}(\text{total}/10)$; อัปเดตยอดสะสมและแด้มลูกค้า รวมทั้งสถิติช่วง เมื่อชำระด้วย เครดิตช่วง
<code>createReturn</code>	กันคืนเกินจำนวน (คืนได้ไม่เกินจำนวนที่ขายลบจำนวนที่คืนไปแล้ว); ย้อนส่วนลด แด้ม และยอดช่วงตาม สัดส่วน; ลดยอดเครดิตเฉพาะกรณี หักจากเครดิต ; ยกเลิก บิลแม่อัตโนมัติเมื่อคืนครบทั้งบิล
<code>receivePO</code>	ต้นทุนถัวเฉลี่ยถ่วงน้ำหนัก $\text{round2}((\text{oldQty} \cdot \text{oldCost} + \text{newQty} \cdot \text{newCost}) / \text{total})$
<code>openShift</code>	เปิดได้หลายกะต่อวัน (แตกต่างจาก db.js โดยตั้งใจ); ถ้า <code>id</code> มีอยู่แล้วคืนกะเดิม มิฉะนั้นจัดเก็บ (archive) กะที่ยังเปิดอยู่ก่อนหน้าก่อนเปิดกะใหม่; บันทึก รายการล้นชักไม่ได้หลังปิดกะ
ใบเสนอราคา และบิลที่หัก	ไม่แตะสต็อกเลย
<code>adjustStock</code>	ปรับสต็อกด้วยมือ และปัดค่าติดลบเป็น 0 (ต่างจากการขาย)

การแยกให้การขาย "ตัดสต็อกแบบเข้มงวด" แต่การปรับด้วยมือ "ปัดที่ศูนย์" เป็นการแยกความหมายโดยตั้งใจ เพราะการขายที่ทำให้สต็อกติดลบหมายถึงข้อมูลคลาดเคลื่อนที่ควรยกเลิกทั้งธุรกรรม (rollback) เพื่อให้เห็นชัด ส่วนการปรับด้วยมือเป็นการแก้ตัวเลขโดยผู้ใช้ นอกจากนี้ฟังก์ชัน `planReturn()` และ `refundedQtyOf()` ในไฟล์ `return_plan.dart` เป็นกฎที่ไม่มีผลข้างเคียง (pure function) ที่ทั้ง `ReturnsRepository` และ `ApiReturnsRepository` ใช้ร่วมกัน และยอดเงินสดที่คาดหวังในล้นชักรับจำนวนจากฟังก์ชันเดียว `ShiftsRepository.cashCountFrom(shift)` ทั้งในหน้าล้นชักและรายงานปิดร้าน เพื่อไม่ให้มีกติกาเดียวกันสองชุดที่เบี่ยงจากกัน ส่วน `SnapshotRepository` มี `exportSnapshot()` และ `importLegacyBackup()` ที่นำเข้าไฟล์สำรองของแอป JavaScript เดิมแบบอะตอมมิก ซึ่งเป็นเส้นทางย้ายข้อมูลของร้านจริง

3.3.4 Repository แบบ API (USE_API_WRITES)

เมื่อคอมไพล์ด้วย `--dart-define=USE_API_WRITES=true` ต้นไม้ provider จะสลับไปใช้ repository ชุดที่เรียกเซิร์ฟเวอร์ เช่น `ApiSalesRepository`, `ApiReturnsRepository` และ `ApiShiftsRepository` ตามที่ ADR-0010 กำหนดว่า `ApiRepository` คือ implementation ใหม่ของ interface เดิมที่เขียนผลที่เซิร์ฟเวอร์ตอบลง Drift (write-through) เหตุผลคือ interface เดิมอยู่ใต้หน้าจอทั้งหมด การสลับ implementation จึงไม่ต้องแก้หน้าจอ และสคิมาของ Drift กับของ PostgreSQL วิวัฒนาการแยกกันได้ กติกาของขั้นนี้มีห้าข้อ

1. ค่า `id` ของบิลและ `Idempotency-Key` สร้างครั้งเดียวต่อตะกร้า ไม่ใช่ต่อการเรียก โดย `PendingWrites` เก็บความพยายามที่ยังไม่ทราบผล มิฉะนั้นการลองใหม่จะได้รหัสใหม่และทำให้บิลซ้ำ
2. เฉพาะรหัส 4xx เท่านั้นที่เป็นคำตัดสิน ส่วน 5xx, 429 หรือคำตอบที่หายไปหมายถึงไม่ทราบว่าการเขียนสำเร็จหรือไม่ ห้ามถอยไปเขียนในเครื่อง ยกเว้นความล้มเหลวของการส่งผ่านเครือข่ายจริง (transport failure) ไฟล์ `api_repository_contract_test.dart` บังคับข้อนี้กับโค้ดทั้งโฟลเดอร์ และเมื่อวันที่ 27 กันยายน พ.ศ. 2569 เจ้าของโครงการตัดสินใจเพิ่มว่า 5xx หรือ 429 ไม่เข้าคิวออฟไลน์ แต่เก็บความพยายามไว้พร้อมแสดงข้อผิดพลาด
3. `ApiRepository` ห้ามเรียกบริการธุรกรรมของ Drift เพราะจะติดสต็อกซ้ำสองรอบ
4. `ApiException` ห้ามหลุดถึงหน้าจอ ต้องแปลงเป็นข้อความภาษาไทยผ่าน `rethrowThai` หรือ `rethrowServerRefusal`
5. ความยินยอม เช่น `overrideCreditLimit` ต้องส่งต่อจากกล่องโต้ตอบที่ผู้ใช้เห็นจริง ไม่ใช่สรุปเองจากการตรวจในเครื่องที่อาจล้าสมัย

3.3.5 หน้าจอ

โคลเอนต์มีไฟล์หน้าจอ 14 ไฟล์ใน `presentation/screens/` โดย 12 หน้าอยู่ในเมนู AppShell อีกสองหน้าคือ `/devices` และ `/login` ดังตารางที่ 3.5 สตรีงภาษาไทยในหน้าจอยึดหลัก "พฤติกรรมเหมือนเดิม" คือคัดลอกจาก `db.js` หรือไฟล์ `.jsx` ของแอปเดิมโดยไม่แปลใหม่ ส่วนสตรีงใหม่ของเฟส 2 ถูกเก็บไว้ใน `02_API_SCREEN.md` ให้เจ้าของโครงการรับรองก่อน

ตารางที่ 3.5 หน้าจอหลักของโคลเอนด์

ไฟล์	หน้าที่
<code>checkout_screen.dart</code>	ขายสินค้า จำนวนส่วนลดและแต้ม รับชำระ ใบเสร็จ พักbil ราคาช่าง
<code>products_screen.dart</code>	จัดการสินค้า สต็อก หมวดหมู่ ชัฟฟลายเออร์ ป้าย ราคา
<code>purchase_orders_screen.dart</code>	ใบสั่งซื้อและการรับสินค้า
<code>vehicle_search_screen.dart</code>	ค้นหาอะไหล่จากรุ่นรถ
<code>customers_screen.dart, mechanics_screen.dart</code>	ลูกค้า แต้มสะสม และช่างพร้อมเครดิตและการรับชำระ
<code>returns_screen.dart</code>	คืนสินค้า ออกใบลดหนี้ และประวัติการคืน
<code>quotes_screen.dart</code>	ใบเสนอราคาและ PDF A4
<code>reports_screen.dart</code>	รายงานสรุป รายงานปิดร้าน สต็อกต่ำ
<code>cash_drawer_screen.dart</code>	เปิดปิดกะ รายการเข้าออกสิ้นชัก
<code>owner_review_screen.dart</code>	รายการที่ซิงก์ไม่ผ่านหรือต้องให้เจ้าของร้านตรวจ
<code>devices_screen.dart</code>	ผูกและถอดถอนเครื่อง ออกรหัสผูกเครื่อง
<code>settings_screen.dart, login_screen.dart</code>	ข้อมูลร้าน การตั้งค่า และการตั้ง PIN ออฟไลน์; การเข้าสู่ระบบ

3.3.6 การแยกข้อมูลแคชตามผู้เช่า

บนบิลด์ API ฐานข้อมูล Drift ต้องไม่ปะปนข้อมูลของร้านต่างกัน ตาราง **AppMeta** เก็บ **tenant_id** ของแคชปัจจุบัน และ **TenantCacheGuard** ตรวจสอบทุกครั้งที่เข้าสู่ระบบ หากพบว่าเปลี่ยนร้านจะล้างแคช แต่จะปฏิเสธการเปลี่ยนร้านเมื่อยังมีงานที่ส่งไม่ครบ ส่วนตัวนับรุ่นของแคช (**cacheGeneration**) และรุ่นของเซสชันใน **ApiClient** ป้องกันไม่ให้คำตอบที่มาจากซ้ำของร้านหรือผู้ใช้ก่อนหน้าไปเขียนทับข้อมูลของคนถัดไป

3.4 รายละเอียดฝั่งเซิร์ฟเวอร์

3.4.1 โมดูลของ NestJS

เซิร์ฟเวอร์อยู่ใน **server/src/** เขียนด้วย TypeScript บน NestJS โดยแต่ละโดเมนเป็นโมดูลแยกตามโฟลเดอร์ ดังตารางที่ 3.6 โค้ดมีสามจุดเริ่มต้นจากอิมเมจเดียวกัน คือ **main.ts** (API), **worker.ts** (ตัวประมวลผลคิว) และ **bull-board.ts** (หน้าจอดูคิว)

ตารางที่ 3.6 โมดูลหลักของเซิร์ฟเวอร์

โมดูล	หน้าที่
auth, devices, people	การเข้าสู่ระบบ โทเค็นต่ออายุ รหัสผ่านชั่วคราว การ enrol เครื่อง และผู้ใช้
sales, returns, shifts, parked-sales, quotes	งานขายหน้าร้านและธุรกรรมเงิน
products, purchasing, customers, mechanics	ข้อมูลหลักและสต็อก
documents	ตัวนับเลขเอกสาร (doc_counters)
idempotency	การกันคำสั่งเขียนซ้ำด้วย Idempotency-Key
sync	POST /sync/push และ POST /sync/discards
reports, review-items, settings	รายงาน รายการตรวจ และการตั้งค่า รวมถึง GET /bootstrap
platform	ระบบผู้ดูแลแพลตฟอร์ม: สร้างและรับร้าน นำเข้าข้อมูล
backup, queue	export ข้อมูลรายร้านแบบงานเบื้องหลัง คิวและตัวตั้งเวลา
metrics, health, rate-limit, audit, config, infra, common, db	สังเกตการณ์ จำกัดอัตรา บันทึกรายงาน ค่าดัชนีระบบ และโครงสร้างพื้นฐาน

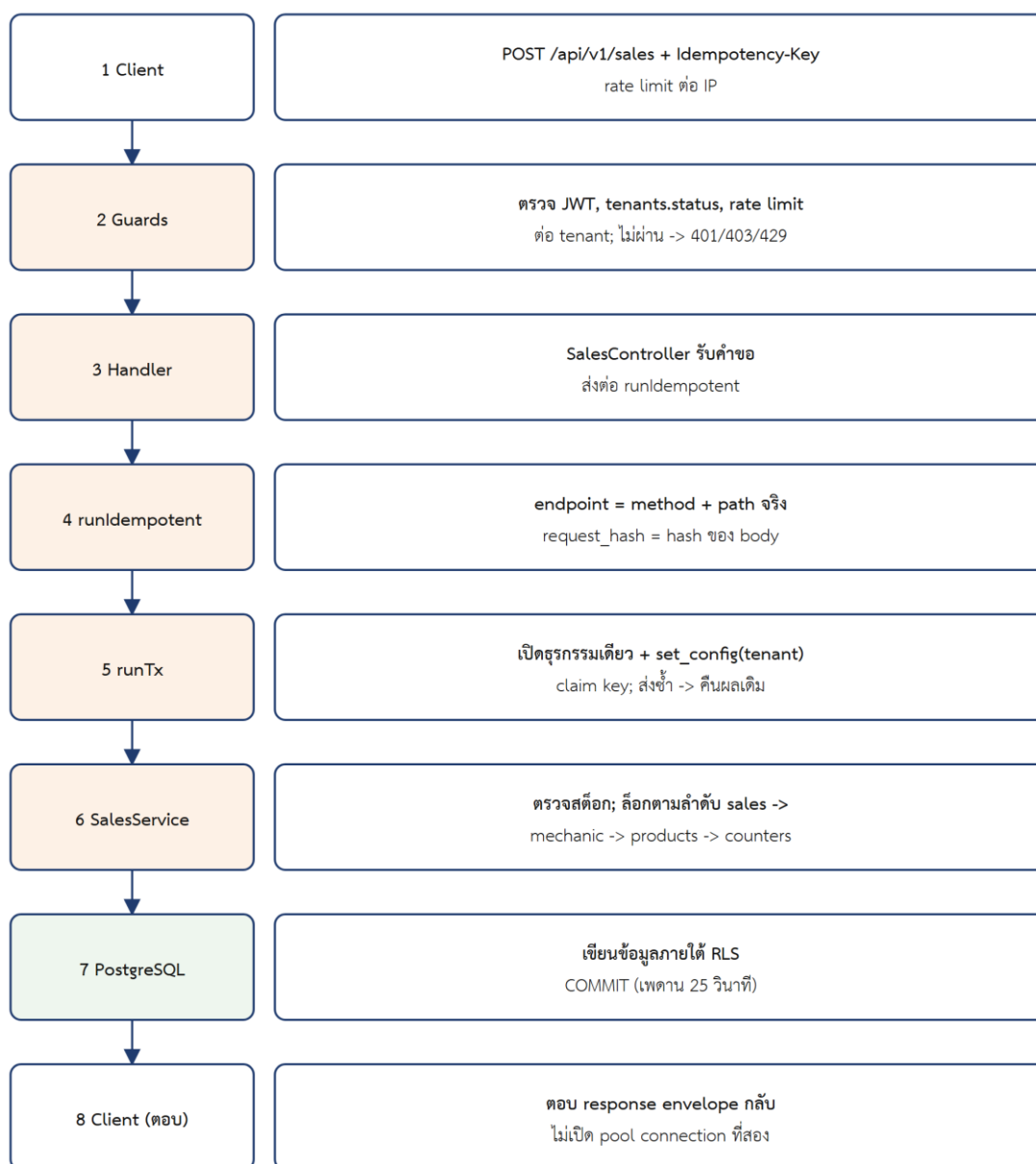
3.4.2 การแยกข้อมูลผู้เช่าด้วย RLS และ TenantService.runTx

ทุกตารางธุรกิจมีคอลัมน์ `tenant_id` และเปิด Row-Level Security แบบ `ENABLE` พร้อม `FORCE` [10] ตามแนวคิดในรูปที่ 2.1 แอปเชื่อมต่อในฐานระบบบาท `pos_app` ที่ไม่ใช่ superuser และไม่ใช่เจ้าของตาราง นโยบาย `tenant_isolation` อ่านค่า `current_setting('app.tenant_id', true)` ที่ตั้งด้วย `set_config` ภายในธุรกรรมของคำขอ และห่อด้วย `NULLIF(..., '')` เพื่อให้ค่าว่างให้ผลเป็นศูนย์แถวแทนข้อผิดพลาด ดังนั้นแม้ได้แอปลืมใส่เงื่อนไข `WHERE tenant_id` ฐานข้อมูลก็ยังคืนเฉพาะแถวของร้านนั้น

หัวใจของการบังคับใช้คือ `TenantService.runTx(fn)` ซึ่งไม่รับรหัสผู้เช่า (tenant id) เป็นอาร์กิวเมนต์ จึงปลอดภัยโดยโครงสร้าง เพราะผู้เช่ามาจาก JWT ที่ guard ตรวจสอบแล้วเท่านั้น ภาคผนวกของ ADR-0003 แบ่งหน้าที่ว่า guard เป็นผู้ตัดสินใจสถานะของร้าน ส่วน `runTx` ซึ่งอยู่ใน handler เป็นผู้เปิดธุรกรรมและตั้งค่าตัวแปรเซสชัน แทนแบบเดิมที่ middleware เปิดธุรกรรมให้ทุกคำขอ กติกาที่ผูกพันกับส่วนนี้มีสี่ข้อ

1. ธุรกรรมอยู่ใน handler ไม่ใช่ middleware, guard หรือ interceptor
2. ไม่มีส่วนใดของการเชื่อมต่อที่สองจาก pool ในคำขอเดียว (รวมถึง `Promise.all` ของ `runTx` หลายตัว) เพราะเคยทำให้ connection pool เต็มจนระบบหยุดตอบสนองมาแล้ว (บทที่ 5)
3. งานที่ช้าแต่ไม่ต้องอยู่ในธุรกรรมทำก่อนเข้าธุรกรรม เช่น การตรวจ PIN ผู้จัดการด้วย `argon2` สำหรับการยกเลิกบิลในเฟส 1 (เฟส 2 ตัด PIN ออกและให้ระบุเหตุผลแทน)
4. มีตัวคุมเพดานเวลา 25 วินาทีที่ย้อนธุรกรรมก่อน `COMMIT` พร้อมตั้ง `statement_timeout=25s` ให้ `pos_app`

ไฟล์ทดสอบเชิงสถาปัตยกรรมสามไฟล์คือ `tenant-door.spec.ts`, `tenant-wrapper.spec.ts` และ `idempotency-routes.spec.ts` บังคับข้อแรกและข้อที่สองในระดับโครงสร้างโค้ด ส่วนเพดานเวลาทดสอบใน `tx-ceiling.e2e.spec.ts` ลำดับการประมวลผลคำขอเขียนหนึ่งคำขอแสดงดังรูปที่ 3.3



รูปที่ 3.3 ลำดับการประมวลผลคำขอเขียนหนึ่งคำขอผ่านชั้นการแยกผู้เช่าและ idempotency

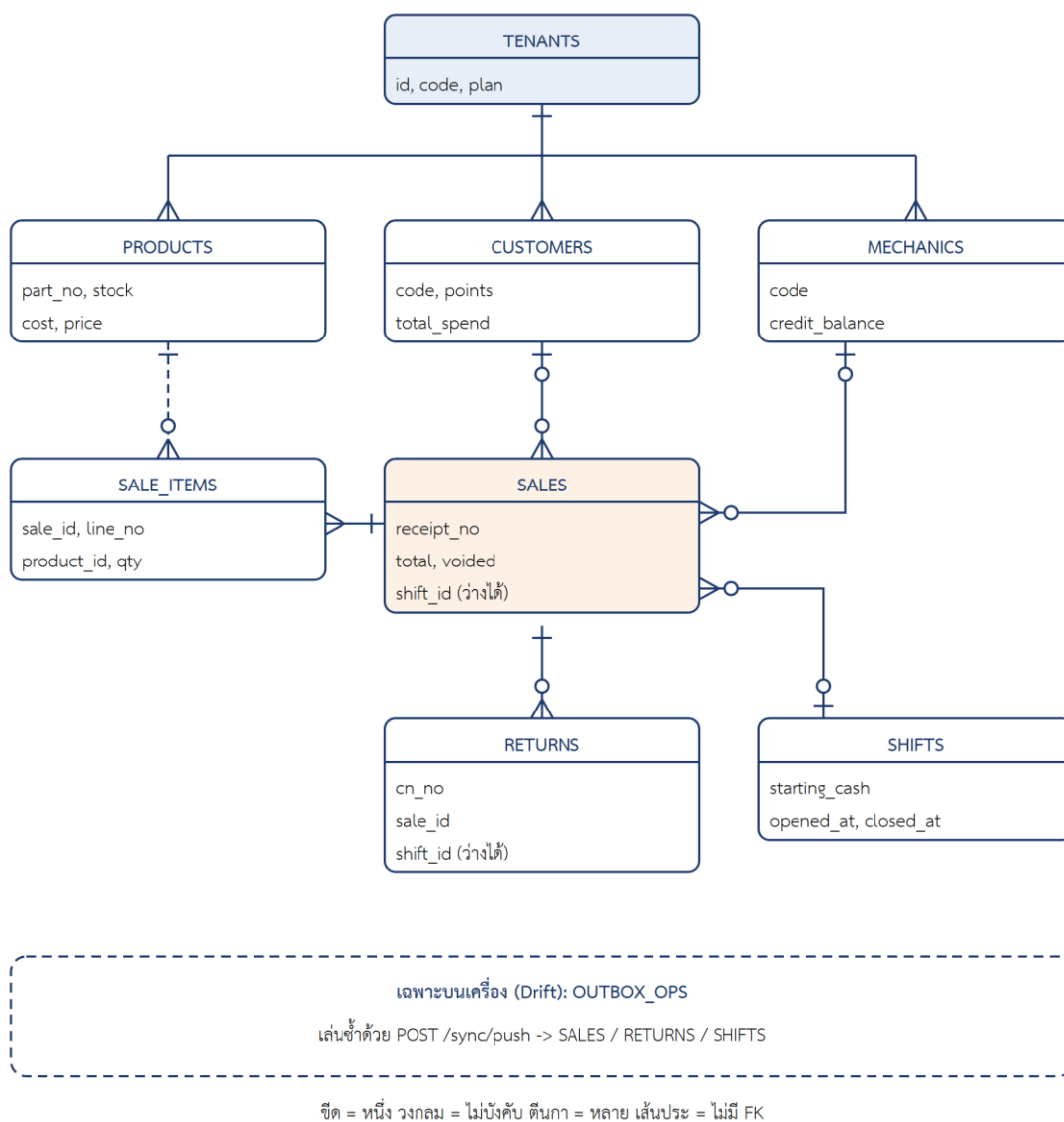
3.4.3 Idempotency และลำดับการล็อก

ระบบบังคับ **Idempotency-Key** กับคำสั่งเขียน ตาราง **idempotency_keys** เก็บคีย์และค่าสรุปของคำขอ (fingerprint) โดย fingerprint คือ method บวกพารามิเตอร์ของคำขอ (ไม่ใช่ pattern ของ route) บวกแฮชของเนื้อหา เพื่อไม่ให้คีย์เดียวกันถูกใช้กับสองบิลที่ต่างกันแล้วชนกัน คำขอซ้ำที่คีย์ตรงกันได้คำตอบเดิม คีย์เดิมกับเนื้อหาต่างกันถูกปฏิเสธ และคีย์ที่คำขอแรกยังประมวลผลอยู่ได้ **503 IDEMPOTENCY_KEY_IN_FLIGHT** การบันทึกคีย์อยู่ในธุรกรรมเดียวกับการเขียน และตัวนับการตอบซ้ำเพิ่มค่าเฉพาะเมื่อธุรกรรมยืนยันแล้ว

เพื่อกันเดดล็อก ทีมกำหนดลำดับล็อกเดียวสำหรับงานด้านการเงินและสต็อก คือ **sales**
 → ช่าง → **products** → **doc_counters** → ลูกค้า โดยการยกเลิกบิลและการคืนสินค้า
 อ่าน **shifts** แบบ **FOR SHARE** แทรกระหว่างล็อกบิลกับล็อกช่าง ส่วนการรับใบสั่งซื้อล็อกแถว
 PO ก่อนแล้วล็อกสินค้าเรียงตาม id และคำนวณต้นทุนแล้วเฉลี่ยแบบปัดสตางค์ครึ่งขึ้น (ต่างจาก
round2 แบบทศนิยมลอยตัวฝั่ง Dart โดยตั้งใจ) โค้ดใหม่ที่เขียนเกินหนึ่งตารางในกลุ่มนี้ต้องคง
 ลำดับเดิม

3.4.4 ฐานข้อมูล PostgreSQL

สคีมามาจาก migrations เท่านั้น ไฟล์ **server/src/db/data-source.ts**
 ลงทะเบียน migration 20 ไฟล์ ซึ่งสร้างตารางรวม 29 ตาราง (27 ตารางจาก **InitialSchema**
 บวก **import_jobs** และ **owner_review_items**) เอกสาร **01_DATABASE.md** ระบุว่า
 ไฟล์ migration เป็นแหล่งความจริงของสคีม่า ส่วนภาษานิยามข้อมูล (Data Definition Language:
 DDL) ในเอกสารเป็นเพียงตัวอย่างประกอบ คีย์หลักของตารางธุรกิจเป็นคีย์ผสม (**tenant_id**,
id) และคีย์นอกอ้างอิงด้วยคูนั่น เช่น **sales** อ้างอิง **customers (tenant_id**,
customer_id) ทำให้บิลของร้านหนึ่งชี้ไปลูกค้าของอีกร้านไม่ได้ ตัวเลขเงินใช้
NUMERIC(12,2) และส่งผ่านเครือข่ายเป็นสตริง เช่น **"1234.50"** ส่วนคอลัมน์ **id** เป็น
 ข้อความที่โคลเอนต์สร้างได้ ซึ่งจำเป็นต่อการออกบิลออฟไลน์ ตารางแกนกลางแสดงในรูปที่ 3.4 และ
 ตารางที่ 3.7



รูปที่ 3.4 แผนภาพ ER ของตารางแกนกลางในเซิร์ฟเวอร์ และตาราง outbox_ops ที่อยู่ในเครื่อง

ตารางที่ 3.7 ตารางแกนกลางและข้อกำหนดสำคัญ

ตาราง	คีย์ / ข้อกำหนดสำคัญ	หมายเหตุ
tenants	id UUID; code UNIQUE; plan $\in$ basic, demo, loadtest; status $\in$ active, suspended, closed	ฐานของผู้เช่า ไม่มี tenant_id เพราะเป็นตัวเอง
products	PK (tenant_id, id); ดัชนีไม่ซ้ำ uq_products_partno_ci บนเลขชิ้นส่วนแบบไม่แยกตัวพิมพ์	ความไม่ซ้ำเป็นข้อจำกัดของฐานข้อมูล ไม่ใช่การตรวจในแอป
sales	PK (tenant_id, id); UNIQUE (tenant_id, receipt_no); voided; shift_id	ผูกลูกค้าและช่างด้วยคีย์ผสม
sale_items	PK (tenant_id, sale_id, line_no); cost_at_sale	บิลเก่าค่า NULL ห้าม backfill (ADR-0008)
returns	PK (tenant_id, id); UNIQUE (tenant_id, cn_no); อ้างอิง sales	ใบลดหนี้
customers, mechanics	PK (tenant_id, id); UNIQUE (tenant_id, code)	มีดัชนีสำหรับการดึงข้อมูลแบบ keyset
shifts	PK (tenant_id, id); uq_shift_active หนึ่งกะ active ต่ออุปกรณ์	หลายกะต่อวันได้
devices	UNIQUE (tenant_id, device_no); one_pos_per_tenant	เครื่อง pos ได้เพียงเครื่องเดียวต่อร้าน
doc_counters	คีย์ (tenant_id, device_id, doc_type, period)	เก็บ high-water mark ของเลขเอกสาร
idempotency_keys	PK (tenant_id, key); endpoint (method + พารามิเตอร์) และ request_hash	ใช้ใน runIdempotent

นอกจากนี้ `audit_log` เป็นแบบเพิ่มอย่างเดียว (append-only) สำหรับ `pos_app` และต้นทุน ณ วันขาย (`cost_at_sale`) ถูกเก็บในบรรทัดขายตาม ADR-0008 เพื่อให้กำไรในรายงานไม่เปลี่ยนเมื่อราคาซื้อเปลี่ยน

3.4.5 จุดปลายทาง API หลัก

เซิร์ฟเวอร์เปิดจุดปลายทางภายใต้ `/api/v1` ตารางที่ 3.8 สรุปกลุ่มหลักที่ตรวจจากตัวควบคุม (`*.controller.ts`) โดยตรง เอกสาร `02_API_SCREEN.md` เป็นสัญญาฉบับเต็ม และระบุบทบาทอุปกรณ์ที่เรียกได้ในแต่ละแถว คำตอบมีรูปแบบของมาตรฐาน (`status, data, meta`) ข้อผิดพลาดมีรหัสที่เครื่องอ่านได้ ซึ่งโคลเอนต์แปลงเป็นข้อความไทยผ่าน `server_error_resolver.dart`

ตารางที่ 3.8 กลุ่มจุดปลายทางหลักของ API

กลุ่ม	จุดปลายทาง	หน้าที่
ยืนยันตัวตน	POST <code>/auth/token, /auth/refresh, /auth/change-password, /auth/device</code> ; GET <code>/auth/me</code>	เข้าสู่ระบบ ต่ออายุ เปลี่ยนรหัสผ่าน และแลก enrolCode เป็น device token
การขาย	POST <code>/sales</code> , GET <code>/sales</code> , GET <code>/sales/:id</code> , GET <code>/sales/:id/refunded-qty</code> , POST <code>/sales/:id/void</code>	ขาย ดูบิล และยกเลิกบิล
การคืน	POST <code>/returns</code> , GET <code>/returns</code>	ใบลดหนี้
กะและลิ้นชัก	GET <code>/shifts/current, /shifts/history</code> ; POST <code>/shifts/open, /shifts/close, /shifts/current/entries</code>	เปิด ปิดกะ และบันทึกเงินเข้าออก
สินค้า	GET/POST/PATCH/DELETE <code>/products</code> , POST <code>/products/:id/adjust-stock</code> , GET <code>/products/:id/suppliers</code>	จัดการสินค้า ปรับสต็อก (ดึงด้วย <code>updatedSince</code> และ <code>afterId</code>)
จัดซื้อ	<code>/purchase-orders</code> พร้อม POST <code>/:id/receive</code> , POST <code>/:id/cancel</code>	ใบสั่งซื้อ รับสินค้า

กลุ่ม	จุดปลายทาง	หน้าที่
หมวดหมู่และซัพพลายเออร์	/categories, /suppliers, GET /movements	ข้อมูลประกอบสินค้าและประวัติความเคลื่อนไหวสต็อก
ลูกค้าและช่าง	/customers, /mechanics, POST /mechanics/:id/credit-payments	ข้อมูลหลักและการรับชำระเครดิต
บิลที่พัก	GET/POST/DELETE /parked-sales	พักตะกร้า
ใบเสนอราคา	/quotes พร้อม /:id/duplicate, /:id/convert, POST /quotes/purge	ใบเสนอราคาและการแปลงเป็นบิล
รายงาน	GET /reports/summary, /closing, /top-products, /by-category, /stock-value, /low-stock, /product-sales	รายงาน
อุปกรณ์	GET/POST /devices, POST /devices/:id/retire; GET /doc-counters	enrol ถอดถอน และอ่านตัวนับเลข
ซิงค์	POST /sync/push, POST /sync/discards	ส่งคิวจากเครื่อง pos และทิ้งรายการที่ติด
รายการตรวจ	GET /review-items, POST /review-items/:id/reviewed	หน้า "รอเจ้าของร้านตรวจ"
การตั้งค่า	GET/PATCH /settings, GET /bootstrap	ตั้งค่าร้านและข้อมูลเริ่มต้น
สำรองข้อมูล	POST /backup/export, GET /backup/jobs/:id, .../download	export รายร้านแบบงานเบื้องหลัง
ผู้ดูแลแพลตฟอร์ม	POST /platform/auth/token; /platform/tenants (สร้าง ระบุ ดู ออกรหัสผ่านชั่วคราว นำเข้า)	ระนาบแยกต่างหาก
สุขภาพและตัวชี้วัด	GET /health/live, /health/ready, /metrics	ตรวจสอบสถานะและ Prometheus

3.4.6 การยืนยันตัวตนและการผูกเครื่อง

ตาม ADR-0009 โทเค็นเข้าถึงมีอายุ 15 นาที และโทเค็นต่ออายุหมดอายุเวลา 04:00 น. ของทุกวัน ไม่ใช่ 24 ชั่วโมงนับจากการเข้าสู่ระบบ เพราะร้านเปิดปิดกะทุกวันอยู่แล้ว เครื่องหน้าเคาน์เตอร์มีคนเดินผ่านจึงไม่ควรค้างเซสชันหลายวัน และการเพิกถอนทำได้โดยไม่ต้องมีรายการห้ามใช้ใน Redis เพราะเมื่อปิด `users.is_active` การต่ออายุครั้งถัดไปจะไม่ผ่าน เวลา 04:00อยู่นอกเวลาทำการ พนักงานจึงไม่ถูกบังคับออกจากระบบขณะลูกค้ายืนรอ ภาควิชาของ ADR-0009 กำหนดลายเซ็น RS256 [23] และบนเว็บให้เก็บโทเค็นเข้าถึงในหน่วยความจำเท่านั้น ส่วนโทเค็นต่ออายุและโทเค็นอุปกรณ์เก็บใน IndexedDB โดยห้ามมี `localStorage` เป็นทางเลือก

ADR-0004 แบ่งเครื่องตามบทบาท คือ `pos` ได้เพียงเครื่องเดียวต่อร้าน (แต่ละลิ้นชัก ออกเลขเอกสาร และเขียนออพไลน์ได้) และ `backoffice` ได้หลายเครื่อง (สต็อกและรายงาน ออนไลน์เท่านั้น) เส้นแบ่งคือ "แต่ละลิ้นชักหรือไม่" ไม่ใช่ "แต่ละสต็อกหรือไม่" เจ้าของร้านออก `enrolCode` ผ่าน `POST /devices` แล้วเครื่องนำรหัสไปแลกเป็น device token ที่ `POST /auth/device` และดัชนีไม่ซ้ำ `one_pos_per_tenant` บังคับกติกาเครื่อง `pos` เดียวที่ระดับฐานข้อมูล ในเฟส 2 เจ้าของโครงการกำหนดเพิ่มว่าร้านมีบทบาทผู้ใช้ `owner` เพียงบทบาทเดียว และการถอดถอนเครื่อง การ enrol และการ export ต้องมี device token ที่ enrol แล้ว รหัสผ่านอย่างเดียวไม่พอ

การตรวจรหัสผ่านใช้ argon2 โดยลงรูปแบบยูนิโคด Normalization Form C (NFC) ก่อน แล้วลองสตริงดิบเป็นทางเลือก เพราะ NFC จัดเรียงสระและวรรณยุกต์ภาษาไทยใหม่ ขณะที่แฮชที่สร้างไว้ก่อนหน้านี้เป็นแฮชของสตริงดิบ หากตัดทางสำรองออก เจ้าของร้านที่ตั้งรหัสไว้ก่อนจะเข้าระบบไม่ได้ ความยาวรหัสผ่านขั้นต่ำกำหนดไว้ที่เดียวใน `src/common/password.ts` และเมื่อสร้างร้านใหม่ ระบบออกรหัสผ่านชั่วคราวอายุ 7 วันให้เจ้าของร้าน และเมื่อเข้าสู่ระบบด้วยรหัสนี้ต้องตั้งรหัสใหม่ภายใน 10 นาที

3.4.7 ระบุผู้ดูแลแพลตฟอร์ม

ADR-0001 กำหนดให้ผู้ดูแลแพลตฟอร์มสร้างร้านใหม่ผ่าน `POST /platform/tenants` ในธุรกรรมเดียว ได้ผู้เช่า เจ้าของ การตั้งค่า และข้อมูลเริ่มต้นพร้อมกัน ADR-0002 แยกผู้ดูแลเป็นตาราง `platform_admins` ต่างหาก ใช้ JWT ที่มีกลุ่มผู้รับ (`aud`) ต่างกัน ใช้การเชื่อมต่อฐานข้อมูลที่ข้าม RLS แยกต่างหาก และบันทึกตรวจสอบทุกครั้ง เหตุผลคือถ้าบัญชีผู้ดูแลอยู่ในตาราง `users` เดียวกับร้าน ขอบเขตของ RLS จะพัวพัน Nginx อนุญาตเส้นทาง `/api/v1/`

`platform/` เฉพาะ loopback และ IP ที่ระบุใน `PLATFORM_ADMIN_IPS` และหน้าจอบริการ `platform-ui` ผูกกับ `127.0.0.1:3200` ซึ่งเข้าถึงผ่านอุโมงค์ SSH เท่านั้น

3.4.8 การจำกัดอัตรา งานเบื้องหลัง และ etcd

ADR-0006 บังคับการจำกัดอัตราสองชั้น ชั้นนอกคือ Nginx ใช้ `limit_req_zone` ต่อ IP (30 คำขอต่อวินาที burst 60) ป้องกันการท่วมจากภายนอก ชั้นในคือ guard ของ NestJS กับ Redis ต่อ `tenant_id` กันไม่ให้ร้านเดียวกันทรัพยากรจนร้านอื่นช้า เหตุผลที่ไม่ทำต่อร้านที่ Nginx คือ Nginx รันฟรีอ่าน JWT ไม่ได้ และการให้ไคลเอนต์ส่งรหัสร้านในส่วนหัวมาเองฝ่าฝืนกติกาว่าผู้เช่าต้องมาจาก JWT เท่านั้น การเข้าสู่ระบบนับความพยายามด้วยคำสั่ง Lua แบบอะตอมิกบน Redis ก่อนทราบผล เพื่อให้ทุกการปฏิเสธถูกนับ หาก Redis ล่ม ตัวนับทำงานแบบ fail-open เพราะร้านหยุดขายไม่ได้ แต่การตรวจสอบสถานะร้านไม่ fail-open ร้านที่ถูกกระบังจึงยังถูกปฏิเสธทันที

งานเบื้องหลังใช้ BullMQ [27] บน `redis-queue` โดย `TenantJobRunner` ใช้เพดานเวลาคอมมิตเดียวกับ `runTx` และมีคิวงานล้มเหลว (dead-letter queue) การ export ข้อมูลรายร้านเขียนไฟล์ลงไวลุ่มที่ใช้ร่วมกันระหว่าง `worker` กับ `api` แทนการฝังข้อมูลทั้งชุดลงในงานของคิว เพราะ `redis-queue` จำกัดหน่วยความจำไว้ที่ 192 MB แบบ `noeviction` ส่วน etcd เก็บเฉพาะค่าตั้งขณะรันที่ปรับได้โดยไม่ต้อง deploy ใหม่ เช่น `log_level` ตามที่ ADR-0013 กำหนด

3.4.9 การวัดผลและสังเกตการณ์

`GET /metrics` คืนข้อความรูปแบบ Prometheus [36] และ Nginx ตอบ 404 ต่อภายนอกสำหรับเส้นทางนี้ ตัวชี้วัดหลักคือ `http_requests_total` และ `http_request_duration_seconds` ซึ่งชื่อถูกอ้างโดยแผงใน Grafana หากเปลี่ยนชื่อ แผงจะว่างโดยไม่มีข้อผิดพลาด การนับทำใน middleware เพื่อให้คำขอที่ guard ปฏิเสธ (401 และ 429) ถูกนับด้วย และ label `route` เป็น pattern ไม่ใช่พารามิเตอร์ เส้นทาง `/metrics`, `/health/live` และ `/health/ready` ถูกตัดออกจาก SLI เพราะการ scrape และ healthcheck สร้างคำขอที่สำเร็จแน่นอนราว 36 ครั้งต่อวินาที ซึ่งทำให้วันที่ทุกบิลจริงล้มเหลวยังอ่านอัตราสำเร็จได้ราวร้อยละ 92 (เจ้าของโครงการอนุมัติเมื่อ 21 กันยายน พ.ศ. 2569) ตัวนับ `pos_idempotency_replay_total` ไม่มี label `tenant_id` เพื่อควบคุมจำนวนอนุกรม

3.5 การซิงค์ข้อมูลเฟส 2

3.5.1 ภาพรวมของการทำงานออฟไลน์

เฟส 2 นำการทำงานออฟไลน์กลับมาโดยยังให้ PostgreSQL เป็นแหล่งความจริง ข้อกำหนดอยู่ใน **08_PHASE2_SPEC.md** ซึ่งเป็นผลของการตัดสินใจของเจ้าของโครงการหลายรอบ เครื่อง **pos** เพียงเครื่องเดียวต่อร้านเป็นผู้เขียนที่ออฟไลน์ได้ ซึ่งตามที่กล่าวในบทที่ 2 เป็นกติกากันป้องกันการขายเกินสต็อกจากสองเครื่องที่เขียนพร้อมกันโดยไม่เห็นกัน โคลเอนต์มีสามสถานะ (Online, Degraded, Syncing) โดย Degraded เกิดเมื่อ **GET /health/ready** ล้มต่อเนื่อง 3 ครั้ง หรือ เข้าเกิน 5 วินาที หรือคำสั่งเขียนไม่ได้คำสั่งตัดสิน ในสถานะ Degraded การเขียนที่อยู่ในรายการ "เข้าคิว" ตามตารางที่ 3.9 จะเข้า **outbox_ops** ส่วนการเขียนที่เป็น "ออนไลน์เท่านั้น" (เช่น สินค้า ใบสั่งซื้อ ใบเสนอราคา การตั้งค่า การยกเลิกบิลออนไลน์ การปิดกะ และการจัดการเครื่อง) จะถูกปิดปุ่ม การพักบิลเป็นข้อมูลในเครื่องอย่างเดียวและไม่ซิงค์

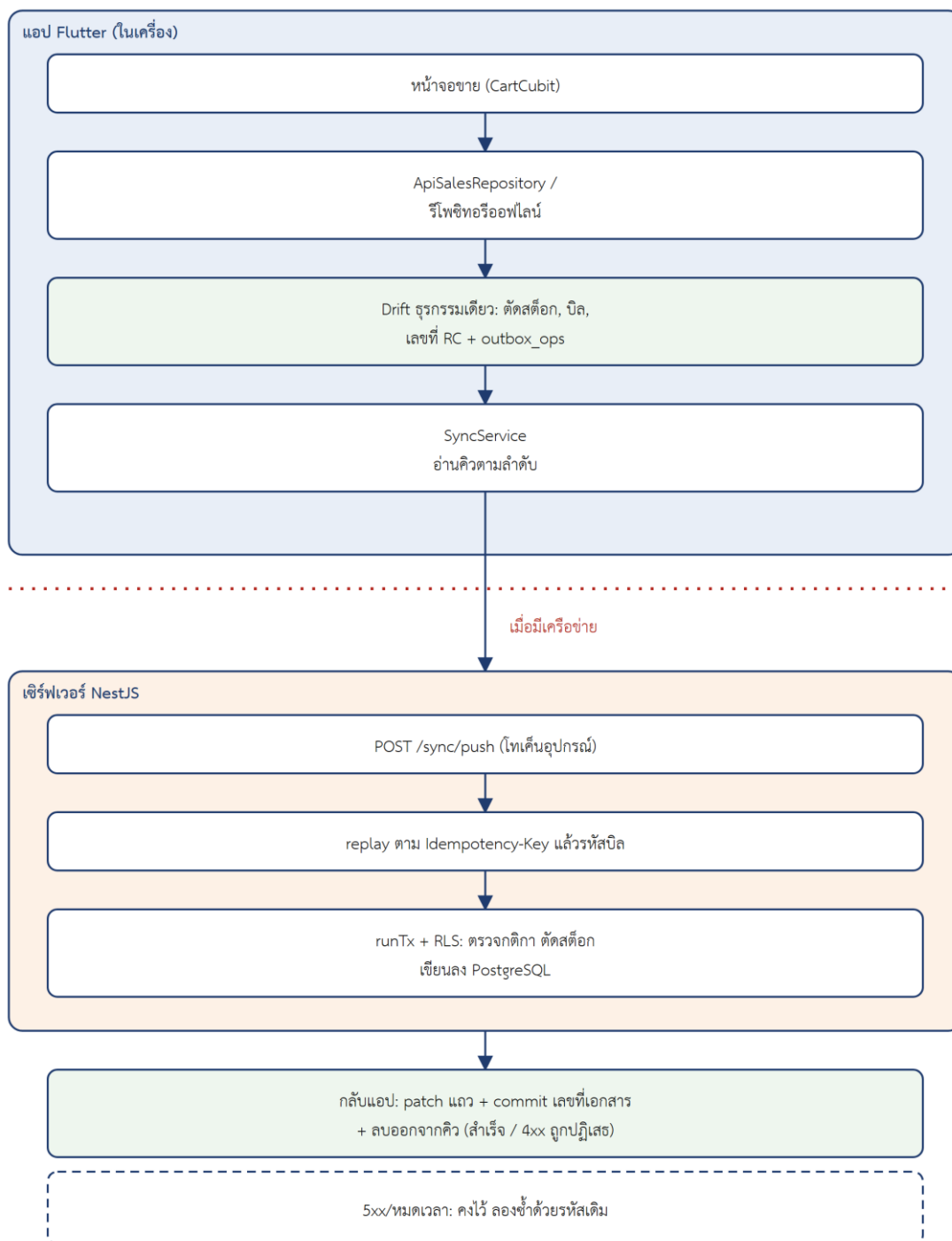
ตารางที่ 3.9 รายการ op ที่เข้าคิวและ endpoint ออนไลน์ที่เป็นคู่กัน

type	endpoint ออนไลน์ที่เทียบเท่า	หมายเหตุ
shift.open	POST /shifts/open	เปิดกะออฟไลน์ได้
sale.create	POST /sales	การขายเกินวงเงินเครดิตถูกส่งไปเป็นรายการตรวจ
return.create	POST /returns	ใบลดหนี้เลข CN ในเครื่อง
drawer.entry	POST /shifts/current/entries	เงินเข้าออกลิ้นชัก
credit_payment.create	POST /mechanics/:id/credit-payments	ย้ายมาจากคิวรับชำระเครดิตเดิม
customer.create, customer.update	POST และ PATCH /customers	เทียบด้วย id อย่างเดียวสำหรับ create
sale.void_offline	ส่งผ่าน push เท่านั้น	ยกเลิกได้เฉพาะบิลที่ขายออฟไลน์

3.5.2 Outbox ในเครื่อง

ตาราง **OutboxOps** ใน Drift เป็นคิวตารางเดียว มีคอลัมน์ **opId** (คีย์หลัก), **idempotencyKey**, **type**, **payload** (เท่ากับเนื้อหาคำขอออนไลน์), **aggregates** (กลุ่มข้อมูลที่คำสั่งนั้นแก้ เช่น บิลหนึ่งใบหรือกะหนึ่งกะ และกลุ่มที่ต้องรอก่อน), **status** (**pending**, **stuck**, **rejected**) และ **attempts** พร้อมรหัสและข้อความผิดพลาดล่าสุด กติกากันทำให้

outbox เชื่อถือได้คือ id และ key ถูกสร้างก่อนส่ง และแถวธุรกรรมกับแถว outbox ถูกเขียนในธุรกรรมเดียวกันของ Drift ถ้าแอปถูกปิดกลางการขาย จะมีทั้งบิลและ op หรือไม่มีทั้งคู่ ดังรูปที่ 3.5



รูปที่ 3.5 การไหลของรูปแบบกล่องขาออกในโครงการ ตั้งแต่การขายออฟไลน์จนถึงการเล่นซ้ำที่เซิร์ฟเวอร์

3.5.3 POST /sync/push

เมื่อเครือข่ายกลับมา **SyncService** ส่ง op เป็นชุดละไม่เกิน 50 รายการด้วยส่วนหัว **X-Device-Token** ฝั่งเซิร์ฟเวอร์ประมวลผลทีละ op ตามลำดับดังนี้

1. เรียก **runIdempotent** ด้วยคีย์และ fingerprint เดียวกับเส้นทางออนไลน์ ถ้าเคยทำแล้วตอบ **applied**
2. ตรวจสอบซ้ำด้วยรหัสที่ไคลเอนต์สร้าง (เช่น **sales.id**) ในธุรกรรมเดียวกัน ถ้า id ซ้ำแต่เนื้อหาไม่ตรง ตอบ **rejected**
3. เรียกบริการเดียวกับ controller ออนไลน์ด้วยลำดับล็อกเดิม ถ้าบริการปฏิเสธ ตอบ **rejected**
4. ทำผลพลอยได้ในธุรกรรมเดียวกัน เช่น การตั้งธงว่าขายออฟไลน์ การปรับวันที่ที่ผิดปกติ และการสร้างรายการตรวจ

ทุก op มี **runTx** ของตัวเองต่อกันทีละตัว เมื่อ op ใดได้ผลที่ไม่แน่นอน (คำขอเดิมยังค้างเกินเพดานเวลา หรือ 5xx) เซิร์ฟเวอร์ตอบ **retry** แล้วหยุดชุดนั้น

ฝั่งไคลเอนต์ส่งทีละคำขอ เมื่อได้ **retry** จะนับ **attempts** และเมื่อครบ 3 ครั้ง op จะเป็น **stuck** และปรากฏในหน้ารายการรอเจ้าของร้านตรวจ op ที่อยู่ใน aggregate เดียวกับ op ที่ติดจะรอต่อเนื่องกันตามลำดับ ส่วน op อื่นยังไปต่อได้

ตัวอย่างเช่น คิว [เปิดกะ S] [ขาย A] [ยกเลิก A] [ขาย B] ถ้าขาย A ติด การยกเลิก A จะรอแต่ขาย B ส่งต่อได้ ทว่าถ้าการเปิดกะ S ติด ทุก op ในกะ S จะรอ

เจ้าของร้านเลือกส่งใหม่ด้วยคีย์เดิมหรือทิ้งพร้อมหมายเหตุผ่าน **POST /sync/discards** และการทิ้งใช้ตาราง **op_effects** ย้อนผลเฉพาะที่ได้ใช้จริง การปิดกะต้องรอให้คิวว่างทั้งหมด **ApiShiftsRepository.closeShift** จึงส่งคิวที่ส่งได้ก่อน แล้วปฏิเสธด้วย **OUTBOX_NOT_EMPTY** เมื่อยังมีแถวใน **outbox_ops** ไม่ว่าสถานะหรือชนิดใด การตรวจนี้ทำให้ไคลเอนต์เท่านั้นเพราะเซิร์ฟเวอร์มองไม่เห็นคิวในเครื่อง

3.5.4 เลข RC และ CN ที่ออกบนเครื่อง

ADR-0007 กำหนดเลขเอกสารแบบเรียงต่อเครื่องและรีเซ็ตรายเดือน รูปแบบ **RC01-2569-08-0042** คือ prefix ตามชนิดเอกสาร (RC ใบเสร็จ, CN ใบลดหนี้, PO ใบสั่งซื้อ, QT ใบเสนอราคา, CP ใบรับชำระเครดิตข้าง) ตามด้วยหมายเลขเครื่องสองหลัก ปี พ.ศ. เดือน และลำดับหลัก การเติมศูนย์ให้หมายเลขเครื่องทำให้ **RC1-** กับ **RC12-** ไม่อ่านกำกวม ในเฟส 1 เซิร์ฟเวอร์ออกเลขทั้งหมดด้วย **UPDATE doc_counters ... RETURNING** ได้ row lock ในธุรกรรมเดียวกับการขาย ในเฟส 2 เครื่อง **pos** ออก RC และ CN เองทั้งออนไลน์และออฟไลน์ ส่วน PO, QT

และ CP เซิร์ฟเวอร์ยังออกให้เสมอ เพราะเครื่อง **backoffice** ออนไลน์เสมอและไม่ได้ประโยชน์จากการออกเลขเอง

เซิร์ฟเวอร์ตรวจว่า prefix ตรงกับชนิดเอกสาร และหมายเลขเครื่องตรงกับเครื่องจากโทเค็น เก็บ high-water mark ด้วย **INSERT ... ON CONFLICT ... DO UPDATE SET last_no = GREATEST(...)** และข้อจำกัด UNIQUE บน **receipt_no** และ **cn_no** เป็นด้านสุดท้ายกัน เลขซ้ำ เมื่อออฟไลน์ข้ามเดือน เลขเริ่มที่ 0001 ใหม่ตามนาฬิกาเครื่อง ส่วนเครื่องที่ยังไม่เคยเตรียมตัวนับ (เพิ่ง enrol หรือเพิ่งอัปเดต) ถูกห้ามออกเลขออฟไลน์ด้วยรหัส **OFFLINE_SEED_REQUIRED** กติกาสำคัญอีกข้อคือเลข RC หรือ CN ที่เซิร์ฟเวอร์ออก ทั้งในคำตอบออนไลน์และจากการเล่นซ้ำของ **/sync/push** ต้องถูกบันทึกลงตัวนับในเครื่องผ่าน **DocNumberService.commitServerIssued** มิฉะนั้นเครื่องอาจออกเลขซ้ำกับที่เซิร์ฟเวอร์เคยให้

3.5.5 การดึงข้อมูลแบบ keyset

หลังส่งคิวหมด ไคลเอนต์ดึงข้อมูลใหม่จากเซิร์ฟเวอร์ (push ก่อน pull เสมอ) โดยใช้ **meta.nextCursor** ของเซิร์ฟเวอร์ที่เก็บในตาราง **SyncCursors** ต่อชนิดข้อมูล เหตุผลที่ต้องเป็น keyset แทน offset หรือ timestamp เปล่า คือหลายแถวอาจมี **updated_at** เท่ากันจาก **now()** ของธุรกรรมเดียว และ timestamp ระดับมิลลิวินาทีจะซ้ำหรือซ้ำแถวที่ใช้ค่าเดียวกัน cursor จึงมีความละเอียดระดับไมโครวินาทีร่วมกับ **afterId** ส่วนหน้าแรกของแต่ละรอบไคลเอนต์ถอย cursor กลับ 30 วินาทีและไม่ส่ง **afterId** เพื่อรองรับธุรกรรมที่ commit ช้ากว่าที่ cursor ผ่านไปแล้ว ซึ่งปลอดภัยเพราะเพดานเวลา 25 วินาทีทำให้ไม่มีธุรกรรมใดค้างนานกว่านั้น แถวที่ถูกลบเป็นสัญลักษณ์ลบ (tombstone) ให้ไคลเอนต์ซ่อน การดึงข้อมูลไม่ทับสต็อกของสินค้าที่ยังมี op ค้าง และไม่ทับยอดลูกค้าหรือช่างที่ถูกอ้างโดย op ทางการเงินที่ยังไม่ส่ง ตามหลักความสอดคล้องในที่สุดที่กล่าวในบทที่ 2

3.6 ความปลอดภัย

มาตรการด้านความปลอดภัยอยู่หลายชั้น ตารางที่ 3.10 รวบรวมไว้พร้อมที่มา ส่วนการสำรองข้อมูลออกนอกเครื่องเสมือนยังไม่ได้ทำ (สถานะรายงานในบทที่ 4)

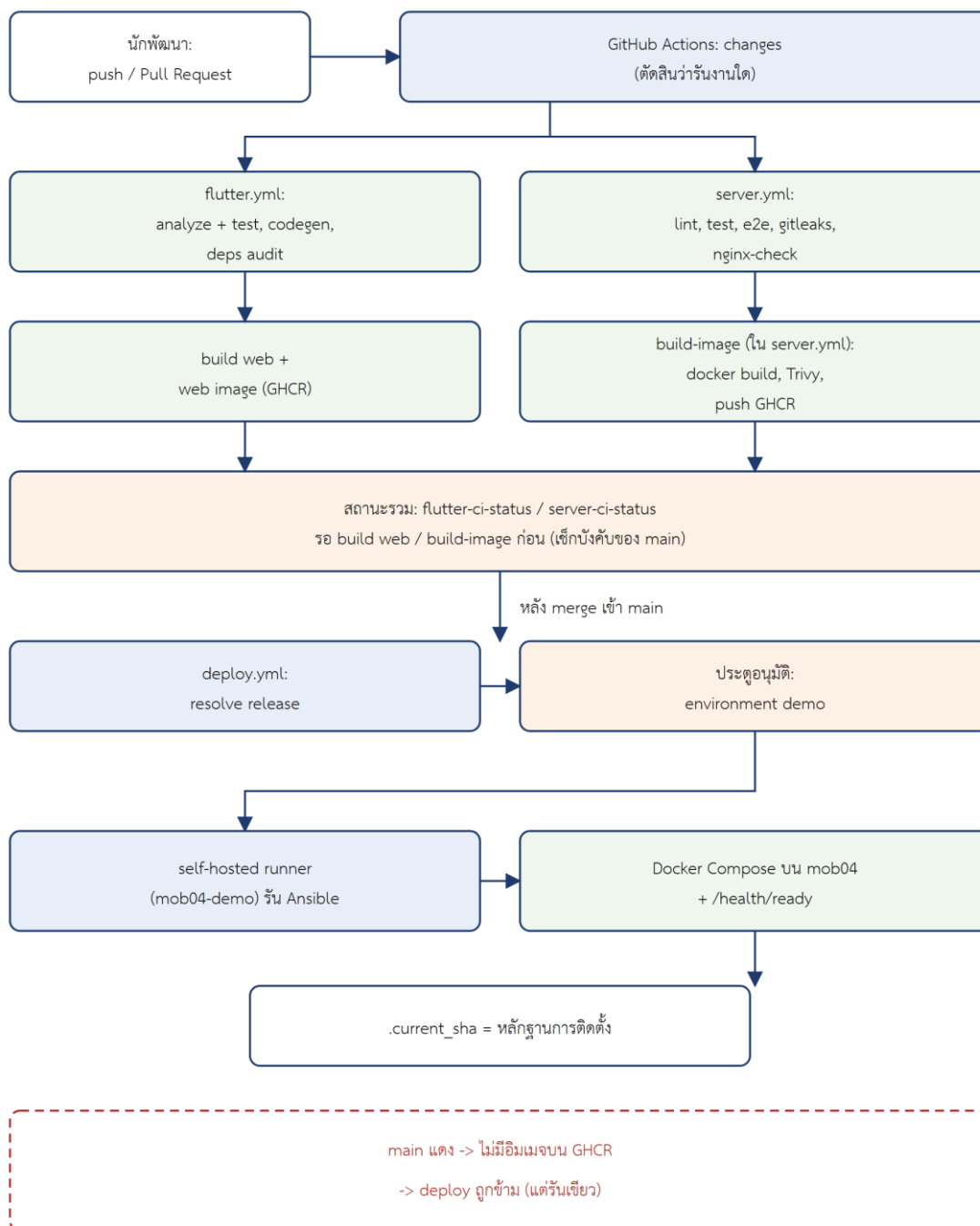
ตารางที่ 3.10 มาตรการด้านความปลอดภัยและที่มา

ชั้น	มาตรการ	เหตุผล / ที่มา
การแยกผู้เช่า	RLS แบบ ENABLE และ FORCE บนตารางธุรกิจ; บทบาท <code>pos_app</code> ไม่ใช่ <code>superuser</code>	ฐานข้อมูลบังคับแม่โด้ดแอปพลิเคชัน (T1, ADR-0003)
การยืนยันตัวตน	JWT RS256; access 15 นาที; refresh หมดอายุ 04:00; รหัสผ่านแฮชด้วย <code>argon2</code>	ADR-0009
โทเค็นบนเว็บ	โทเค็นเข้าถึงในหน่วยความจำ, โทเค็นต่ออายุหรือโทเค็นอุปกรณ์ใน IndexedDB, ห้าม <code>localStorage</code>	ภาคผนวก ADR-0009 (25 ก.ย. 2569)
งานเสี่ยง	การถอดถอน การ <code>enrol</code> และการ <code>export</code> ต้องมี <code>device token</code> ที่ <code>enrol</code> แล้ว	สเปกเฟส 2
ระนาบผู้ดูแล	ตาราง <code>platform_admins</code> แยก, <code>aud</code> ต่างกัน, การเชื่อมต่อที่ข้าม RLS แยก, Nginx จำกัด IP	ADR-0002
การเขียนซ้ำ	<code>Idempotency-Key</code> พร้อม <code>fingerprint</code> ของพาจจริง	กันบิลซ้ำเมื่อคำตอพบาย
การจำกัดอัตรา	Nginx ต่อ IP และ <code>guard</code> ต่อผู้เช่า; เข้าสู่ระบบนับก่อนรู้ผลด้วย Lua <code>INCR</code> อะตอมิก	ADR-0006
บันทึกตรวจสอบ	<code>audit_log</code> เพิ่มได้อย่างเดียวสำหรับ <code>pos_app</code>	migration <code>AuditLogAppendOnly</code>
ความลับ	ค่าตัวอย่างสาธารณะทำให้แอปไม่ยอมเริ่มทำงาน เว้นแต่ <code>ALLOW_DEV_SECRETS=true</code> ซึ่งห้ามตั้งบนเครื่องจริง	#398, #410
อิมเมจและ dependency	อิมเมจตริงด้วย <code>digest</code> ; Trivy สแกนและบล็อกการ <code>push</code> ; <code>pnpm audit</code> ; <code>gitleaks</code> สแกนความลับ	ADR-0013
ส่วนหัวและต้นทาง	<code>helmet</code> ; Cross-Origin Resource Sharing (CORS) ตาม <code>CORS_ORIGINS</code> ที่ต้องไม่ว่าง	#367

3.7 การนำระบบขึ้นใช้งาน (deploy)

3.7.1 โครงสร้างการ deploy และระดับของ CI/CD

สภาพแวดล้อมจริงเพียงแห่งเดียวคือเครื่องเสมือนของคณะชื่อ **mob04** (สภาพแวดล้อม **demo**) ตาม ADR-0013 ซึ่งอยู่ในเครือข่ายมหาวิทยาลัย Docker Compose ประกอบจากสามไฟล์ คือ **server/docker-compose.yml** (ฐาน), **deploy/compose/vm.override.yml** (สลับเป็นอิมเมจจาก GHCR ตามแท็กของคอมมิต) และ **deploy/compose/monitoring.yml** (Prometheus, Grafana, Node Exporter) ไปป์ไลน์ทั้งหมดแสดงดังรูปที่ 3.6



รูปที่ 3.6 ภาพรวมไปป์ไลน์ CI/CD ของโครงการ ตั้งแต่คำขอรวมถึงการติดตั้งบนเครื่องสาธิต

CI/CD บน GitHub Actions [31] แบ่งเป็นสี่ระดับ ระดับ 1 คือ Flutter CI (**flutter.yml: analyze-and-test, deps-audit, codegen-check, build-web**) ระดับ 2 คือ backend CI (**server.yml: secrets, lint, audit, unit, nginx-check, integration, build-image**) ระดับ 3 คือการสร้างอิมเมจลง GHCR พร้อม Trivy ที่บล็อกเมื่อพบช่องโหว่ระดับสูง และระดับ 4 คือการ deploy ด้วย Ansible (**deploy.yml**) ทั้งสองเวิร์ก

ไฟล์แรกถูกเรียกทุก push และ pull request โดยไม่กรองพาที่ขึ้นนอก แต่มีงาน **changes** ภายในตัดสินใจว่าจะรันงานใด และลงท้ายด้วยงานสถานะที่รายงานผลเสมอ (**flutter-ci-status**, **server-ci-status**) ซึ่งเป็นเงื่อนไขบังคับของสาขา **main** เพียงสองรายการ เหตุผลคือการตรวจสอบสถานะที่บังคับต้องรายงานผลทุกครั้ง หากกรองพาที่ขึ้นนอก PR ที่แก้ไขเฉพาะ **server/** จะไม่มีสถานะของ Flutter CI ให้ผ่าน push ที่แก้ไขเฉพาะเอกสาร (***.md** หรือ **docs/****) ข้ามการทดสอบและการสร้างอิมเมจ เครื่องเสมือนจึงอยู่ที่ SHA ของโค้ดล่าสุดเสมอ

3.7.2 Ansible และ self-hosted runner

playbook **deploy.yml** ของ Ansible [35] ทำตามลำดับ ตรวจสอบ SHA ที่รันอยู่และออกก่อนเมื่อซ้ำ ซึ่งก็ไฟล์ Compose และ Nginx ดึงอิมเมจจาก GHCR รัน migrations ผ่านคอนเทนเนอร์ **migrate** (สคีม่าก่อนโค้ด) รีสตาตแบบหมุนเวียน **api-1** → **api-2** → **api-3** พร้อมตรวจสอบสุขภาพ รีสตาต worker และ bull-board อัปเดตไฟล์เว็บแล้วสร้าง Nginx ใหม่ ตรวจสอบ **GET /health/ready** ผ่าน Nginx บันทึก SHA ลง **/opt/pos/.current_sha** และสุดท้ายเริ่มชุดมอนิเตอร์ ซึ่งหากล้มเหลวจะเพียงแจ้งเตือน migration รันเป็นงานครั้งเดียวก่อน api ทุกตัว ไม่ใช่ต่อบนชุด เพราะอินสแตนซ์สามตัวจะแย่งกัน เมื่อ readiness ล้มเหลว สคริปต์ **deploy** จะติดตั้งรุ่นก่อนหน้ากลับอัตโนมัติ

เครื่องเสมือนอยู่หลังเครือข่ายมหาวิทยาลัยที่ตัวรันของ GitHub เข้าไม่ถึง ทีมจึงติดตั้ง self-hosted runner บน **mob04** และสภาพแวดล้อม **demo** กำหนดให้ผู้ตรวจที่ระบุอนุมัติก่อนงาน **deploy** จะแตะเครื่อง (ภาคผนวกของ ADR-0013) การ trigger อัตโนมัติทุกครั้งที่มี **main** ผ่านยังคงอยู่ แต่งานจะค้างในสถานะรออนุมัติ ซึ่งเป็นกลไกเดียวที่กั้นการ merge ไม่ให้ไปทับเครื่องระหว่างการสาธิต ผลการ deploy จริงรายงานในบทที่ 4

3.8 เทคโนโลยีที่ใช้

ตารางที่ 3.11 รวบรวมเทคโนโลยีและเวอร์ชันของไลบรารี โดยเป็นช่วงเวอร์ชันที่ประกาศใน **frontend/pubspec.yaml** และ **server/package.json** ส่วนเวอร์ชันของอิมเมจโครงสร้างพื้นฐานแสดงไว้แล้วในตารางที่ 3.3

ตารางที่ 3.11 เทคโนโลยีที่ใช้ในโครงการ

ชั้น	เทคโนโลยี	เวอร์ชัน	วัตถุประสงค์
ไคลเอนต์	Flutter / Dart	Flutter 3.44.3 (ใน CI); Dart SDK ^3.12.2	โค้ดชุดเดียวหลายแพลตฟอร์ม (CI สร้างเฉพาะเว็บ)
ไคลเอนต์	Drift + sqlite3	drift ^2.34.0; sqlite3 ^3.4.0 (ทดสอบ)	ฐานข้อมูล SQLite ในรูปแบบ type-safe
ไคลเอนต์	flutter_bloc / bloc, go_router, http	^9.1.1 / ^9.2.1, ^17.3.0, ^1.5.0	จัดการสถานะ กำหนดเส้นทาง และเรียก API
ไคลเอนต์	pdf, printing, barcode_widget	^3.13.0, ^5.15.0, ^2.0.4	ใบเสร็จ ใบเสนอราคา A4 ป้ายบาร์โค้ด
ไคลเอนต์	ฟอนต์ Sarabun, BarlowCondensed	ฝังเป็น asset (OFL)	แสดงผลภาษาไทยและสร้าง PDF โดยไม่ต้องใช้เครือข่าย
เซิร์ฟเวอร์	Node.js, pnpm	>=22 (node:22-alpine), 10.34.5	รันไทม์และจัดการแพ็คเกจ
เซิร์ฟเวอร์	NestJS, Express, helmet	^12.0.1, ^5.2.1, ^8.3.0	เฟรมเวิร์ก API แบบโมดูลและส่วนหัวความปลอดภัย
เซิร์ฟเวอร์	TypeORM, pg	^1.1.1, ^8.23.0	เชื่อมต่อ PostgreSQL และรัน migrations
เซิร์ฟเวอร์	BullMQ, ioredis	^6.3.4, ^6.0.0	คิวงานเบื้องหลังและไคลเอนต์ Redis
เซิร์ฟเวอร์	argon2, jsonwebtoken	^0.45.1, ^9.0.3	แฮชรหัสผ่านและออก JWT
เซิร์ฟเวอร์	pino, prom-client	^10.3.1, ^15.1.3	บันทึกแบบมีโครงสร้างและตัวชี้วัด Prometheus
เซิร์ฟเวอร์	TypeScript, vitest, oxlint	^6.0.2, ^4.1.2, ^1.58.0	ภาษา ทดสอบ และตรวจโค้ด
ทดสอบโหลด	k6	ตามสคริปต์ใน server/test/k6/	วัดโหลดและสภาวะแข่งกันบนสต็อก
CI/CD	GitHub Actions, GHCR, Trivy, gitleaks, Ansible	ตามไฟล์ workflow	สร้าง ตรวจสอบ เผยแพร่ อิมเมจ และ deploy

3.9 วิธีการทำงานของทีม

3.9.1 การแบ่งงานและการติดตามงาน

ทีมสามคนแบ่งเลนตามตารางที่ 1.4 ในบทที่ 1 และติดตามงานด้วยตัวงานบน GitHub (issue) ตัวงานหลัก (parent issue) ไม่ถูกลงมือโดยตรง และตัวงานที่เป็นการตัดสินใจของเจ้าของโครงการไม่ถูกตัดสินใจใน pull request สำหรับเฟส 2 เอกสาร `09_PHASE2_LANES.md` แบ่งตัวงาน 35 ตัวเป็นสามเลน โดยเลน B ดูแลกลไกฝั่งเครื่องทั้งหมด เลน C ดูแลเซิร์ฟเวอร์ หน้าจอใหม่ และงานปฏิบัติการ ส่วนเลน A มีตัวงานปลดล็อกสามตัว หลักสำคัญคือ "การรอกันไม่ข้ามเลน" กล่าวคือความต้องการข้ามเลนต้องกำหนดเป็นสัญญา (contract) เช่น ไฟล์ตัวอย่างคำขอ `/sync/push` และอินเทอร์เฟซ `SyncFacade` ไม่ใช้การรอกัน ตัวงานที่ถูกแบ่งเป็นสองส่วน (ไคลเอนต์กับเซิร์ฟเวอร์) ไม่ถือว่าทำงานกับของจริงได้จนกว่าอีกส่วนจะรวมโค้ด แต่ละฝั่งจึงทดสอบกับตัวจำลอง (fake) ของฝั่งตนเอง

3.9.2 ADR และลำดับความสำคัญของเอกสาร

การตัดสินใจเชิงสถาปัตยกรรมบันทึกเป็น ADR หนึ่งไฟล์ต่อหนึ่งเรื่องตามแนวทางของ Nygard [38] ปัจจุบันมี 13 หมายเลข (0001 ถึง 0013) ครอบคลุมการสร้างผู้เช่า ระบายผู้ดูแล วงจรชีวิตผู้เช่า บทบาทอุปกรณ์ การส่งออกข้อมูล ชิดจำกัดอัตรา เลขที่ใบเสร็จ ต้นทุน ณ เวลาขาย อายุเซสชัน การเขียนผ่านแคช โมโนรีโป CouchDB (ถูกปฏิเสธ) และชุดเครื่องมือ CI/CD กติกาของทีมคือเมื่อเอกสารขัดกับ ADR ให้ยึด ADR และ ADR ที่เปลี่ยนใจภายหลังใช้ภาคผนวกลงวันที่แทนการเขียนทับประวัติ

3.9.3 ขั้นตอนการพัฒนาและการตรวจ

ทุกงานเข้า `main` ผ่าน pull request ที่ต้องผ่านการตรวจสอบสถานะบังคับสองรายการ การป้องกันสาขาตั้งแต่ 15 กันยายน พ.ศ. 2569 ห้าม force-push และห้ามลบสาขา แต่ไม่ได้กำหนดจำนวนผู้อนุมัติขั้นต่ำ (ตั้งไว้ 0 คน) ที่เก็บโค้ดมีสาขายาวสองสาขาคือ `main` และ `POC_sample_offline_first` ตัวงานแต่ละตัวเริ่มด้วยการทบทวนแนวทางจากมุมมองคนนอก (ตั้งคำถามว่าเจตนาถูกหรือไม่และมีทางที่ง่ายกว่าหรือไม่ แล้วตามรอยเส้นทางโค้ดจริง) แก่โค้ดเท่าที่จำเป็นพร้อมเกณฑ์ที่ตรวจได้ และปิดด้วยการทบทวนโค้ดสองแกน คือมาตรฐานของที่เก็บโค้ด และความตรงกับสเปกของ

ตัวงานต้นทาง บทเรียนจากกระบวนการนี้ถูกบันทึกกลับเป็นกติกาของโครงการ ซึ่งกรณีสำคัญสรุปไว้ในบทที่ 5

ในทางปฏิบัติ PR ที่รวมแล้ว 330 รายการถูกรวมโดยผู้เขียน PR เองทั้งหมดยกเว้นหนึ่งรายการ และไม่มีรายการใดได้รับการอนุมัติผ่านฟังก์ชันรีวิวของ GitHub จากสมาชิกคนอื่น การตรวจสอบก่อนรวมจึงอาศัยการตรวจสอบสถานะของ CI ที่บังคับ ความเห็นจากบริการรีวิวโค้ดอัตโนมัติ CodeRabbit ซึ่งแสดงความเห็นใน 328 PR และการทบทวนโค้ดด้วยเครื่องมือช่วยพัฒนาที่อธิบายในหัวข้อถัดไป

3.9.4 เครื่องมือช่วยพัฒนาด้วยปัญญาประดิษฐ์

ทีมใช้ Claude Code ซึ่งเป็นผู้ช่วยเขียนโค้ดด้วยปัญญาประดิษฐ์ ในการสร้างโค้ดและชุดทดสอบ การทบทวนโค้ด และการร่างเอกสาร จากคอมมิตบน `main` ทั้งหมด 1,058 รายการ มี 591 รายการ (ร้อยละ 56) ที่มีบรรทัด **Co-Authored-By: Claude** ระบุผู้ร่วมเขียน และหากไม่นับคอมมิตที่เป็นการ merge มี 567 จาก 706 รายการ (ร้อยละ 80) ตัวเลขนี้นับจากข้อความคอมมิต จึงบอกได้เพียงว่าคอมมิตใดมีเครื่องมือมีส่วนร่วม ไม่ได้วัดสัดส่วนของโค้ดที่เครื่องมือเขียน งานที่สมาชิกทีมทำเอง ได้แก่ การกำหนดความต้องการ การตัดสินใจเชิงออกแบบที่บันทึกเป็น ADR การกำหนดและตรวจรับเกณฑ์ของตัวงาน การรวม PR และการอนุมัติการ deploy บนสภาพแวดล้อม **demo** ผลลัพธ์ของเครื่องมือถูกตรวจด้วยกลไกเดียวกับโค้ดอื่น คือ CI ที่บังคับ ชุดทดสอบ e2e และการนับเกณฑ์ตรวจรับซ้ำกับไฟล์ทดสอบ

บทที่ 4

ความก้าวหน้าการดำเนินงาน

บทนี้รายงานผลการดำเนินงาน ณ วันที่ 1 ตุลาคม พ.ศ. 2569 โดยส่วนแรกครอบคลุมงานฝั่งไคลเอนต์ Flutter ได้แก่ การย้ายระบบเดิม หน้าจอ ขึ้นเขียนข้อมูลผ่าน API ระบบทำงานออฟไลน์ฝั่งอุปกรณ์ และผลการทดสอบ ส่วนที่สองครอบคลุมเซิร์ฟเวอร์ CI/CD การ deploy และเกณฑ์ตรวจรับของเฟส 1 การออกแบบอยู่ในบทที่ 3 บทนี้จึงเน้นสิ่งที่ทำได้จริงและหลักฐาน

ตัวเลข วันที่ และสถานะทุกรายการตรวจสอบย้อนกลับได้จากประวัติ git ตั้วงาน หรือผลการรันชุดทดสอบ งานที่ยังไม่เสร็จจะบ่งว่ายังไม่เสร็จ คำว่า "ผ่านการรวมโค้ด" (merged) หมายถึงคำขอรวมโค้ด (pull request: PR) ถูกรวมเข้าสาขา **main** แล้ว และ "ปิดตั้ว" หมายถึงตั้วงานมีสถานะ closed บน GitHub

4.1 ผลการย้ายระบบเดิมสู่ Flutter

4.1.1 ลำดับความก้าวหน้าของการย้ายระบบ ชั้น 0 ถึง ชั้น 6

การย้ายระบบแบ่งเป็นชั้น 0 ถึง 6 (ไม่ใช่เฟส 1–2 ของเซิร์ฟเวอร์) ผลที่ตรวจสอบได้จากประวัติ git แสดงในตารางที่ 4.1 งานช่วงแรกเกิดขึ้นในวันเดียวคือ 23 มิถุนายน พ.ศ. 2569 เพราะโครงสร้างและข้อกำหนดถูกเขียนไว้ล่วงหน้าในเอกสารสัญญา **CONTRACT.md** ซึ่งระบุตารางลายเซ็นของ repository เส้นทาง และความเป็นเจ้าของหน้าจอย่อย

ตารางที่ 4.1 ลำดับการย้ายระบบเดิมสู่ Flutter (ตรวจสอบจากประวัติ git)

วันที่	งาน	หลักฐาน (commit)
23 มิ.ย. 2569	ขั้น 0: สร้างโครงโปรเจกต์ Flutter	5f8e8163
23 มิ.ย. 2569	ขั้น 1: ชั้นข้อมูลด้วย Drift และ repository แบบธุรกรรม พร้อมการทดสอบหน่วย	ea9a9778
23 มิ.ย. 2569	ขั้น 3-6: หน้าจอทั้งหมดในรอบแรก ชุดวิดเจ็ตกลาง เมนูนำทาง และชั้นเปิดปิดกะ	8eac3bfb
23-24 มิ.ย. 2569	แก้ไขข้อค้นพบจากการตรวจสอบเชิงวิพากษ์ และเชื่อมฐานข้อมูล Drift บนเว็บ	453bc3f5, 621f86c1, 6daf8d20
25-26 มิ.ย. 2569	ปรับหน้าจอสำหรับแท็บเล็ตและมือถือ และคู่มือผู้ใช้พร้อมภาพหน้าจอจริง	11d43adb, 594d75db, bca4dc1a
2-10 ก.ค. 2569	ตัวปรับขนาดตัวอักษร ฟอนต์ไทย ในใบเสนอราคา A4 และการปรับโครงสร้างโค้ดพร้อมการทดสอบ repository	a110a672, 19d3191c, 279b90ac
14 ก.ค. 2569	ย้ายการจัดการสถานะจาก Riverpod เป็น flutter_bloc	d8344503

การตรวจสอบเชิงวิพากษ์ (adversarial scrutiny) หลังขั้น 6 พบข้อบกพร่องหลายประเด็น เช่น การส่งออกข้อมูลบนเว็บ รายงานปิดร้านที่ไม่ปรับตามขนาดหน้าจอ และปัญหาข้อความล้นกรอบ (overflow) บนจอโทรศัพท์ เมื่อแก้แล้วระบบผ่าน **dart analyze** โดยไม่มีข้อผิดพลาดและชุดทดสอบผ่านทั้งหมด ก่อนเริ่มงานเชื่อมต่อกับเซิร์ฟเวอร์

การย้ายไลบรารีจัดการสถานะจาก Riverpod เป็น flutter_bloc ดำเนินการครบ 12 ขั้นตอนตามแผน เนื่องจากชั้น repository ไม่ได้ผูกกับ Riverpod การย้ายจึงกระทบเฉพาะชั้นการแสดงผล ขณะนั้นชุดทดสอบ 122 กรณีผ่านทั้งหมด

4.1.2 การนำเข้าข้อมูลสำรองและฐานข้อมูลบนเว็บ

importLegacyBackup() นำเข้าไฟล์ JSON ที่ส่งออกจากระบบ JavaScript เดิมในธุรกรรมเดียว พร้อมแปลงข้อมูลโชนเป็นหมวดหมู่ ร้านจึงย้ายข้อมูลเดิมได้โดยไม่ต้องป้อนข้อมูลใหม่

บนบิลด์ที่ใช้เซิร์ฟเวอร์ยังมี `purgeDemoSeed()` ที่ลบข้อมูลตัวอย่าง (seed data) ซึ่งระบบสร้างไว้ตอนเริ่มต้นออกหนึ่งครั้ง โดยข้ามเมื่อยังมียางค้ำในคิวส่ง และลบเฉพาะแถวตัวอย่างที่ไม่เคยถูกแก้ไขและไม่มีระเบียบในอ้างอิง

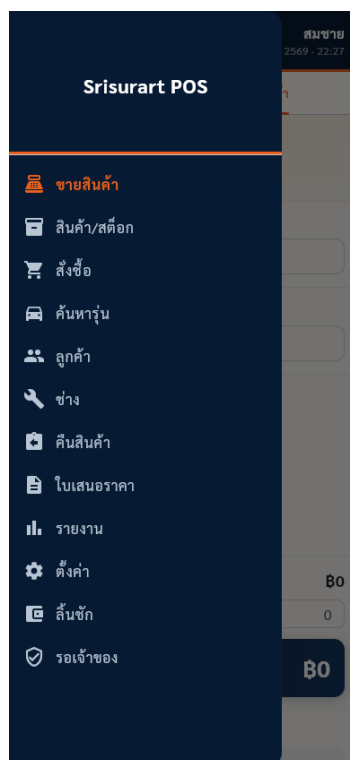
ฐานข้อมูลบน Flutter Web ทำงานผ่าน SQLite ที่คอมไพล์เป็น WebAssembly [5] ปัญหาที่พบคือเบราว์เซอร์ที่ไม่รองรับความสามารถ `dedicatedWorkersInSharedWorkers` เกิดข้อผิดพลาด `LinkError` เมื่อ Drift ถอยไปใช้โหมดสำรอง (#266) ซึ่งแก้ด้วย PR #310 (รวมโค้ด 17 กันยายน พ.ศ. 2569 ปิดตัว 19 กันยายน พ.ศ. 2569)

4.2 หน้าจอของระบบ

ไคลเอนต์มีไฟล์หน้าจอ 14 ไฟล์ตามตารางที่ 3.5 รวมโค้ดหน้าจอประมาณ 20,800 บรรทัด (ผลรวมจาก `wc -l`) ภาพประกอบในหัวข้อนี้มาจากคู่มือผู้ใช้ `docs/tutorial/sri-pos-manual/` ซึ่งถ่ายจากแอปที่ทำงานจริงบนร้านสาธิต บางภาพมีข้อมูลตัวอย่างของนักพัฒนา และบางภาพถ่ายก่อนการแก้ไขข้อบกพร่องซึ่งระบุไว้ได้ภาพนั้น

4.2.1 เมื่อนำทางและหน้าขายสินค้า

เมนูหลัก 12 หน้าอยู่ด้านซ้ายบนจอกว้าง และซ่อนในปุ่มเมนูบนจอแคบ ดังรูปที่ 4.1 บนโทรศัพท์ หน้าขายสินค้ายังแยกเป็นแท็บ "สินค้า" และ "ตะกร้า" เพื่อให้ใช้ด้วยมือเดียวได้



รูปที่ 4.1 เมื่อนำทางบนโทรศัพท์ เรียงครบ 12 หน้า

หน้าขายสินค้า (`checkout_screen.dart` ขนาด 2,956 บรรทัด) เป็นหน้าที่พนักงานใช้ทุกวัน แบ่งเป็นรายการสินค้าตรงกลาง ตะกร้าและการชำระเงินด้านขวา และป้ายสถานะการเชื่อมต่อมูวบาบ ดังรูปที่ 4.2 พนักงานคืนสินค้าด้วยชื่อ รหัส หรือบาร์โค้ด ปรับจำนวน เลือกลูกค้า หรือช่าง และใช้ปุ่มลัดรับเงินสำหรับธนบัตรที่ใช้บ่อย เมื่อเลือกช่าง ระบบเปลี่ยนเป็นราคาช่างและเปิดวิธีชำระ "เครดิตช่าง" ดังรูปที่ 4.3 ใบเสร็จเงินสดแสดงเงินทอนและแต้มสะสม ดังรูปที่ 4.4 ส่วนฟังก์ชันพัคบิลให้พนักงานเก็บตะกร้าไว้ชั่วคราวโดยไม่ตัดสต็อก

ศรีสุวิทย์มอเตอร์ (ร้านตัวอย่าง)
Siriratt Autopart (Demo)

ออนไลน์ ๒๗. 27 ก.ย. 2569 - 22:11

ค้นหาสินค้า / รหัสอะไหล่

ทั้งหมด เครื่องยนต์ ไฟฟ้า น้ำมัน เบรก ตัวถัง ช่วงล่าง

Oil Filter กรองน้ำมันเครื่อง HN-15412-KVB ฿885	Spark Plug NGK หัวเทียน NGK NGK-BR8ES-11 ฿120	Drive Chain #52 โซ่ขับ 520 DID-520VX-110 ฿350	Air Filter กรองอากาศ HN-17210-KWB ฿180	Battery 12V 60Ah แบตเตอรี่ 12V 60Ah YT235-B5-GS ฿2,800
Brake Pad Set ผ้าเบรกดุม BK-FA323-R ฿650	LED Headlight H4 ไฟหน้า LED H4 LED-H4-6000K ฿480	Piston Kit STD ชุดลูกสูบ STD TY-13101-QL010 ฿3,200	Fuel Injector หัวฉีดน้ำมัน IS-8971385180 ฿1,800	Wiring Tape เทปพันสายไฟ 3M-08080-TAPE ฿35
Engine Oil 10W40 น้ำมันเครื่อง 10W40 MOTUL-3100-1L ฿220	Disc Brake Rotor จานเบรกดุม FD-AX7-2020-DS ฿1,450	Air Filter Vigo กรองอากาศ วีโก้ 17801-QL040 ฿450	Timing Belt สายพานเวลา 13568-09130 ฿890	Radiator Vigo หม้อน้ำ วีโก้ (เกียจอร์มาตา) 16400-QL160 ฿4,200
Spark Plug NGK BKR6E-11 หัวเทียน NGK BKR6E-11 BKR6E-11 ฿95	Battery GS NS60 แบตเตอรี่ GS NS60 (45Ah) GS-NS60 ฿2,450	Headlight Bulb H4 หลอดไฟหน้า H4 Philips H4-12V-60/55W ฿280	Shell Helix HX8 5V น้ำมันเครื่อง Shell Helix H... HX8-5W30-4L ฿1,290	Caltex Delo Gold น้ำมันเครื่องคัลเท็กซ์ Caltex D... DELO-10W30-7L ฿1,150
Brake Fluid DOT3 น้ำมันเบรก DOT3 (0.5 ลิตร) ฿120	Front Brake Pads D ผ้าเบรกดุมหน้า ดีแม็คซ์ Dometec ฿6	Front Shock Abs โช้คอัพหน้า KVB Foxal-G ฿25	Lower Arm Bushing ยางตีนถ่วง ฿15	Wiper Blade Bosch ใบปัดน้ำฝน Bosch 29 นิ้ว ฿12

ดูสินค้า

สมัครใจ

081-234-5678 - 0 เม็ม

ช่วง (ปรับราคาช่วง)

ค้นหาช่วง / ชื่อเล่น / เบอร์...

รายการสินค้า (3)

Shell Helix HX8 5W-3...	-	1	+	฿1,290
Air Filter Vigo	-	1	+	฿450
Oil Filter Toyota	-	1	+	฿180

ยอดรวม 81,920

ส่วนลด 0

รวมทั้งสิ้น 81,920

ชำระเงิน

เงินสด โฉนด QR

รับเงิน 8...

รูปที่ 4.2 หน้าขายสินค้า แสดงเมนู รายการสินค้า และตะกร้า

ศรีสุวิทย์มอเตอร์ (ร้านตัวอย่าง)
Siriratt Autopart (Demo)

ออนไลน์ ๒๗. 27 ก.ย. 2569 - 22:15

ค้นหาสินค้า / รหัสอะไหล่

ทั้งหมด เครื่องยนต์ ไฟฟ้า น้ำมัน เบรก ตัวถัง ช่วงล่าง

Oil Filter กรองน้ำมันเครื่อง HN-15412-KVB ฿885	Spark Plug NGK หัวเทียน NGK NGK-BR8ES-11 ฿120	Drive Chain #52 โซ่ขับ 520 DID-520VX-110 ฿350	Air Filter กรองอากาศ HN-17210-KWB ฿180	Battery 12V 60Ah แบตเตอรี่ 12V 60Ah YT235-B5-GS ฿2,800
Brake Pad Set ผ้าเบรกดุม BK-FA323-R ฿650	LED Headlight H4 ไฟหน้า LED H4 LED-H4-6000K ฿480	Piston Kit STD ชุดลูกสูบ STD TY-13101-QL010 ฿3,200	Fuel Injector หัวฉีดน้ำมัน IS-8971385180 ฿1,800	Wiring Tape เทปพันสายไฟ 3M-08080-TAPE ฿35
Engine Oil 10W40 น้ำมันเครื่อง 10W40 MOTUL-3100-1L ฿220	Disc Brake Rotor จานเบรกดุม FD-AX7-2020-DS ฿1,450	Air Filter Vigo กรองอากาศ วีโก้ 17801-QL040 ฿450	Timing Belt สายพานเวลา 13568-09130 ฿890	Radiator Vigo หม้อน้ำ วีโก้ (เกียจอร์มาตา) 16400-QL160 ฿4,200
Spark Plug NGK BKR6E-11 หัวเทียน NGK BKR6E-11 BKR6E-11 ฿95	Battery GS NS60 แบตเตอรี่ GS NS60 (45Ah) GS-NS60 ฿2,450	Headlight Bulb H4 หลอดไฟหน้า H4 Philips H4-12V-60/55W ฿280	Shell Helix HX8 5V น้ำมันเครื่อง Shell Helix H... HX8-5W30-4L ฿1,290	Caltex Delo Gold น้ำมันเครื่องคัลเท็กซ์ Caltex D... DELO-10W30-7L ฿1,150
Brake Fluid DOT3 น้ำมันเบรก DOT3 (0.5 ลิตร) ฿120	Front Brake Pads D ผ้าเบรกดุมหน้า ดีแม็คซ์ Dometec ฿6	Front Shock Abs โช้คอัพหน้า KVB Foxal-G ฿25	Lower Arm Bushing ยางตีนถ่วง ฿15	Wiper Blade Bosch ใบปัดน้ำฝน Bosch 29 นิ้ว ฿12

ดูสินค้า

ค้นหาสินค้า / เบอร์โทร...

ช่วง (ปรับราคาช่วง)

ค้นหาช่วง / ชื่อเล่น / เบอร์...

รายการสินค้า (2) คลิกเพื่อดูสินค้า

Front Shock Absorber...	-	2	+	฿3,500
Lower Arm Bushing	-	2	+	฿700

ยอดรวม 84,200

ส่วนลด 0

เครดิตช่วง (เครดิต)

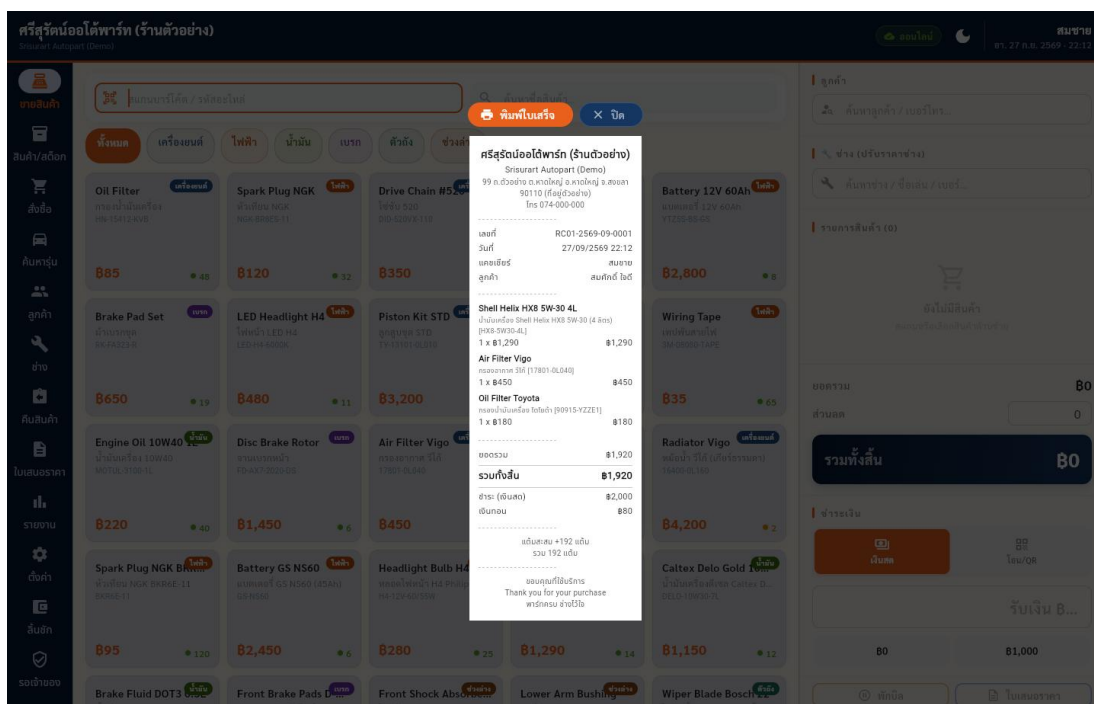
รวมทั้งสิ้น 84,200

ชำระเงิน

เงินสด โฉนด QR เครดิตช่วง

ยอดค้างคืน 80

รูปที่ 4.3 การเลือกช่วง ราคาช่วง และการชำระด้วยเครดิตช่วง



รูปที่ 4.4 ใบเสร็จเงินสด แสดงเงินทอนและแต้มสะสมของลูกค้า

4.2.2 หน้าสินค้า ช่าง และใบเสนอราคา

หน้าสินค้าและสต็อก (`products_screen.dart` ขนาด 3,843 บรรทัด) รวมแท็บสต็อก คำนวณราคา ซัพพลายเออร์ และรายงานสต็อก ดังรูปที่ 4.5 การปรับสต็อกด้วยมือแสดง "สต็อกหลังปรับ" ก่อนบันทึก และมีหน้าต่างพิมพ์ป้ายราคาที่มีบาร์โค้ด Code128 หน้าช่าง (`mechanics_screen.dart`) แสดงส่วนลดราคาช่าง ยอดค้างเครดิตเทียบวงเงิน และการรับชำระหนี้ ดังรูปที่ 4.6 ส่วนหน้าค้นหาตามรุ่นรถให้พนักงานค้นอะไหล่ด้วยชื่อรุ่น เพราะลูกค้ามักถามด้วยรุ่นรถมากกว่ารหัสสินค้า

ศรีสุวัฒน์ออโต้พาร์ท (ร้านตัวอย่าง)
Sirisurart Autopart (Demo)

ออนไลน์ ๒๗. 27 ก.ย. 2569 - 22:20

สต็อก - Stock คำนวณราคา - Price Calc ซัพพลายเออร์ - Suppliers รายงานสต็อก - Reports

29 ทั้งหมด 3 สต็อกต่ำ 0 หมดสต็อก ค้นหา ชื่อ / รหัส...

+ เพิ่มสินค้า ลบสินค้า

ทั้งหมด เครื่องยนต์ ไฟฟ้า น้ำมัน เบรก ตัวถัง ช่วงล่าง จัดการประเภท

รหัสสินค้า	ชื่อสินค้า	ประเภท	ราคาขาย	ราคาทุน	คงเหลือ	Min	สถานะ				
HN-15412-KVB	Oil Filter กรองน้ำมันเครื่อง	เครื่องยนต์	885	845	48	10	In Stock	ปรับสต็อก	แก้ไข		
NGK-BR8ES-11	Spark Plug NGK หัวเทียน NGK	ไฟฟ้า	8120	865	32	15	In Stock	ปรับสต็อก	แก้ไข		
DID-520VX-110	Drive Chain #520 โซ่ขับ 520	เครื่องยนต์	8350	8190	14	5	In Stock	ปรับสต็อก	แก้ไข		
HN-17210-KWB	Air Filter กรองอากาศ	เครื่องยนต์	8180	890	22	8	In Stock	ปรับสต็อก	แก้ไข		
YTZ5S-BS-GS	Battery 12V 60Ah แบตเตอรี่ 12V 60Ah	ไฟฟ้า	82,800	81,900	8	3	In Stock	ปรับสต็อก	แก้ไข		
RK-FA323-R	Brake Pad Set ผ้าเบรกชุด	เบรก	8650	8350	19	6	In Stock	ปรับสต็อก	แก้ไข		
LED-H4-6000K	LED Headlight H4 ไฟหน้า LED H4	ไฟฟ้า	8480	8240	11	4	In Stock	ปรับสต็อก	แก้ไข		
TY-13101-0L010	Piston Kit STD ลูกสูบชุด STD	เครื่องยนต์	83,200	81,900	5	2	In Stock	ปรับสต็อก	แก้ไข		

รูปที่ 4.5 หน้าสินค้า/สต็อก แท็บสต็อก

ศรีสุวัฒน์ออโต้พาร์ท (ร้านตัวอย่าง)
Sirisurart Autopart (Demo)

ออนไลน์ ๒๗. 27 ก.ย. 2569 - 22:20

ช่าง - Mechanics

6 คน - ยอดค่ารวม 84,200

+ เพิ่มช่าง

ทั้งหมด (6) มียอดค้าง (1)

ค้นหาชื่อ / ชื่อเล่น / เบอร์ / รหัส

รหัส	ช่าง	เบอร์	ยอดค้าง	ยอดรวม	สถานะ
M001	คุณมานพ (ลุง)	081-555-1111	80	85,000	
M002	ช่างดำ (เผ่า)	089-222-3344	80	83,000	
M003	พี่บอย (บอย)	063-789-0001	80	82,000	
M001	อัครินทร์ แก้วดี (ช่างหนุ่ม)	084-222-3344	84,200	84,200	820,000
M002	อนันต์ พรหมมา (ช่างบัณฑิต)	087-333-4455	80	815,000	
M003	สุทธิพงษ์ รักดี (ช่างสี)	090-444-5566	80	810,000	

รูปที่ 4.6 หน้าช่าง แสดงยอดค้างและวงเงินเครดิตของช่างแต่ละราย

หน้าใบเสนอราคา (quotes_screen.dart) กรองตามสถานะ (ยังใช้ได้ หมดอายุ แปลงแล้ว) และออกเอกสาร A4 ที่มีหัวร้าน รายการสินค้า เงื่อนไข และช่องลายมือชื่อ ดังรูปที่ 4.7 จุดที่

แก้ไขสำคัญคือการไม่ใช้ระยะห่างตัวอักษรกับข้อความไทย เพราะทำให้สระและวรรณยุกต์หลุดจากพยัญชนะ (#479, PR #482) ภาพนี้ถ่ายก่อนการแก้ จึงยังเห็นป้ายกำกับที่วรรณยุกต์หลุด

ศรีสุรัตน์อโต้พาร์ท (ร้านตัวอย่าง)
SRISURART AUTOPART (DEMO)
99 ถ.ตัวอย่าง ต.หาดใหญ่ อ.หาดใหญ่ จ.สงขลา 90110 (ที่อยู่ตัวอย่าง)
โทร 074-000-000

ใบเสนอราคา
QUOTATION

เสนอแก่: T0
วิภา ศรีสุข
โทร 089-876-5432

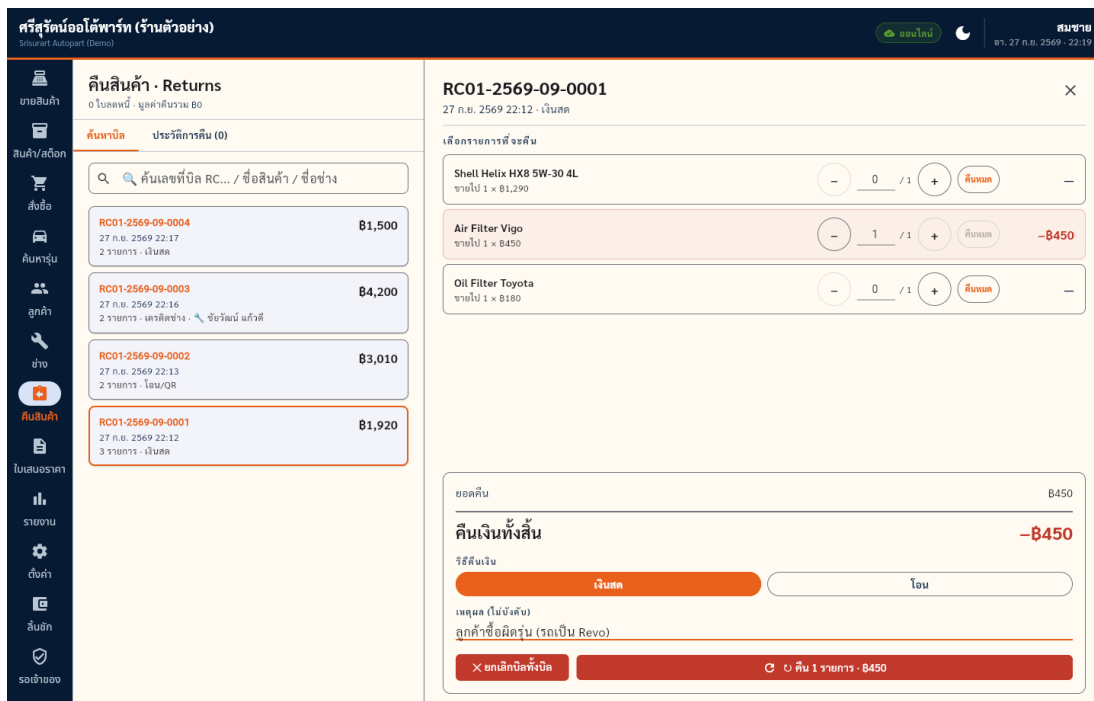
เลขที่: QT01-2569-09-0002
วันที่: 27 กันยายน 2569
ใช้ได้ถึง: 11 ตุลาคม 2569
ผู้ออก: สมชาย

#	รายการ - DESCRIPTION	จำนวน	ราคา/หน่วย	รวม
1	น้ำมันเครื่อง Shell Helix HX8 5W-30 (4 ลิตร)	1	฿1,290	฿1,290
2	กรองน้ำมันเครื่อง โตโยต้า	1	฿180	฿180
3	กรองอากาศ วีโก้	1	฿450	฿450

รูปที่ 4.7 ใบเสนอราคา A4 ส่วนหัวและรายการสินค้า (ถ่ายก่อนแก้ #479)

4.2.3 หน้าคืนสินค้า ลินซ์ และรายงานปิดร้าน

หน้าคืนสินค้า (`returns_screen.dart` ขนาด 2,057 บรรทัด) ให้ค้นบิลเดิม เลือกรายการคืน และออกใบลดหนี้ตามกติกาในตารางที่ 3.4 พร้อมแท็บประวัติการคืนที่อ้างเลข CN เลขบิล RC และเหตุผล ดังรูปที่ 4.8



รูปที่ 4.8 หน้าคั่นสินค้า

หน้าลิ้นชัก (cash_drawer_screen.dart) บังคับให้เปิดกะพร้อมระบุเงินตั้งต้นก่อน
รับเงิน จากนั้นบันทึกเงินเข้าและออก และสรุปเงินที่ควรมีในลิ้นชักในการ์ดหกช่อง ดังรูปที่ 4.9
ภายหลังมีการแก้ไขหนึ่งวันมิได้หลายกะ (#453, PR #456) และให้ยอดเงินสดที่คาดหวังใช้หน้าต่าง
การนับเดียวกันทุกที่ (#452, PR #469) รายงานปิดร้านรวบรวมรายได้ วิธีชำระ สินค้าขายดี และผล
ตรวจนับเงินในลิ้นชัก ดังรูปที่ 4.10 ภาพนี้ถ่ายก่อนการแก้ไข #461 ซึ่งเพิ่มแถวเครดิตข้างในรายงาน

ขายสินค้า

สินค้า/บริการ

สั่งซื้อ

สมัคร

ลูกค้า

ช่าง

พนักงาน

ใบเสนอราคา

รายงาน

ตั้งค่า

ล็อกอิน

รอลงชื่อ

สรุปยอดปิดร้าน

บัญชีเงินสด - Cash Drawer

27 กันยายน 2569

รายการเงิน

ปิดสิ้นชั้ก

฿2,000
เงินคงต้น

฿3,420
ยอดขายเงินสด

฿0
รับชำระเครดิต (สต)

฿450
คืนเงินสด

฿150
เงินออก

฿500
เงินเข้า (เพิ่ม)

เงินในสิ้นชั้กที่ควรมี

฿5,320

บันทึกการเคลื่อนไหวเงิน

เงินออก

เงินเข้า

จำนวนเงิน

หมายเหตุ เช่น จ่ายชีพ / เติมทอน / คำนว้น

+ เพิ่ม

฿100

฿200

฿500

฿1,000

฿2,000

ประวัติรายการวันนี้

เปิดร้าน 22:09 - สิ้นวัน ฿2,000

เติมเงินทอน
22:10

+฿500

ค่าส่งของ (แนสเซนเจอร์)
22:10

-฿150

รูปที่ 4.9 แท็บรายการเงินและการดสรุปเงินในลิ้นชัก

สรุปยอดปิดบ้าน - Daily Closing Report

27 กันยายน 2569 - ปิดเวลา 22:22 น.

฿10,630 รายได้รวม	4 บิล จำนวนบิล
฿2,658 เฉลี่ย/บิล	฿2,138 กำไรประมาณ

วิธีชำระเงิน

เงินสด (2 บิล)	83,420
โอน/QR (1 บิล)	83,010
รวมทั้งหมด	฿10,630

สินค้าขายดีวันนี้

#1 Front Shock Absorber KYB Excel-G	83,500 2 ชิ้น
#2 Battery GS NS60	82,450 1 ชิ้น
#3 Front Brake Pads Revo	81,450 1 ชิ้น
#4 Shell Helix HX8 5W-30 4L	81,290 1 ชิ้น

ข้อมูลยอดขายเชิกร์

ชื่อแคชเชียร์: สมชาย

ตารางนับเงินสดในคืนนี้

เปิดเคาน์เตอร์	฿2,000
+ ยอดขายเงินสด	+83,420
- คืนเงินสด	-8,450
+ เงินเพิ่มระหว่างวัน	+8,500
- เงินออกระหว่างวัน	-8,150
ยอดเงินสดที่ควรมี	฿5,320
นับเงินสดจริงได้	0

หมายเหตุ (ถ้ามี)

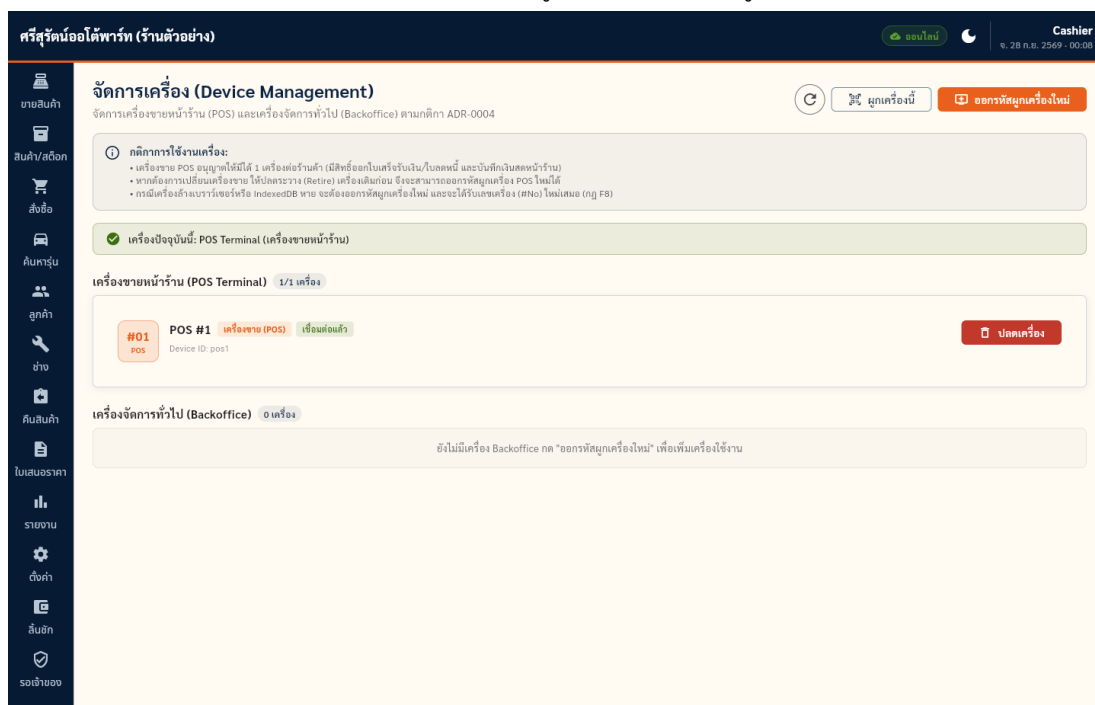
เช่น มีสินค้าเสียหาย, ปฏิกิริยาของ...

พิมพ์รายงานปิดร้าน

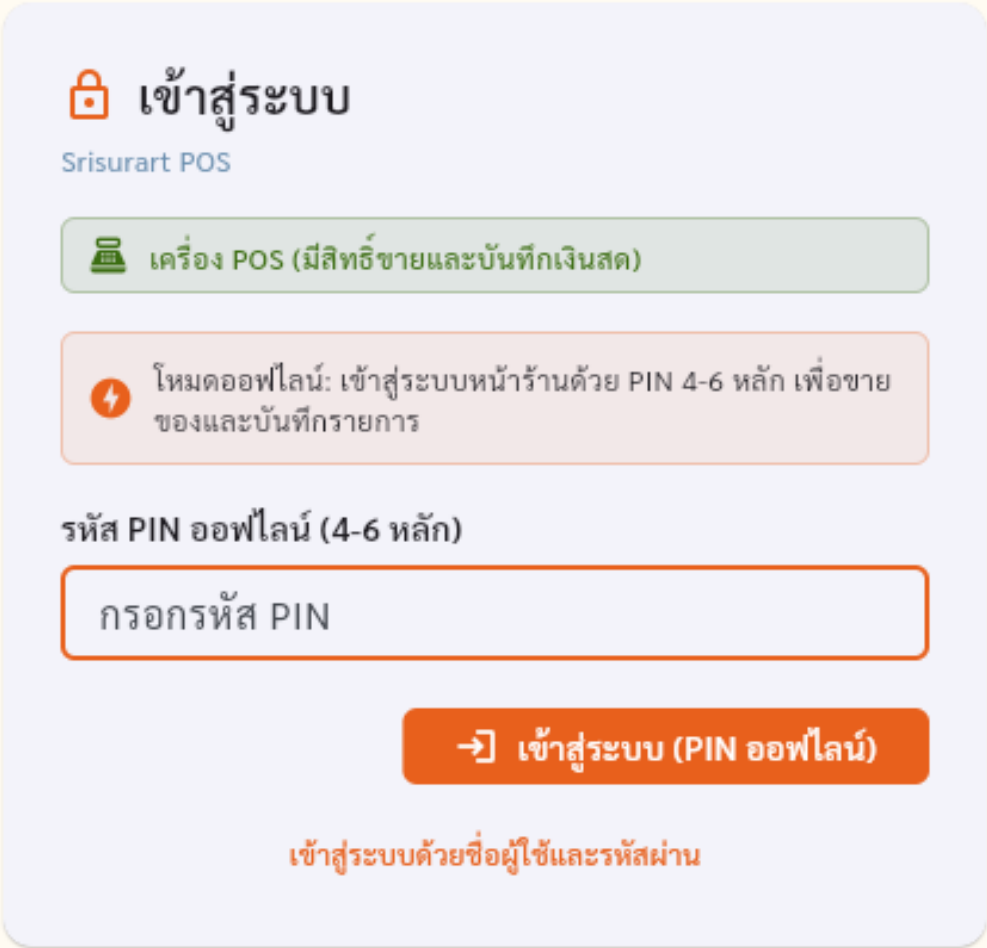
รูปที่ 4.10 รายงานปิดร้าน (Daily Closing Report, ถ่ายก่อนแก้ #461)

4.2.4 หน้าจัดการเครื่อง การเข้าสู่ระบบ และสถานะการเชื่อมต่อ

หน้าจัดการเครื่อง (`devices_screen.dart`) ใช้บนบิลด์ที่มีเซิร์ฟเวอร์ เจ้าของร้านผูกเครื่องขายและเครื่องหลังร้านแยกตามบทบาทตาม ADR-0004 ดังรูปที่ 4.11 โดยกด "ออกรหัสผูกเครื่องใหม่" ได้รหัส 8 อักขระ แล้วนำไปกรอกบนเครื่องที่จะผูก ผู้ใช้เข้าสู่ระบบด้วยชื่อผู้ใช้และรหัสผ่าน หรือด้วย PIN ออฟไลน์บนเครื่องขายที่ผูกไว้เมื่อติดต่อเซิร์ฟเวอร์ไม่ได้ ดังรูปที่ 4.12 ป้ายสถานะมุมขวามือบอกว่าออนไลน์หรือออฟไลน์ และมีแถบเหลืองเตือนเมื่อออฟไลน์ ดังรูปที่ 4.13 ส่วนร้านที่ยังใช้บิลด์ Drift อย่างเดียวไม่มีหน้าเข้าสู่ระบบและไม่ต้องผูกเครื่อง



รูปที่ 4.11 หน้าจัดการเครื่อง แยกเครื่องขาย (POS Terminal) และเครื่องจัดการทั่วไป (Backoffice)



 **เข้าสู่ระบบ**

Srisurart POS

 เครื่อง POS (มีสิทธิ์ขายและบันทึกเงินสด)

 โหมดออฟไลน์: เข้าสู่ระบบหน้าร้านด้วย PIN 4-6 หลัก เพื่อขายของและบันทึกรายการ

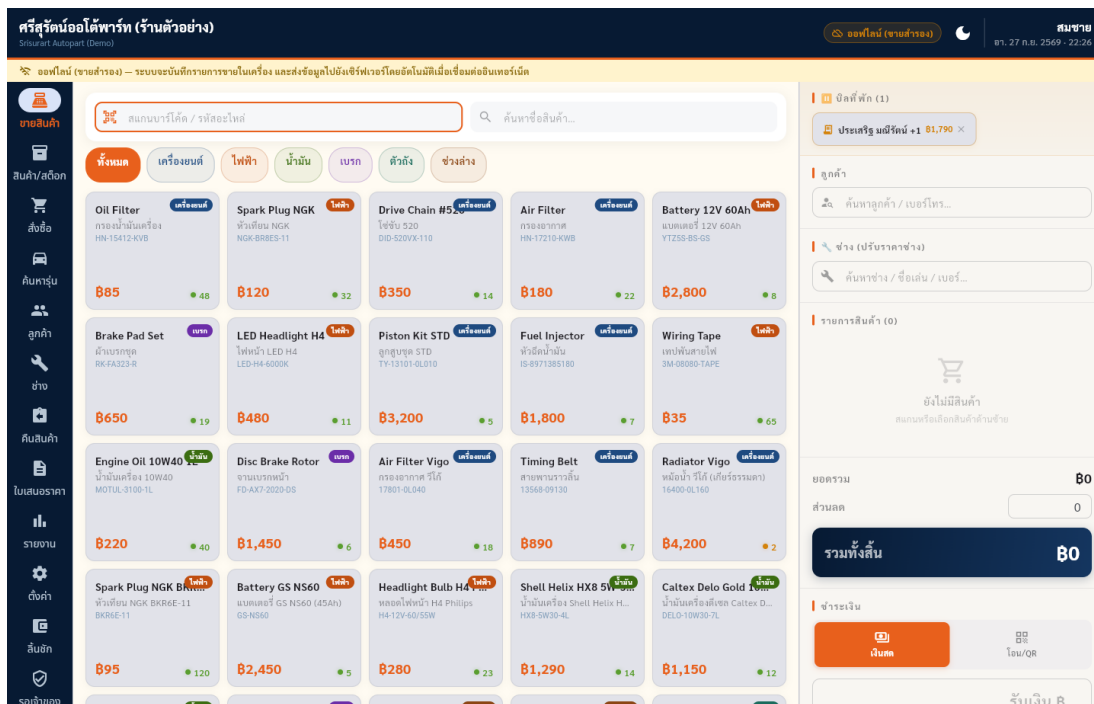
รหัส PIN ออฟไลน์ (4-6 หลัก)

กรอกรหัส PIN

→ **เข้าสู่ระบบ (PIN ออฟไลน์)**

เข้าสู่ระบบด้วยชื่อผู้ใช้และรหัสผ่าน

รูปที่ 4.12 ฟอรั่มเข้าสู่ระบบด้วย PIN ออฟไลน์บนเครื่องขาย



รูปที่ 4.13 สถานะออฟไลน์ ป้ายขายสำรองและแถบเตือนสีเหลือง

4.2.5 ความเชื่อมโยงกับปัญหาของร้าน

ตารางที่ 4.2 โยงปัญหา P1-P8 ในบทที่ 1 กับฟังก์ชันที่ตอบปัญหานั้น โดยยังไม่ได้วัดผลกับผู้ใช้จริง

ตารางที่ 4.2 ปัญหาของร้านกับฟังก์ชันและหลักฐานในรายงาน

ปัญหา	ฟังก์ชันหรือหน้าจอที่ตอบ	หลักฐาน
P1 เขียนบิลด้วยมือ	หน้าขายสินค้า ใบเสร็จ และเลขที่ใบเสร็จอัตโนมัติ	รูปที่ 4.2, 4.4
P2 ไม่แน่ใจสต็อก	ดัดสต็อกแบบเข้มงวดเมื่อขาย และหน้าสต็อก	ตารางที่ 3.4, รูปที่ 4.5
P3 ราคาและต้นทุนต่างลือต	ต้นทุนเฉลี่ยถ่วงน้ำหนักเมื่อรับสินค้า และแท็บคำนวณราคา	ตารางที่ 3.4
P4 ยอดรับ-จ่ายประจำวัน	หน้าลิ้นชักและรายงานปิดร้าน	รูปที่ 4.9, 4.10
P5 ยอดเครดิตช่วง	หน้าช่วง ยอดค้างเทียบวงเงิน และการยืนยันก่อนขายเกินวงเงิน	รูปที่ 4.6, 4.3
P6 ไม่ทราบกำไร	กำไรในรายงานปิดร้านคิดจากต้นทุน ณ เวลาขาย (ADR-0008, PR #408)	<code>closing_report.dart</code>
P7 ไม่ทราบยอดขาย	รายงานยอดขายในหน้ารายงาน	<code>reports_screen.dart</code>
P8 ไม่ทราบสินค้าขายดี	รายการสินค้าขายดีในหน้ารายงานและรายงานปิดร้าน	<code>reports_screen.dart</code> , <code>closing_report.dart</code>

4.3 ผลการพัฒนาชั้นเขียนข้อมูลผ่าน API

ชั้นเขียนข้อมูลผ่าน API สร้างตามแนวทางแคชแบบเขียนผ่านของ ADR-0010 (ยอมรับเมื่อ 4 กันยายน พ.ศ. 2569) และกติกาคำขอในหัวข้อ Repository แบบ API ของบทที่ 3 งานแบ่งเป็นสี่ชั้นซึ่งปิดตัวทั้งหมดแล้ว ดังตารางที่ 4.3 ตัวงานหลัก (parent issue) #52 ปิดเมื่อ 13 กันยายน พ.ศ. 2569

ตารางที่ 4.3 งานชิ้นเชื่อมต่อ API ฝั่งไคลเอนต์ (fe.0 ถึง fe.3)

ชั้น	เนื้อหา	ตัว	PR	ปิดตัว
fe.0	Drift schema v3: Sales.shiftId และ Shifts.id เป็น TEXT	#53	#58	10 ก.ย. 2569
fe.1	การยืนยันตัวตนฝั่ง ไคลเอนต์ การเก็บ โทเค็น การผูก เครื่อง และข้อความ ผิดพลาดจาก เซิร์ฟเวอร์	#54	#81	13 ก.ย. 2569
fe.2	ApiRepository สำหรับการอ่าน: แค็ตตาล็อก ผู้คน การจัดซื้อ ใบเสนอ ราคา รายงาน	#55	#87 และ #128	14 ก.ย. 2569
fe.3	การเขียน: ขาย สินค้า คำนวณสินค้า สิ้นซากเงินสด (ApiSales , ApiReturns , ApiShifts)	#56	#84	12 ก.ย. 2569

กติกากว่า "เฉพาะ 4xx เป็นคำตัดสิน" มีที่มาจากการทบทวน PR #84 ซึ่งพบว่าไคลเอนต์รุ่นแรกยังเขียนในเครื่องหลังได้ 5xx ทั้งที่เซิร์ฟเวอร์อาจบันทึกการเขียนนั้นไปแล้ว (commit **d5d22f8** และ **a5358d1**) ปัจจุบันกติกานี้ถูกบังคับด้วย **api_repository_contract_test.dart** ส่วนการเก็บโทเค็นบนเว็บตามภาคผนวกของ ADR-0009 สร้างเสร็จใน PR #404 (#400 ปิดตัว 25 กันยายน พ.ศ. 2569) โดยโทเค็นที่บิลด์เก่าทิ้งไว้ใน **localStorage** ถูกย้ายครั้งเดียวด้วยลำดับเขียน อ่านกลับ แล้วจึงลบ และเมื่อ 1 ตุลาคม พ.ศ. 2569 PR #534 และ #536 เพิ่มรุ่นของเซสชันใน **ApiClient** เพื่อให้การต่ออายุหรือ 401 ที่ล่าช้าจากผู้ใช้คนก่อนไม่มีผลกับผู้ใช้คนถัดไป

4.4 ผลการพัฒนาระบบทำงานออฟไลน์ เฟส 2 ฝั่งอุปกรณ์

4.4.1 การแบ่งงานและตัวที่ปิดแล้ว

งานฝั่งอุปกรณ์ของเฟส 2 ได้แก่ เว็บแอปแบบก้าวหน้า (Progressive Web App: PWA) [40] คิว **outbox_ops** ตัว **SyncService** การออกเลขเอกสาร PIN ออฟไลน์ และการดึงข้อมูล เป็นความรับผิดชอบของเลน B ตามแผน ในการลงมือจริง PR ส่วนหนึ่งเขียนโดยสมาชิกเลน C และงานแก้ไขข้อบกพร่องหลัง 27 กันยายนเขียนโดยสมาชิกเลน A ดังคอลัมน์ผู้เขียนในตารางที่ 4.4 ส่วนเลน A ส่งมอบสามตัวเพื่อปลดล็อกเลนอื่นเมื่อ 17 กันยายน พ.ศ. 2569 คือข้อความภาษาไทยของเฟส 2 (#268, PR #305) อินเทอร์เฟซ **SyncFacade** พร้อมไฟล์ตัวอย่างคำขอ **/sync/push** 18 ไฟล์ (#269, PR #307) และการจำกัดเส้นทางผู้ดูแลแพลตฟอร์ม (#270, PR #308) ตัวหลายรายการเป็น "ครึ่ง" ของคู่งานที่แบ่งฝั่งไคลเอนต์และเซิร์ฟเวอร์ ไคลเอนต์จึงทดสอบกับตัวจำลองเซิร์ฟเวอร์ ตารางที่ 4.4 สรุปตัวที่ปิดแล้ว (วันที่ปิดตรวจด้วย **gh issue view** เมื่อ 1 ตุลาคม พ.ศ. 2569)

ตารางที่ 4.4 ตัวเฟส 2 ฝั่งอุปกรณ์ที่ปิดแล้ว

ตัว	งาน	PR/commit	ผู้เขียน	ปิดตัว
#189	ห้ามออกเลข ออฟไลน์เมื่อเครื่อง ยังไม่เคยเตรียม ตัวนับ	1dff9a9	LomerAlloys	19 ก.ย. 2569
#273	PWA shell: service worker, persist, แท็บเดียว	61143b3	LomerAlloys	19 ก.ย. 2569
#228	outbox_ops และ SyncService พร้อมการบล็อกหัว คิวต่อ aggregate	PR #324	LomerAlloys	19 ก.ย. 2569
#230	หน้ารายการรอ เจ้าของร้านตรวจ 2 แท็บ และการทิ้ง รายการ	PR #318	PattaraponKitcharoen	19 ก.ย. 2569
#275	ย้ายการชำระ เครดิตเข้า outbox_ops	PR #329	PattaraponKitcharoen	19 ก.ย. 2569

ตัว	งาน	PR/commit	ผู้เขียน	ปิดตัว
#194	ส่งความยินยอม ขายเงินวงเงินในคิว ขายออฟไลน์	PR #327, #328	LomerAlloys, PattaraponKitcharoen	19 ก.ย. 2569
#229	ลูกค้าเข้าคิวได้ ปิด ปั๊มที่เป็นออนไลน์ เท่านั้น	PR #330	PattaraponKitcharoen	19 ก.ย. 2569
#211	PIN ออฟไลน์บน เครื่อง pos	PR #332	PattaraponKitcharoen	19 ก.ย. 2569
#193	contract test ของ SyncService กับตัวจำลอง เซิร์ฟเวอร์	PR #331	PattaraponKitcharoen	19 ก.ย. 2569
#276	ยกเลิกบิลออฟไลน์ ได้เฉพาะบิลที่ขาย ออฟไลน์ พร้อม เหตุผล	PR #334	PattaraponKitcharoen	20 ก.ย. 2569
#212	ดึงข้อมูลด้วย cursor ของ เซิร์ฟเวอร์ ถอย ย้อน 30 วินาที ไม่ ทับสต็อกใต้งาน ค้าง	PR #347	PattaraponKitcharoen	20 ก.ย. 2569
#452, #453	ปิดกะต้องให้คิวว่าง หลายกะต่อวัน และคิว shift. open, drawer. entry, return. create	PR #456, #469	NuimanLP	27 ก.ย. 2569
#472, #475	เลข RC ออฟไลน์ใช้ กติกาเดียวกับ CN และเตรียมตัวนับ ให้เครื่องใหม่	PR #484	NuimanLP	28 ก.ย. 2569

ตัว	งาน	PR/commit	ผู้เขียน	ปิดตัว
#489, #490	บันทึกเลขที่ เซิร์ฟเวอร์ออก ตัวนับในเครื่อง; ขึ้น เดือนใหม่เริ่ม 0001	PR #491, #492, #494	NuimanLP	28 ก.ย. 2569
#488	การทิ้งรายการย้อน ผลตรงตามค่าที่ใช้ จริง (op_ effects, schema v13)	PR #495	NuimanLP	28 ก.ย. 2569

4.4.2 ผลเพิ่มเติมที่เกิดจากการแก้ไขข้อบกพร่อง

งานหลังจากตารางข้างต้นเกิดจากการแก้ไขข้อบกพร่องที่พบระหว่างทาง ดังนี้

- **การทิ้งรายการในคิว (#473, #488)** การทิ้งรายการขายหรือคืนที่ค้างเดิมไม่ย้อนสต็อกและยอดในเครื่อง PR #483 จึงย้อนสต็อก ลูกหนี้ และสถานะยกเลิกบิลในธุรกรรมเกี่ยวกับการทิ้ง แต่การคำนวณย้อนไม่แม่นยำเมื่อการเขียนครั้งแรกถูกปิดค่าที่ศูนย์ (เช่น เครดิตช่วง 50 บาทกับใบลดหนี้หักเครดิต 180 บาท) PR #495 จึงเพิ่มตาราง **op_effects** ที่เก็บผลต่างที่ใช้จริง และปฏิเสธการทิ้งเมื่อสต็อกที่เกี่ยวข้องถูกขายต่อไปแล้วตามการตัดสินใจของเจ้าของโครงการ (รายละเอียดในบทที่ 5)
- **เลขเอกสารซ้ำ (#489)** เลขที่เซิร์ฟเวอร์ออกไม่ถูกบันทึกลงตัวนับในเครื่อง แก้แล้วตามตาราง (บทที่ 5)
- **การดึงข้อมูลกับยอดเงินในคิว (PR #544, 1 ตุลาคม พ.ศ. 2569)** การดึงข้อมูลต้องไม่ทับยอดลูกค้าและช่วงที่คำสั่งทางการเงินในคิวยังอ้างอิงอยู่ จึงคุ้มครองเฉพาะคำสั่งที่ย้ายเงินสี่ชนิด (**sale.create**, **sale.void_offline**, **return.create**, **credit_payment.create**)
- **แคชหลายร้าน (PR #539 และ #541, 1 ตุลาคม พ.ศ. 2569)** เครื่องหนึ่งเครื่องอาจถูกใช้กับหลายร้าน จึงผูกแคชในเครื่องกับรหัสร้านและปฏิเสธการสลับร้านเมื่อยังมีงานที่ไม่ได้ส่ง ส่วน PR #541 แก่กรณีบิลต์เก่าที่ยังไม่มีรหัสร้าน ข้อจำกัดที่ยอมรับแล้วคือคำตอบของการเขียนออนไลน์ที่กำลังส่งอยู่ไม่ถูกกั้นด้วยรุ่นของแคช

4.4.3 งานที่ยังไม่เสร็จของฝั่งอุปกรณ์

งานที่ยังเปิดอยู่ ได้แก่ การย้ายร้านจากบิลด์ Drift ไปใช้เซิร์ฟเวอร์ (#231) ทางต้นของการจัดการเครื่องเมื่อสูญเสียเบราร์เซอร์ที่ผูกไว้เครื่องสุดท้าย (#476) การสาธิตครบวงจรบน **mob04** (#344) และข้อความภาษาไทยสามรายการจาก PR #469 และ #470 ที่ยังเป็นร่างรอการรับรองใน **02_API_SCREEN.md** (รายละเอียดในบทที่ 5)

4.5 ผลการทดสอบฝั่งไคลเอนต์

โฟลเดอร์ **frontend/test/** มีไฟล์ทดสอบ (***_test.dart**) 90 ไฟล์ การรัน **flutter test** เมื่อ 1 ตุลาคม พ.ศ. 2569 รายงานผล "All tests passed!" ที่ 820 กรณี ซึ่งมากกว่าจำนวนจุดนิยาม **test()** และ **testWidgets()** ในซอร์ส (741 จุด) เพราะบางกรณีสร้างในรูปแบบพารามิเตอร์ ส่วน **dart analyze** รายงาน "No issues found!" (ใช้ **dart analyze** แทน **flutter analyze** ด้วยเหตุผลในบทที่ 5)

ชุดทดสอบแบ่งเป็นสี่ระดับ ระดับแรกคือการทดสอบหน่วยของกฎธุรกิจแต่ละข้อในตารางที่ 3.4 บนฐานข้อมูลในหน่วยความจำ ระดับที่สองคือการทดสอบสัญญา (contract test) ที่บังคับกฎสถาปัตยกรรม เช่น **api_repository_contract_test.dart** และการทดสอบ **SyncService** กับไฟล์ตัวอย่างของ **/sync/push** ระดับที่สามคือการทดสอบวิดเจ็ต เช่น ความกว้างหน้าจอโทรศัพท์ 390 พิกเซล แถบสถานะการซิงค์ และการเปลี่ยนเส้นทางเมื่อเข้าและออกจากระบบ ระดับสุดท้ายคือการทดสอบเส้นทางพื้นฐาน (route smoke test) ของทุกหน้า

ข้อจำกัดของผลนี้มีสองประการ ประการแรก ผลการรันมีข้อความเตือนเรื่องที่เก็บโทเค็นใช้ไม่ได้ ซึ่งมาจากการทดสอบที่จำลองความล้มเหลวของ IndexedDB โดยตั้งใจ ไม่ใช่ข้อผิดพลาดประการที่สอง การทดสอบฝั่งอุปกรณ์ใช้ตัวจำลองเซิร์ฟเวอร์ ผลผ่านจึงยังไม่พิสูจน์ว่าไคลเอนต์ทำงานถูกต้องกับเซิร์ฟเวอร์จริงทั้งหมด การพิสูจน์ขั้นสุดท้ายคือการสาธิตครบวงจร (#344) ซึ่งยังไม่ได้ดำเนินการ

4.6 ผลการพัฒนาฝั่งเซิร์ฟเวอร์

4.6.1 ภาพรวมของฝั่งเซิร์ฟเวอร์ เฟส 1

ผลลัพธ์ของเฟส 1 คือเซิร์ฟเวอร์ NestJS [25] บน PostgreSQL ที่รับการเขียนข้อมูลธุรกรรมทั้งหมดของร้าน ได้แก่ การขาย การคืนสินค้า การรับสินค้าเข้าตามใบสั่งซื้อ และการขาย โดยย้ายกฎความถูกต้องของข้อมูลจากฝั่ง Dart มาบังคับที่ฝั่งเซิร์ฟเวอร์ตามโครงสร้างโมดูลในตารางที่ 3.6

ขนาดของโค้ดและชุดทดสอบสรุปไว้ในตารางที่ 4.9 ท้ายบท ส่วนตารางที่ 4.5 สรุปองค์ประกอบสำคัญคู่กับไฟล์ทดสอบที่เป็นหลักฐาน

ตารางที่ 4.5 สรุปองค์ประกอบสำคัญของฝั่งเซิร์ฟเวอร์และหลักฐานการพิสูจน์

องค์ประกอบ	หน้าที่	หลักฐาน
RLS และคีย์ผสม (<code>tenant_id</code> , <code>id</code>)	แยกข้อมูลรายร้านในระดับฐานข้อมูล	<code>cross-tenant-read.e2e-spec.ts</code> (25 ตาราง + 12 จุดปลายทาง HTTP)
<code>runTx / runIdempotent</code>	ธุรกรรมในตัวจัดการคำขอ และการกันการเขียนซ้ำ	สเปกสถาปัตยกรรม 3 ไฟล์, <code>tx-hold-measure.e2e-spec.ts</code>
เพดานเวลาคอมมิต 25 วินาที	กันธุรกรรมค้างยาว	<code>tx-ceiling.e2e-spec.ts</code>
บทบาทอุปกรณ์ <code>pos / backoffice</code>	จำกัดสิทธิ์การเขียนบิลตามเครื่อง	<code>security.e2e-spec.ts</code>
จำกัดอัตราต่อร้านบน Redis	กันร้านหนึ่งกินทรัพยากรร้านอื่น	<code>rate-limit.e2e-spec.ts</code> , <code>rate-limit-pool.e2e-spec.ts</code>
ระนาบแพลตฟอร์ม	สร้างร้านและจัดการเครื่องโดยไม่แตะ <code>psql</code>	<code>platform.e2e-spec.ts</code> , <code>platform-cli.e2e-spec.ts</code>
<code>POST /sync/push</code>	เล่นใช้งานออฟไลน์ของอุปกรณ์	<code>sync-push.e2e-spec.ts</code> , <code>sync-push-fixtures.e2e-spec.ts</code>
คิวงาน BullMQ และ Bull-Board	ส่งออกข้อมูลร้านและงานตามเวลา	<code>queue.e2e-spec.ts</code> , <code>worker-jobs.e2e-spec.ts</code>

4.6.2 การแยกข้อมูลรายร้าน

การพิสูจน์ว่า RLS [10] ทำงานจริงอยู่ในไฟล์ `server/test/cross-tenant-read.e2e-spec.ts` ซึ่งตรวจครบทั้ง 25 ตารางที่มีข้อมูลรายร้าน และทดสอบผ่าน HTTP อีก 12 จุดปลายทาง โดยทุกกรณีต้องได้ 0 แถวหรือสถานะ 404

ระหว่างการตรวจโค้ดทั้งระบบพบข้อบกพร่องสองข้อใน migration ของตาราง `owner_review_items` ข้อแรกคือนโยบาย RLS ไม่ห่อค่าตัวแปรเชสชันด้วย `NULLIF(..., '')` ทำให้เมื่อค่าว่างเกิดข้อผิดพลาดและตอบ HTTP 500 แทนศูนย์แถว ข้อที่สองคือคีย์นอกที่ตั้ง `ON DELETE SET NULL` ทำให้ `tenant_id` ซึ่งเป็น NOT NULL ถูกตั้งเป็นค่าว่างไปด้วย ทั้งสองข้อแก้ไขเมื่อวันที่ 25 กันยายน พ.ศ. 2569 ด้วย migration ใหม่ `1788652804200 -`

`OwnerReviewItemsFixes.ts` และพิสูจน์ใน `server/test/schema.e2e-spec.ts` โดยไม่แก้ migration ที่ถูกใช้งานไปแล้ว ปัจจุบันมี migration ทั้งหมด 20 ไฟล์

4.6.3 จุดเชื่อมต่อธุรกรรม (transaction seam) และ idempotency

การย้ายธุรกรรมจาก middleware มาอยู่ใน handler ตามกติกาในบทที่ 3 แบ่งเป็นหกชิ้นงาน ซึ่งรวมเสร็จครบเมื่อวันที่ 15 กันยายน พ.ศ. 2569 การวัดด้วย `tx-hold-measure` หลังย้ายการตรวจ PIN ออกจากธุรกรรม แสดงว่าธุรกรรมที่ยาวที่สุดลดจากราว 101–131 มิลลิวินาที เหลือราว 14–22 มิลลิวินาที และเวลาตอบเลิกเพิ่มเป็นขั้นบันไดตามจำนวนคำขอพร้อมกัน สเปกสถาปัตยกรรมสามไฟล์ (`tenant-door.spec.ts`, `tenant-wrapper.spec.ts` และ `idempotency-routes.spec.ts`) ตรวจสอบโครงสร้างของโค้ดว่าไม่มีเส้นทางเขียนใดหลบเลี่ยงจุดเชื่อมนี้ แต่มีจุดบอดหนึ่งข้อ คือเส้นทางเขียนที่ไม่ได้อ้างสิทธิ์ idempotency เลยจะไม่ปรากฏในสเปกทั้งสามไฟล์ จึงยังต้องอาศัยการทบทวนโดยคน

ในเชิงพฤติกรรม ระบบรับประกันสิ่งต่อไปนีซึ่งมีการทดสอบรองรับตามตารางที่ 4.8 การขายที่สต็อกไม่พอหลายบรรทัดตอบข้อความภาษาไทยครบทุกบรรทัดในคำตอบเดียว การขายพร้อมกัน 200 คำขอบนสินค้าที่มี 50 ขึ้นได้บิลสำเร็จ 50 ใบพอดี (วัดบน VM สาธิตเมื่อ 15 กันยายน พ.ศ. 2569 ได้ 201 จำนวน 50 ครั้ง 409 จำนวน 150 ครั้ง และ 5xx ศูนย์ครั้ง) และสภาวะแข่งกัน (race condition) ของสามคำขอบนสินค้าตัวเดียว 200 รอบไม่มีการสูญหายของการอัปเดต นอกจากนี้การนำเข้าสแนปช็อตข้อมูลจริงของร้านผ่านเกณฑ์ตรวจ 44 ข้อ (#185 เมื่อ 17 กันยายน พ.ศ. 2569) โดยไฟล์ที่นำเข้าให้ยอดขาย 33,700 บาทและสต็อกรวม 191 ชิ้น ระบบนำเข้าตรวจข้อมูลก่อน (pre-flight) แล้วจึงปรับค่า เพราะการบีบค่า (clamp) บนข้อมูลที่ยังไม่ตรวจจะเปลี่ยนความเสียหายที่เห็นชัดให้กลายเป็นความเสียหายเงียบ

4.6.4 ระบายผู้ดูแลแพลตฟอร์ม

การทดสอบ `platform.e2e-spec.ts` ครอบคลุมวงจรเต็มผ่าน HTTP คือสร้างร้านเข้าสู่ระบบเป็นเจ้าของ สร้างสินค้า ลงทะเบียนเครื่อง `pos` เข้าสู่ระบบบนเครื่องนั้น เปิดกะ และขายจริง ตัวงานหลัก #443 มีโค้ดที่รวมแล้วเมื่อ 27 กันยายน พ.ศ. 2569 ได้แก่ เครื่องมือบรรทัดคำสั่งของแพลตฟอร์ม การออกรหัสลงทะเบียนเครื่องใหม่ รหัสผ่านชั่วคราวของเจ้าของร้านพร้อมบังคับเปลี่ยน และแดชบอร์ดเว็บ `platform-ui` การจำกัดเส้นทาง `/api/v1/platform/` สองชั้น (Nginx และแอปพลิเคชัน) พิสูจน์แล้วว่าได้ 403 ทั้งสองชั้นบนเครื่อง `mob04` เมื่อ 30 กันยายน พ.ศ. 2569 ตัว #443 ยัง **ไม่ปิด** เพราะเกณฑ์ข้อ "เฉพาะ `bootstrap:admin` เท่านั้นที่สร้างผู้ดูแลได้"

ขัดกับการชิงผู้ดูแลจากตัวแปร `PLATFORM_ADMINIS` ที่เจ้าของโครงการรับรองแล้ว และยังมีคำถามที่รอเจ้าของโครงการตัดสินใจ

4.6.5 เซิร์ฟเวอร์ของเฟส 2

จุดปลายทาง `POST /sync/push` สร้างตามสเปกเฟส 2 และทดสอบฝั่งเซิร์ฟเวอร์กับไฟล์ตัวอย่างคำขอ 18 ไฟล์ชุดเดียวกับที่ฝั่งไคลเอนต์ใช้ (`sync-push-fixtures.e2e-spec.ts` และ `sync-push.e2e-spec.ts`) ทำให้สองเลนพัฒนาแยกกันได้โดยไม่ต้องรอกัน การตรวจโค้ดทั้งระบบเมื่อ 24 กันยายน พ.ศ. 2569 พบข้อบกพร่องระดับสูงสองข้อ ข้อแรกคือค่าสรุปของคำขอ (fingerprint) ของ `/sync/push` ไม่ตรงกับของเส้นทางออนไลน์ (`POST /sales` เทียบกับ `POST /api/v1/sales`) แก้ด้วย PR #413 ข้อที่สองคือเส้นทางออนไลน์เก็บวันที่จากเนื้อหาคำขอของไคลเอนต์แทนเวลาเซิร์ฟเวอร์ แก้ด้วย PR #414 ทั้งสองข้อแก้แล้วเมื่อ 25 กันยายน พ.ศ. 2569 รายงานเดียวกันพบช่องว่างของสเปกกระดักกลางสามข้อซึ่งแก้แล้วเมื่อ 27 กันยายน (PR #456, #458, #469) ข้อที่ยังเหลือสรุปไว้ในบทที่ 5

4.7 ผลการพัฒนา CI/CD

4.7.1 ระดับที่ 1 ถึง 3

สามระดับแรกของ CI/CD ตามบทที่ 3 เสร็จสมบูรณ์ ประกอบด้วยเวิร์กโฟลว์ GitHub Actions [31] สามไฟล์ ได้แก่ `flutter.yml`, `server.yml` และ `deploy.yml` โดย Flutter CI ตรวจโค้ด ทดสอบ ตรวจไฟล์ที่สร้างอัตโนมัติ และตรวจว่าไฟล์ฐานข้อมูลบนเว็บตรงกับเวอร์ชันแพ็คเกจ ส่วน Server CI รันการตรวจโค้ด การทดสอบหน่วย การทดสอบรวมระบบกับ PostgreSQL และ Redis จริง และสร้างอิมเมจที่ผ่านการสแกนด้วย Trivy [33] ก่อนเผยแพร่ไปยัง GHCR อิมเมจใน Docker Compose ถูกตรึงด้วย digest (#401, PR #403) และมีการทดสอบ `compose-image-pins.spec.ts` ตรวจเรื่องนี้ การป้องกันสาขา `main` ตั้งแต่ 15 กันยายน พ.ศ. 2569 บังคับให้ผ่าน PR และผ่านงานสถานะสองรายการ

เมื่อวันที่ 1 ตุลาคม พ.ศ. 2569 มีการเพิ่มด้านความปลอดภัยสามรายการ ได้แก่ งานสแกนความลับด้วย gitleaks [34] ใน `server.yml` ที่สแกนคอมมิตที่ PR หรือ push เพิ่มเข้ามาทุกครั้งโดยไม่กรองพาร และเป็นเงื่อนไขของ `server-ci-status` การเปิด secret scanning และ push protection ของ GitHub และการเปิด Dependabot alerts พร้อม security updates นอกจากนี้ก็กลไกข้ามการทดสอบเมื่อ push เป็นเอกสารล้วนทำงานได้หลังแก้ข้อบกพร่องใน PR #548 ซึ่งอธิบายไว้ในบทที่ 5

4.7.2 ระดับที่ 4: การ deploy ด้วย Ansible และ self-hosted runner

self-hosted runner บน **mob04** ติดตั้งเมื่อ 30 กันยายน พ.ศ. 2569 และการ deploy ด้วย Ansible [35] ทำงานจริงในวันเดียวกัน ตัว #67 (การติดตั้ง runner) ปิดในวันนั้นด้วยหลักฐานครบ 15 จาก 15 ข้อ หลักฐานหลักแสดงในตารางที่ 4.6 ข้อที่ต้องระบุคือ เกณฑ์ย่อยด้านการจัดการ PR จาก fork พิสูจน์จากโค้ดและค่าติดตั้งเท่านั้น ไม่มีการรันจริงจาก fork และเจ้าของโครงการยอมรับหลักฐานระดับนี้

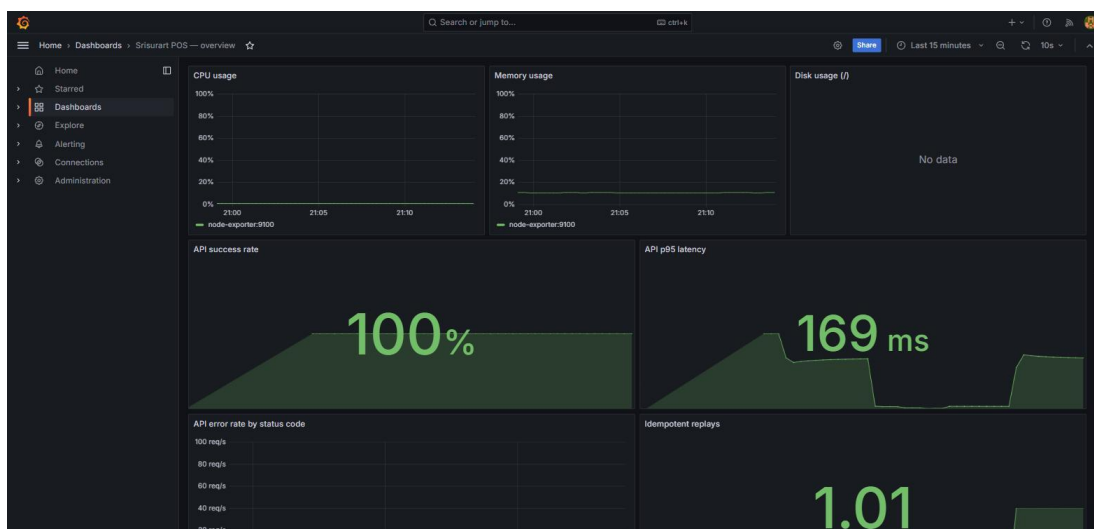
ตารางที่ 4.6 หลักฐานการ deploy บนเครื่อง mob04 (30 กันยายน พ.ศ. 2569)

กรณี	คอมมิต	ผล
deploy ครั้งแรก (อนุมัติโดย NuimanLP)	e50f4fa	Ansible failed=0 , current_sha = e50f4fa , / health/ready ตอบ 200
merge เข้า main แล้ว deploy อัตโนมัติ	494ace3	deploy สำเร็จ
ย้อนรุ่นผ่าน workflow_dispatch	e50f4fa	สำเร็จ สคิมาไม่เปลี่ยน
รัน SHA เดิมซ้ำ	SHA ที่ติดตั้งอยู่แล้ว	ข้ามด้วยข้อความ "Skipping duplicate deployment"
readiness ล้มเหลว	–	รันเป็นสีแดง และติดตั้งรุ่นเดิมกลับอัตโนมัติ
สั่ง deploy จากสาขาที่ไม่ใช่ main	–	ถูกปฏิเสธ
deploy การแก้ CORS (PR #516)	00d3488	Ansible failed=0

ประตุนุมัติบนสภาพแวดล้อม **demo** ทำงานตามที่ออกแบบ คือทุกรันของ **Deploy (demo)** เข้าสู่สถานะ "Waiting for review" จนกว่าผู้ตรวจที่กำหนดจะอนุมัติ ข้อควรระวังคือการรัน **Deploy (demo)** ที่เป็นสีเขียวไม่ใช่หลักฐานว่ามีการ deploy เพราะงาน **deploy** ถูกข้ามเมื่ออิมเมจของ SHA นั้นยังไม่อยู่บน GHCR โดยเวิร์กโฟลว์ยังรายงานสำเร็จ เช่น เมื่อ 30 กันยายน การตรวจ **pnpm audit** ระดับสูงทำให้ **server-ci-status** บน **main** เป็นสีแดง จึงไม่มีอิมเมจ และทุกรัน deploy ถูกข้ามจนกว่า PR #512 จะรวม หลักฐานที่ใช้ได้มีเพียงไฟล์ **/opt/pos/.current_sha** บนเครื่อง

4.7.3 การติดตามผลด้วย Grafana

การซ้อมสำเร็จวันที่ 21 กันยายน พ.ศ. 2569 บันทึกแฟงควบคุม Grafana [37] ขณะมีการเข้าสู่ระบบและขายจริงบนสแตกที่ทำงานอยู่ ดังรูปที่ 4.14 แฟงแสดงอัตราความสำเร็จ ความหน่วง p95 อัตราข้อผิดพลาดแยกตามรหัสสถานะ และตัวนับการเล่นซ้ำของ idempotency ที่ Prometheus [36] เก็บจากตัวชี้วัดของเซิร์ฟเวอร์ตามการออกแบบในบทที่ 3



รูปที่ 4.14 แฟงควบคุม Grafana ระหว่างการซ้อมสำเร็จ (บันทึกเมื่อ 21 กันยายน พ.ศ. 2569)

4.8 ผลการนำระบบขึ้นใช้งานบนเครื่อง mob04 และการติดตามระบบ

4.8.1 สแตกบนเครื่องสำเร็จ

เครื่อง **mob04** เป็น VM ของภาควิชาที่เป็นทั้งเครื่องสำเร็จและเป้าหมายการ deploy ตามการตัดสินใจเมื่อ 15 กันยายน พ.ศ. 2569 ที่ให้ระบบเฟส 1 อยู่ภายในเครือข่ายของมหาวิทยาลัย สแตกประกอบด้วยคอนเทนเนอร์ตามตารางที่ 3.3 เครื่องดึงอิมเมจจาก ghcr.io ได้ตั้งแต่ 29–30 กันยายน พ.ศ. 2569 ก่อนหน้านั้นการ deploy อัตโนมัติ (CD) ซึ่งต้องดึงอิมเมจจาก GHCR ถูกไฟร์วอลล์ของคณะปิดกั้น (บทที่ 5)

ตัว #343 (การ deploy จริงครั้งแรก) ปิดเมื่อ 30 กันยายน พ.ศ. 2569 ครบ 5 จาก 5 ข้อ รวมถึงการย้อนกลับด้วย Ansible ด้วยมือไปยัง **e50f4fa** ที่ได้ **failed=0** ส่วนการสาธิตแบบครบวงจร (#344) ยัง **ไม่ได้ดำเนินการ** ข้อติดขัดสรุปไว้ในบทที่ 5

4.8.2 การสำรองข้อมูลและ etcd

สคริปต์ `deploy/scripts/backup-db.sh` dump ฐานข้อมูลลงใน `/opt/pos/backups/` ทุกวันเวลา 03:00 ตัว #346 ปิดเมื่อ 30 กันยายน พ.ศ. 2569 หลังรันสคริปต์ในสภาพแวดล้อมแบบเดียวกับ cron สำเร็จและตรวจไฟล์ dump แล้ว ในคืนวันที่ 29 กันยายน สคริปต์ทำงานล้มเหลวและทิ้งไฟล์ว่างไว้ PR #519 จึงให้สคริปต์เขียนไฟล์ชั่วคราวแล้วเปลี่ยนชื่อเมื่อสำเร็จเท่านั้น พร้อมการทดสอบใน CI และสคริปต์ใหม่ที่ติดตั้งบน `mob04` แล้วได้ไฟล์ dump ที่ผ่านการตรวจสอบความสมบูรณ์ ข้อควรระวังคือมีเพียง playbook `provision.yml` ที่ติดตั้งสคริปต์นี้ การ deploy ผ่าน CD ไม่ได้อัปเดตสคริปต์บนเครื่อง

ต้องระบุอย่างตรงไปตรงมาว่า การสำรองข้อมูลยังไม่ออกนอก VM กลไกส่งสำเนาออกนอกเครื่องผ่าน `rcclone` ถูกสร้างไว้แต่ยังไม่ได้เชื่อมกับปลายทางจริง (#363 และตัวงานหลัก #288 ยังเปิดและถูกพักไว้จนหลังการสาธิต) หากติสก์ของ `mob04` เสียจะสูญเสียข้อมูลของร้านสาธิต เจ้าของโครงการรับความเสี่ยงนี้โดยรู้ล่วงหน้า และบันทึกของ cron ที่ไม่มีข้อผิดพลาดไม่ได้พิสูจน์ว่าสำเนาออกนอก VM เพราะเมื่อยังไม่ได้ตั้งปลายทาง สคริปต์จะพิมพ์เพียงคำเตือนแล้วจบด้วยรหัสสำเร็จ

ตัว #365 (etcd ไม่มีการยืนยันตัวตนบน VM) ปิดเมื่อ 30 กันยายน พ.ศ. 2569 ครบ 4 จาก 4 ข้อ โดยเปิดการยืนยันตัวตนตั้งแต่การ deploy ครั้งแรก พิสูจน์การยืนยันตัวตนทั้งสองด้านและยืนยันว่า `RuntimeConfigService` อ่านค่า `log_level` จาก etcd ได้

4.9 ผลการทดสอบฝั่งเซิร์ฟเวอร์

ชุดทดสอบของเซิร์ฟเวอร์แบ่งเป็นการทดสอบหน่วย (`*.spec.ts`) ที่อยู่ข้างไฟล์ต้นทางและการทดสอบรวมระบบ (`*.e2e-spec.ts`) ที่รันกับ PostgreSQL และ Redis จริงภายใต้ `server/test/` จำนวนที่นับเมื่อ 1 ตุลาคม พ.ศ. 2569 แสดงในตารางที่ 4.7 ตัวเลขฝั่งเซิร์ฟเวอร์นับจากบรรทัด `it(` หรือ `test(` ในซอร์ส จึงเป็นค่าประมาณที่ไม่นับกรณีที่สร้างในลูป ส่วนฝั่งไคลเอนต์แสดงทั้งจำนวนจากตัวรันและจากซอร์สเพื่อเทียบกัน

ตารางที่ 4.7 จำนวนไฟล์และกรณีทดสอบของเซิร์ฟเวอร์และไคลเอนต์ (นับเมื่อ 1 ตุลาคม พ.ศ. 2569)

กลุ่มทดสอบ	ไฟล์	กรณีทดสอบ
เซิร์ฟเวอร์ — unit (<code>*.spec.ts</code>)	54	ประมาณ 492 (นับจากซอร์ส)
เซิร์ฟเวอร์ — e2e (<code>*.e2e-spec.ts</code>)	57	ประมาณ 655 (นับจากซอร์ส)
ไคลเอนต์ Flutter (<code>*_test.dart</code>)	90	820 ตามตัวรัน (741 จุดนิยามในซอร์ส)

ชุด e2e ครอบคลุมความปลอดภัยและบทบาทอุปกรณ์ การแยกข้อมูลรายร้าน ธุรกิจและ idempotency สภาวะแข่งกันของผู้เขียนหลายราย การจำกัดอัตรา แคลชและการลุ่มของ Redis การขาย คินสินค้า ใบสั่งซื้อ กะ และรายงาน การนำเข้าและการสำรองข้อมูล ระบายแพลตฟอร์ม คิวและ worker การชิงก์ และเกณฑ์ของสคีม่า การรันชุด e2e เต็มเมื่อ 16 กันยายน พ.ศ. 2569 บันทึกว่าผ่าน 490 กรณีและข้าม 2 กรณีจากทั้งหมด 492 กรณี (ไม่มีกรณีล้มเหลว) ซึ่งเป็นจำนวนของชุด e2e ในวันนั้น รายงานนี้ไม่อ้างผลการรันล่าสุดที่ไม่ได้ทำซ้ำ ข้ออ้างว่าชุดทดสอบผ่านในแต่ละช่วงยึดตามสถานะ `server-ci-status` บน `main` ซึ่งเป็นผลจริงของการรันใน CI

4.10 ตรวจสอบเกณฑ์ตรวจรับ (DoD)

เอกสาร `docs/Backend_design/03_ARCHITECTURE.md` หัวข้อ 8 กำหนดเกณฑ์ตรวจรับ (DoD) ของเฟส 1 จำนวน 17 ข้อ การนับซ้ำกับตัวทดสอบเอง (ไม่ใช่สถานะของตัว) เมื่อ 22 กันยายนและอีกครั้งในการเขียนรายงานนี้พบว่า ผ่านแล้ว 16 ข้อ และยังเปิดอยู่ 1 ข้อ คือ เกณฑ์ k6 ซึ่งผูกกับตัว #380 ดังตารางที่ 4.8

ตารางที่ 4.8 สถานะเกณฑ์ตรวจรับ (DoD) ของเฟส 1 (17 ข้อ)

ลำดับ	เกณฑ์	สถานะ	หลักฐานหลัก
1	<code>docker compose up</code> ครั้งเดียวได้ Nginx + NestJS x3 + Postgres + Redis + worker + Bull-Board	ผ่าน	#14 (6 ก.ย. 2569)
2	k6 ผ่านเกณฑ์ตาม <code>02_API_SCREENSHOTS.md</code> หัวข้อ 9	ยังเปิด	ตัว #380 ยังไม่มีผลวัดจริง
3	<code>POST /sales</code> พร้อมกัน 200 ครั้งบนสินค้า 50 ชิ้น ขายได้ 50 บิล สต็อก 0	ผ่าน	#184 ส่วน <code>close.3</code> (15 ก.ย. 2569, VM สาธิต) 201x50 / 409x150 / 5xx 0
4	อ่านข้ามร้านได้ 0 แถวทุกเคส	ผ่าน	<code>cross-tenant-read.e2e-spec.ts</code> (17 ก.ย. 2569)

ลำดับ	เกณฑ์	สถานะ	หลักฐานหลัก
5	/health/live ไม่ แตะ DB และ / health/ready แตะ DB และ Redis	ผ่าน	#14 (6 ก.ย. 2569)
6	นำเข้าสแนปช็อตร้านจริง ผ่านเกณฑ์ 6 ข้อใน 01_ DATABASE.md หัวข้อ 9	ผ่าน	#185 (17 ก.ย. 2569) 44 checks
7	redis-cache แยก จาก redis-queue และ Bull-Board มี auth	ผ่าน	#14 (6 ก.ย. 2569)
8	ขายสินค้าไม่พอ 3 บรรทัดได้ข้อความไทย ครบ 3 บรรทัดในครั้ง เดียว	ผ่าน	sales.e2e-spec. ts, #20
9	ลบลูกค้าที่มีบิลได้ 200 (soft delete) ไม่ใช่ 500	ผ่าน	people.e2e-spec. ts, #17
10	สร้างร้านใหม่ด้วย POST /platform/ tenants แล้วเข้าสู่ ระบบและขายได้โดยไม่ แตะ psql	ผ่าน	platform.e2e- spec.ts (17 ก.ย. 2569)
11	เครื่อง backoffice เรียก POST /sales แล้วได้ 403 DEVICE_ ROLE_FORBIDDEN	ผ่าน	security.e2e- spec.ts, #44
12	ระงับร้านแล้วคำขอถัดไป ถูกปฏิเสธทันที	ผ่าน	request-context. e2e-spec.ts, #20
13	ระงับร้านแล้วงานค้างใน คิวของร้านนั้นไม่ถูกรัน	ผ่าน	queue.e2e-spec. ts, #34
14	ดับ redis-cache แล้วร้านที่ถูกระงับยังถูก ปฏิเสธและร้านปกติยังใช้ งานได้	ผ่าน	redis-cache- outage.e2e-spec. ts, #383 (22 ก.ย. 2569)

ลำดับ	เกณฑ์	สถานะ	หลักฐานหลัก
15	สภาวะแข่งกันของสามคำ ขอบนสินค้าตัวเดียว 200 รอบ ไม่มี lost update	ผ่าน	<code>purchase-orders.e2e-spec.ts</code> , #196
16	<code>owner</code> ปลดเครื่อง <code>pos</code> ที่มีกะเปิด: กะปิดใน ธุรกรรมเดียว <code>token</code> เดิม <code>refresh</code> ไม่ได้ ลงทะเบียนเครื่องใหม่ แล้วขายได้	ผ่าน	<code>shifts.e2e-spec.ts</code> , #384 (22 ก.ย. 2569)
17	<code>backoffice</code> ที่ไม่มี <code>deviceToken</code> อ่าน สินค้าได้แต่ขายได้ 403	ผ่าน	<code>security.e2e-spec.ts</code> , #196

เกณฑ์ข้อ 14 และข้อ 16 ได้รับการยืนยันผ่านใหม่เมื่อมีหลักฐานที่แท้จริง เพราะหลักฐานเดิมของข้อ 14 เพียงจำลอง (spy) การเรียกเมธอดของไคลเอนต์แคชแทนที่จะทำให้ Redis เข้าไม่ถึงจริง และหลักฐานเดิมของข้อ 16 สร้างโทเค็นเองแทนที่จะแลกรหัสลงทะเบียนผ่านจุดปลายทางจริง

เกณฑ์ข้อ 2 ยังไม่มีการวัด ตัว #184 เดิมที่ผูกกับการวัดนี้ถูกปิดและเปิดใหม่รวมสามครั้งในสองวัน จนเจ้าของโครงการปิดเมื่อ 21 กันยายน พ.ศ. 2569 โดยเกณฑ์ย่อยด้านการวัดยังไม่ผ่าน และตัว #380 เข้ามาแทน

วิธีวัดที่ตกลงแล้วคือใช้สามเครื่องรัน k6 โดยแต่ละเครื่องอยู่ภายใต้ขีดจำกัดอัตราต่อ IP ของตัวเอง และส่งผลเข้า Prometheus ของ VM ส่วนการยกเว้น IP ของเครื่องสร้างโหลดจากขีดจำกัดอัตราถูกพิจารณาแล้วปฏิเสธ

4.11 สรุปผลงานที่มีความก้าวหน้า

ตารางที่ 4.9 สรุปตัวเลขภาพรวมของโครงการ วัดเมื่อ 1 ตุลาคม พ.ศ. 2569 จากสาขา `origin/main` ด้วย `git rev-list --count`, `git ls-files` ร่วมกับ `wc -l`, `gh pr list` และ `gh issue list` จำนวนบรรทัดนับรวมไฟล์ที่สร้างอัตโนมัติที่ถูกคอมมิตไว้ จึงเป็นเพียงมาตรวัดขนาด ไม่ใช่มาตรวัดผลิตภาพ

ตารางที่ 4.9 สถิติภาพรวมของโครงการ (วัดเมื่อ 1 ตุลาคม พ.ศ. 2569)

รายการ	ค่าที่วัดได้
จำนวนคอมมิตบน <code>origin/main</code>	1,058
คอมมิตแรกของที่เก็บโค้ด	23 มิถุนายน พ.ศ. 2569
Pull request ที่รวมแล้ว	330
Issue ทั้งหมดบน GitHub	209 (ปิดแล้ว 198, เปิดอยู่ 11)
โค้ดไคลเอนต์ <code>frontend/lib/</code>	120 ไฟล์ ประมาณ 65,600 บรรทัด
ชุดทดสอบไคลเอนต์ <code>frontend/test/</code>	93 ไฟล์ (ไฟล์ทดสอบ 90 ไฟล์) ประมาณ 29,800 บรรทัด
โค้ดเซิร์ฟเวอร์ <code>server/src/</code>	213 ไฟล์ ประมาณ 33,500 บรรทัด
ชุดทดสอบเซิร์ฟเวอร์ <code>server/test/</code>	82 ไฟล์ ประมาณ 29,800 บรรทัด
การ deploy และโครงสร้างพื้นฐาน <code>deploy/</code>	22 ไฟล์ ประมาณ 2,800 บรรทัด
เวิร์กโฟลว์ CI/CD <code>.github/</code>	7 ไฟล์ ประมาณ 1,100 บรรทัด
เอกสาร <code>docs/</code>	286 ไฟล์ (ไฟล์ข้อความ 215 ไฟล์ ประมาณ 56,200 บรรทัด ที่เหลือเป็นภาพและ PDF)
ตารางใน Drift (schemaVersion 13)	26 ตาราง
ตารางใน PostgreSQL	29 ตาราง จาก migration 20 ไฟล์
เกณฑ์ตรวจรับ (DoD) ของเฟส 1	16 จาก 17 ข้อ (เหลือ k6, #380)
บันทึกการตัดสินใจเชิงสถาปัตยกรรม (ADR)	13 หมายเลข (ADR-0001 ถึง ADR-0013)

ตารางที่ 4.10 สรุปผลงานรายบุคคล คอมมิตนับจากอีเมลผู้เขียนคอมมิตบน `main` โดยบัญชี git สามชื่อ (NuiGates, NuiGates_2456 และ Chavatik Thorarit) เป็นของสมาชิกคนเดียวกัน และบัญชี LomerAlloys สองอีเมลเป็นของสมาชิกคนเดียวกัน ส่วน PR นับด้วย `gh pr list --state merged --json author` จำนวนคอมมิตและ PR ไม่ได้สะท้อนขนาดหรือความยากของงาน และคอมมิตส่วนหนึ่งทำร่วมกับเครื่องมือช่วยพัฒนา (บทที่ 3) ตารางนี้จึงแสดงว่าใครลงมือในส่วนใด ไม่ใช่การจัดอันดับ

ตารางที่ 4.10 ผลงานของสมาชิกแต่ละคน (นับเมื่อ 1 ตุลาคม พ.ศ. 2569)

สมาชิก (เลน)	คอมมิตบน main	PR ที่รวมแล้ว	ตัวอย่างงานที่ส่งมอบ (PR)
Chavatik Thorarit (NuimanLP, A)	890	273	สแตก Compose และ health (#41), migration และ RLS (#42), idempotency และเส้นทางขาย คีน ลีนซัก (#51, #75, #78, #84), SyncFacade และไฟล์ตัวอย่าง (#307), แก้ไขเอกสารและการทิ้งรายการในคิว (#483–#495), สคริปต์สำรองข้อมูล (#519)
LomerAlloys (B)	53	15	ระนาบผู้ดูแลแพลตฟอร์ม และการสร้างร้าน (#59), การตั้งค่าและ bootstrap (#116), จัดซื้อและต้นทุนถั่วเฉลี่ย (#119), แก้วฐานข้อมูลบนเว็บ (#310), outbox_ops และ SyncService (#324)
Pattarapon Kitcharoen (PattaraponKitcharoen, C)	115	43	JWT และบทบาทอุปกรณ์ (#57), rate limit ต่อร้าน (#77), BullMQ และ worker (#88), Ansible และ runner (#107, #348), POST /sync/push (#313), หน้าจอเจ้าของร้านตรวจ (#318), PIN ออฟไลน์ (#332), gitleaks (#526)

ตัวงานที่เปิดอยู่ 11 ตัว ได้แก่ ตัวงานหลักของเฟส 1 สองตัว (#2 และ #196) ซึ่งยังเปิดอยู่ ขณะที่เกนต์ k6 ยังไม่ผ่าน การวัดภาระ k6 (#380) การสาธิตและสเปกการสาธิตบน [mob04](#) (#344, #335) การสร้างร้านโดยไม่แตะ psql พร้อมคู่มือ (#338) เครื่องมือผู้ดูแลแพลตฟอร์ม (#443) การสำรองข้อมูลที่ถูกพักโดยเจตนา (#288, #363) งานที่รอเจ้าของโครงการตัดสินใจ (#476) และการย้ายร้านไปใช้ระบบใหม่ (#231)

บทที่ 5

ผลการดำเนินงาน

5.1 สรุปผลการดำเนินงาน

ตารางที่ 5.1 เทียบวัตถุประสงค์ทั้งเจ็ดข้อในบทที่ 1 กับผลที่ได้ หลักฐาน และสถานะ ณ วันที่ 1 ตุลาคม พ.ศ. 2569

ตารางที่ 5.1 ผลการดำเนินงานเทียบกับวัตถุประสงค์

วัตถุประสงค์	ผลที่ได้	หลักฐาน	สถานะ
1. แอป Flutter offline-first ที่คงกติกาเดิม	พอร์ตกฎธุรกรรมครบ หน้าจอ 14 ไฟล์ รายงาน ยอดขาย กำไร และสินค้า ขายดี ชุดทดสอบ 820 กรณีผ่านทั้งหมด	ตารางที่ 3.4 และ 4.2	เสร็จบนเว็บ ยังไม่ทดสอบ บน Android/iOS
2. เซิร์ฟเวอร์ที่ PostgreSQL เป็นแหล่ง ความจริง	กฎธุรกรรมบังคับที่ฝั่ง เซิร์ฟเวอร์ ชุดทดสอบ ประมาณ 492 กรณี หน่วยและ 655 กรณี แบบ e2e	ตารางที่ 4.5 และ 4.7	เสร็จ
3. แยกข้อมูลรายร้านด้วย RLS	อ่านข้ามร้านได้ 0 แถว ครบ 25 ตารางและ 12 จุดปลายทาง	ตารางที่ 4.8 ข้อ 4	เสร็จ
4. การเขียนที่ปลอดภัย เมื่อเข้าหรือพร้อมกัน	ขายพร้อมกัน 200 คำขอ บนสินค้า 50 ขึ้นได้ 50 บิลพอดี ธุรกรรมที่ยาว ที่สุดลดจากราว 101- 131 เหลือ 14-22 มิลลิวินาที	ตารางที่ 4.8 ข้อ 3 และ 15	เสร็จ
5. ทำงานออฟไลน์ (เฟส 2)	outbox_ops, SyncService, POST /sync/push, เลข RC/CN ฝั่งเครื่อง และ PIN ออฟไลน์ รวมโค้ด แล้ว แต่ละฝั่งทดสอบกับ ไฟล์ตัวอย่างคำขอชุด เดียวกัน 18 ไฟล์	ตารางที่ 4.4	โค้ดเสร็จ ยังไม่พิสูจน์ แบบครบวงจรกับ เซิร์ฟเวอร์จริง (#344)

วัตถุประสงค์	ผลที่ได้	หลักฐาน	สถานะ
6. CI/CD และการเฝ้าสังเกตระบบ	CI สองเวิร์กโฟลว์พร้อมสแกนความลับและช่องโหว่ การ deploy ผ่านการอนุมัติของผู้ตรวจที่กำหนดบนสภาพแวดล้อม demo การย้ายรุ่นอัตโนมัติ และแดชบอร์ด Grafana	ตารางที่ 4.6 และรูปที่ 4.14	เสร็จ ยกเว้นการสำรองข้อมูลออกนอก VM (#363)
7. บันทึกการตัดสินใจเป็น ADR	ADR 13 หมายเลข	หัวข้อ ADR ในบทที่ 3	เสร็จ

สิ่งที่ **ยังไม่เสร็จ** ได้แก่ การวัดโหลด k6 (#380) ซึ่งเป็นเกณฑ์ตรวจรับข้อเดียวที่ยังเปิด การสาธิตครบวงจร (#344) การสำรองข้อมูลออกนอก VM (#363, #288) และการย้ายร้านไปใช้ระบบใหม่ (cutover, #231) ร้านยังใช้บิลด์ Drift แบบเดิม ผลงานจึงเป็นระบบที่พัฒนากับร้านสาธิตและผ่านการทดสอบอัตโนมัติ ยังไม่ได้ใช้งานจริงในร้านและยังไม่มีผลการประเมินผลจากผู้ใช้อย่างเป็นทางการ

5.2 ปัญหาและอุปสรรค

หัวข้อนี้เลือกเฉพาะปัญหาที่เกิดขึ้นจริงและมีบันทึกในเอกสารของโครงการ แต่ละข้อแสดงเป็นลำดับ ปัญหา สาเหตุ วิธีแก้ และบทเรียน

5.2.1 พารามีอักษรไทยทำให้ build_runner ล้มเหลว

ปัญหา ที่เก็บโค้ดอยู่ในโฟลเดอร์ชื่อภาษาไทยที่ซิงก์ผ่าน BeeStation คำสั่ง `build_runner` [41] สำหรับสร้างโค้ดของ Drift และคำสั่ง `flutter analyze` ล้มเหลวด้วยข้อความว่าเขียนไฟล์ `build.dart.aot` ไม่ได้ สาเหตุ คอมไพเลอร์ AOT ของ Dart แปลงพารามีอักษรรนอก ASCII เป็นเครื่องหมายคำถาม และ Windows junction แก้ไม่ได้เพราะ `build_runner` ใช้พารัจจริง **วิธีแก้** คัดลอกที่เก็บโค้ดไปยังพาร ASCII เพื่อรัน `build_runner` แล้วคอมไพล์ไฟล์ `*.g.dart` ที่สร้างใหม่ และใช้ `dart analyze` แทน `flutter analyze` เสมอ **บทเรียน** ข้อจำกัดของเครื่องมือเชิงพารต้องบันทึกเป็นกฎของโครงการตั้งแต่แรก และงาน `codegen-check` ใน CI ช่วยตรวจว่าไฟล์ที่สร้างตรงกับสคีม่า

5.2.2 ไฟร์วอลล์ของคณะปิดกั้นการดึงอิมเมจจาก ghcr.io

ปัญหา การ deploy อัตโนมัติไปยัง `mob04` ถูกปิดกั้น เครื่องดึงอิมเมจจากรีจิสทรีภายนอกไม่ได้และเกิดข้อผิดพลาด `x509: certificate is not valid for any names` สาเหตุ ไฟร์วอลล์ FortiGate ของคณะตรวจและแทนที่ใบรับรองของการเชื่อมต่อ TLS ด้วยใบรับรองที่ไม่มีชื่อตรงกับโดเมนปลายทาง **วิธีแก้** แก้ได้เมื่อ 29-30 กันยายน พ.ศ. 2569 หลังทีมเครือข่ายเพิ่มชื่อทางเลือก (Subject Alternative Name: SAN) ในใบรับรอง หากอาการกลับมา ทางแก้ที่ถูกคือให้ทีมเครือข่ายยกเว้นรีจิสทรีเหล่านี้สำหรับ IP ของ VM ส่วนการคัดลอกอิมเมจด้วยมือเป็นเพียงทางแก้ฉุกเฉินในวันสาธิต ไม่ใช่ CD **บทเรียน** การออกแบบการ deploy ต้องรวมความเสี่ยงของเครือข่ายที่ทีมไม่ได้ควบคุม และกำหนดขอบเขตของทางสำรองไว้ล่วงหน้า

5.2.3 pool ฐานข้อมูลหมดเมื่อมีคำขอพร้อมกัน (#162)

ปัญหา เมื่อมีคำขอพร้อมกันเท่ากับขนาด pool ระบบหยุดตอบสนอง สาเหตุ guard แบบ global ที่ทำงานกับทุกคำขอและอ่านแผนของร้านตอนแคชว่าง ขอการเชื่อมต่อจาก pool เป็นครั้งที่สองภายในคำขอที่ถือการเชื่อมต่อแรกอยู่แล้ว ทุกคำขอจึงรอกันเอง **วิธีแก้** กำหนดเป็นกฎว่าไม่มีส่วนประกอบใดของการเชื่อมต่อที่สองในคำขอเดียว และรวม `runTx` ที่ทำพร้อมกันเป็นตัวเดียว พร้อมสเปกสถาปัตยกรรมที่บังคับจุดเชื่อมนี้ **บทเรียน** ความผิดพลาดของทรัพยากรที่จำกัดไม่ปรากฏเมื่อทดสอบทีละคำขอ ต้องมีการทดสอบที่ใช้จำนวนคำขอพร้อมกันเท่าขนาด pool (เช่น `rate-limit-pool.e2e-spec.ts` และ `health-pool.e2e-spec.ts`)

5.2.4 การทิ้งรายการในคิวย่อนผลไม่ตรง (#473, #488)

ปัญหา เมื่อเจ้าของร้านทิ้งรายการขายหรือคืนที่ค้างในคิว สต็อกและยอดในเครื่องไม่ถูกย่อนและเมื่อแก้ไขย่อนแล้ว ยอดที่ย่อนยังไม่ตรงในบางกรณี สาเหตุ การย่อนคำนวณจากข้อมูลปัจจุบัน แต่การเขียนครั้งแรกอาจถูกปิดค่าที่ศูนย์ เช่น ช่างมีเครดิตค้าง 50 บาทแต่ใบลดหนี้หักเครดิต 180 บาท ยอดจึงถูกปิดเป็น 0 และการย่อนคืนยอดเป็น 180 บาท ซึ่งสร้างหนี้ 130 บาทที่ไม่มีอยู่จริง **วิธีแก้** PR #483 ย่อนผลในธุรกรรมเดียวกับการทิ้ง และ PR #495 เก็บผลต่างที่ใช้จริงในตาราง `op_effects` (Drift schema v13) แล้วย่อนตามค่านั้น **บทเรียน** การย่อนผลที่ถูกต้องต้องบันทึกสิ่งที่เกิดขึ้นจริง ณ เวลาเขียน ไม่ใช่คำนวณกลับจากสถานะปัจจุบัน

5.2.5 เลขที่เอกสารชี้ระหว่างการออกเลขออนไลน์กับออฟไลน์ (#489)

ปัญหา เครื่องอาจออกเลขใบเสร็จหรือใบลดหนี้แบบออฟไลน์ซ้ำกับเลขที่เซิร์ฟเวอร์เคยออกให้ **สาเหตุ** เลข RC หรือ CN ที่เซิร์ฟเวอร์ออกในคำตอบออนไลน์ไม่ถูกบันทึกลงตัวนับ **DocCounters** ในเครื่อง ตัวนับในเครื่องจึงไม่รู้เลขสูงสุดที่ใช้ไปแล้วจริง **วิธีแก้** บันทึกเลขที่เซิร์ฟเวอร์ออกผ่าน **DocNumberService.commitServerIssued** ทั้งในคำตอบออนไลน์ (PR #491, #492) และในการเล่นซ้ำของ **/sync/push** (PR #494) **บทเรียน** เมื่อมีผู้ออกเลขสองฝ่าย ทุกเลขที่ฝ่ายหนึ่งออกต้องไหลกลับไปยังอีกฝ่าย กรณีนี้ยังเผยปัญหาด้านกระบวนการในหัวข้อการควบคุมการรวมโค้ดด้านล่าง

5.2.6 การควบคุมการรวมโค้ดและหลักฐาน

ปัญหา พบสามกรณี กรณีแรก ครั้งหนึ่งของการแก้ #489 (การบันทึกเลขเมื่อ **/sync/push** เล่นซ้ำ) ไม่เข้า **main** ทั้งที่ PR #491 และ #492 รวมแล้ว และคอมมิตแก้ตามการทบทวนของ PR #486 ก็หายในลักษณะเดียวกัน กรณีที่สอง ลิงก์ PR ที่ GitHub แสดงบนตัวงานให้ข้อสรุปผิดว่าบางตัวไม่มีงานส่งมอบ และตัว #184 ถูกปิดและเปิดใหม่สามครั้ง ทั้งที่เกณฑ์ด้านการวัด latency และหน่วยความจำของตัวไม่เคยมีผลวัด กรณีที่สาม สองสาขาที่ไม่มี PR เก็บสำเนาเดียวของเอกสารวิจัย (424 บรรทัด) ขณะกำลังจะถูกลบพร้อมสาขาเก่า 44 สาขาเมื่อ 22 กันยายน พ.ศ. 2569 **สาเหตุ** คอมมิตถูก push หลังจกรวม PR แล้ว [42] GitHub ไม่ผูก PR ที่ไม่มีคำหลักปิดตัวหรือมีคำหลักเฉพาะในชื่อ PR (93 จาก 201 PR ที่รวมแล้ว ณ 22 กันยายน) และการตัดสินใจว่าสาขารวมแล้วจากชื่อสาขาเป็นการเดา **วิธีแก้** cherry-pick คอมมิตที่ตกหล่นไปยัง PR #494 กู้การแก้ของ PR #486 ด้วย PR #521 ตรวจสอบส่งมอบด้วย **git log --all --grep** และตัวทดสอบเอง และรวมเอกสารวิจัยผ่าน PR #386 ก่อนลบ โดยตรวจทุกสาขาด้วย **git merge-base --is-ancestor** **บทเรียน** ก่อนรวมต้องตรวจว่า head SHA ของ PR ตรงกับที่ทบทวน สถานะของตัวไม่ใช่หลักฐาน (การนับเกณฑ์ตรวจรับในรายงานนี้จึงอ้างไฟล์ทดสอบ) และการลบที่ย้อนกลับไม่ได้ต้องพิสูจน์ก่อนว่ารวมแล้ว

5.2.7 การทดสอบที่ผูกกับเดือนปัจจุบัน (#525)

ปัญหา เมื่อขึ้นเดือนใหม่ **main** กลายเป็นสีแดง ซึ่งทำให้ไม่มีอิมเมจบน GHCR และทุกการรัน deploy ถูกข้าม **สาเหตุ** ไฟล์ **sync-push.e2e-spec.ts** ฝังเลขที่ใบเสร็จของเดือนกันยายนไว้ตายตัว แต่เซิร์ฟเวอร์ประทับวันที่ตามเวลาจริง เมื่อเดือนเปลี่ยน เซิร์ฟเวอร์จึงตั้งธงวันที่ผิดปกติตามสเปก **วิธีแก้** PR #525 คำนวณงวดของเลขเอกสารจากเวลาปัจจุบันในเขตเวลา **Asia/**

Bangkok บทเรียน การทดสอบที่พึ่งเวลาต้องอ้างอิงเวลาเดียวกับระบบที่ทดสอบ และสีแดงของ **main** ปิดกั้นการส่งมอบทั้งสาย ไม่ใช่แค่เรื่องของการทดสอบ

5.2.8 ค่า 0 เป็นเท็จในนิพจน์ของ GitHub Actions

ปัญหา เงื่อนไขข้ามการทดสอบเมื่อ push เป็นเอกสารล้วนไม่เคยทำงาน **สาเหตุ** ในนิพจน์ของ GitHub Actions [43] ตัวเลข **0** เป็นเท็จ นิพจน์รูปแบบ **cond && 0 || 1** จึงได้ **1** เสมอ ทำให้ขั้นตอน checkout ดึงประวัติเพียงคอมมิตเดียว ขึ้นจําแนกประเภทของ push จึงมองไม่เห็นคอมมิตก่อนหน้านี้ **วิธีแก้** ใช้สตริง **'0'** และ **'1'** แทนตัวเลข (PR #548 รวมเมื่อ 1 ตุลาคม พ.ศ. 2569) **บทเรียน** ชนิดข้อมูลในภาษานิพจน์ของเครื่องมือ CI ทำงานต่างจากภาษาโปรแกรมทั่วไป และการข้ามการทดสอบแบบเงียบต้องมีหลักฐานจากการรันที่สังเกตได้ว่าเงื่อนไขทำงานจริง

5.3 งานที่จะดำเนินการต่อไป

งานที่เหลือเรียงตามลำดับความสำคัญได้ดังนี้ โดยตัวที่อ้างอิงยังเปิดอยู่ ณ วันที่ 1 ตุลาคม พ.ศ. 2569

1. **การสาธิตครบวงจรบน `mob04` (#344)** รันตามรายการตรวจ [docs/handoff_log/demo-344-checklist-2026-09-30.md](#) โดยต้องจัดการข้อติดขัดที่ระบุไว้ก่อน ได้แก่ รหัสผ่านชั่วคราวของเจ้าของร้านที่ต้องตั้งรหัสใหม่ภายใน 10 นาที หลังเข้าสู่ระบบครั้งแรก ร้านใหม่ที่ยังไม่มีสินค้า การที่แอปยังส่ง **Idempotency-Key** เดิมซ้ำไม่ได้ เกณฑ์ย่อยสามข้อที่พิสูจน์บน VM ไม่ได้ และตัว #476 นอกจากนี้ยังไม่มีผู้ดูแลแพลตฟอร์มคนใดเข้าสู่ระบบ **platform-ui** จริง
2. **การวัดโหลด k6 สามเครื่องพร้อมการวัดหน่วยความจำของคอนเทนเนอร์ (#380)** ตามวิธีที่ตกลงใน [03_ARCHITECTURE.md](#) หัวข้อ 8.1 ผลที่ได้จะใช้ยืนยันเกณฑ์ตรวจรับข้อสุดท้าย และเป็นการวัดหน่วยความจำรวมของสแตกบน **mob04** ครั้งแรก
3. **การสำรองข้อมูลออกนอก VM (#363 และ #288)** ถูกพักไว้จนหลังการสาธิต เมื่อกลับมาทำต้องทดสอบเส้นทางเครือข่ายจาก **mob04** ไปยังร้านก่อนสร้างข้อมูลรับรองเลือกโปรโตคอล (อุปกรณ์ BeeStation ไม่เปิด SSH/SFTP ที่ใช้ได้ จึงต้องเลือกระหว่างการเชื่อมผ่านโปรโตคอล Server Message Block (SMB) กับการซื้ออุปกรณ์จัดเก็บข้อมูลบนเครือข่าย (Network-Attached Storage: NAS) ที่รองรับ SSH File Transfer Protocol (SFTP)) ติดตั้ง **rclone** ส่งสำเนาจริงครั้งแรก และกู้คืนจากสำเนานอก VM ให้ได้ ระหว่างนี้ระบบจึงยังไม่มีสำเนาข้อมูลนอก VM

4. **ตัว #476** ทางต้นของการจัดการอุปกรณ์เมื่อเบราว์เซอร์ที่ลงทะเบียนไว้เครื่องสุดท้ายสูญหาย ซึ่งต้องให้เจ้าของโครงการตัดสินใจ
 5. **การย้ายร้านไปใช้ระบบใหม่ (cutover, #231)** ซึ่งเป็นตัวสุดท้ายของลำดับงานเริ่มต้นเฟส 2 ต้องมีแผนทดสอบก่อนสลับและแผนย้อนกลับ
 6. **ข้อค้นพบที่เหลือจากการตรวจโค้ดทั้งระบบ** ได้แก่ ข้อบกพร่องระดับต่ำเรื่อง **POST / shifts/open** ออนไลน์ที่ละเลย **startingCash** เมื่อส่ง **id** เดิม และข้อค้นพบด้านมาตรฐานโค้ดที่ยังไม่ได้จัดลำดับ
 7. **ตัวงานหลัก #443** ต้องให้เจ้าของโครงการตัดสินใจว่าเกณฑ์ "เฉพาะ **bootstrap:admin** เท่านั้นที่สร้างผู้ดูแลได้" จะปรับให้สอดคล้องกับการชิง **PLATFORM_ADMINS** อย่างไร
 8. **ฮาร์ดแวร์เฉพาะร้าน** เครื่องพิมพ์ใบเสร็จแบบความร้อน ลินซ์เก็บเงิน และการสแกนบาร์โค้ด ซึ่งต้องทำที่หน้าร้านจริง และงานเสริมความมั่นคงที่ยังเหลือ เช่น การให้เว็บไม่ต้องดึงฟอนต์สำรองจากภายนอก และ service worker ที่ยังไม่ precache ทรัพยากร
 9. **การรับรองข้อความภาษาไทย** ข้อความสามชุดที่คณะผู้จัดทำร่างไว้ (ข้อความ "เปิดกะใหม่" ข้อความอธิบายรหัสลงทะเบียน 8 ตัวอักษร และแบนเนอร์หลังลงทะเบียนเครื่อง) รอการรับรองใน **02_API_SCREEN.md** ตามนิยามผู้รับรองในบทที่ 1
 10. **การสร้างและทดสอบบน Android และ iOS** ปัจจุบัน CI สร้างเฉพาะบิลด์เว็บ แอปบนอุปกรณ์เคลื่อนที่จึงยังไม่ได้ทดสอบบนอุปกรณ์จริง
- นอกจากนี้ต้องตรวจบันทึกการสำรองข้อมูลรอบ 03:00 บน **mob04** เป็นระยะ และปรับไฟล์ฐานข้อมูลบนเว็บทุกครั้งที่ยกระดับ **drift** หรือ **sqlite3**

บรรณานุกรม

- [1] "01 — Product & Pitch," ไฟล์ docs/study/01_pitch.md ของที่เก็บโค้ดโครงการ srisurart-pos-flutter, 2026.
- [2] Mozilla Developer Network, "Window: localStorage property." [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage>. Accessed: Oct. 1, 2026.
- [3] "02 — System Architecture Overview," ไฟล์ docs/study/02_architecture.md ของที่เก็บโค้ดโครงการ srisurart-pos-flutter, 2026.
- [4] Google, "Flutter documentation." [Online]. Available: <https://docs.flutter.dev/>. Accessed: Oct. 1, 2026.
- [5] S. Binder, "Drift documentation." [Online]. Available: <https://drift.simonbinder.eu/>. Accessed: Oct. 1, 2026.
- [6] SQLite Consortium, "SQLite documentation." [Online]. Available: <https://www.sqlite.org/docs.html>. Accessed: Oct. 1, 2026.
- [7] "03 — Architecture options and phase-1 plan," ไฟล์ docs/Backend_design/03_ARCHITECTURE.md, 2026.
- [8] "ADR-0004: Device roles," ไฟล์ docs/Backend_design/adr/0004-device-roles.md, 2026.
- [9] "ADR-0010: Client write-through cache," ไฟล์ docs/Backend_design/adr/0010-client-write-through-cache.md, 2026.
- [10] The PostgreSQL Global Development Group, "Row Security Policies," PostgreSQL Documentation. [Online]. Available: <https://www.postgresql.org/docs/current/ddl-rowsecurity.html>. Accessed: Oct. 1, 2026.
- [11] "ADR-0012: Replace PostgreSQL with CouchDB (Rejected)," ไฟล์ docs/Backend_design/adr/0012-couchdb-replaces-postgres.md, 2026.
- [12] "08 — Phase 2 spec: offline shell + production ในมหาวิทยาลัย," ไฟล์ docs/Backend_design/08_PHASE2_SPEC.md, 2026.

- [13] "ADR-0013: CI/CD toolchain," ไฟล์ docs/Backend_design/adr/0013-cicd-toolchain.md, 2026.
- [14] "ADR-0001: Tenant provisioning," ไฟล์ docs/Backend_design/adr/0001-tenant-provisioning.md, 2026.
- [15] "09 — Phase 2: lane split and ticket list," ไฟล์ docs/Backend_design/09_PHASE2_LANES.md, 2026.
- [16] M. Kleppmann, A. Wiggins, P. van Hardenberg, and M. McGranaghan, "Local-first software: You own your data, in spite of the cloud," in Proc. ACM SIGPLAN Int. Symp. New Ideas, New Paradigms, and Reflections on Programming and Software (Onward!), 2019. [Online]. Available: <https://www.inkandswitch.com/local-first/>. Accessed: Oct. 1, 2026.
- [17] W. Vogels, "Eventually consistent," Commun. ACM, vol. 52, no. 1, pp. 40-44, Jan. 2009.
- [18] F. Angelov, "bloc / flutter_bloc documentation." [Online]. Available: <https://bloclibrary.dev/>. Accessed: Oct. 1, 2026.
- [19] Flutter Team, "go_router package." [Online]. Available: https://pub.dev/packages/go_router. Accessed: Oct. 1, 2026.
- [20] PostgreSQL Global Development Group, "PostgreSQL documentation: Concurrency control, explicit locking, deadlocks." [Online]. Available: <https://www.postgresql.org/docs/current/mvcc.html>. Accessed: Oct. 1, 2026.
- [21] R. Fielding, M. Nottingham, and J. Reschke, Eds., "HTTP Semantics," RFC 9110, IETF, Jun. 2022. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc9110>. Accessed: Oct. 1, 2026.
- [22] J. Jena and S. Dalal, "The Idempotency-Key HTTP Header Field," IETF Internet-Draft, draft-ietf-httpapi-idempotency-key-header. [Online]. Available: <https://datatracker.ietf.org/doc/draft-ietf-httpapi-idempotency-key-header/>. Accessed: Oct. 1, 2026.
- [23] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, IETF, May 2015. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc7519>. Accessed: Oct. 1, 2026.

- [24] A. Biryukov, D. Dinu, D. Khovratovich, and S. Josefsson, "Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications," RFC 9106, IETF, Sep. 2021. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc9106>. Accessed: Oct. 1, 2026.
- [25] NestJS, "NestJS documentation." [Online]. Available: <https://docs.nestjs.com/>. Accessed: Oct. 1, 2026.
- [26] Redis Ltd., "Redis documentation." [Online]. Available: <https://redis.io/docs/>. Accessed: Oct. 1, 2026.
- [27] Taskforce.sh, "BullMQ documentation." [Online]. Available: <https://docs.bullmq.io/>. Accessed: Oct. 1, 2026.
- [28] F5 NGINX, "NGINX documentation." [Online]. Available: <https://nginx.org/en/docs/>. Accessed: Oct. 1, 2026.
- [29] Docker Inc., "Docker documentation." [Online]. Available: <https://docs.docker.com/>. Accessed: Oct. 1, 2026.
- [30] Docker Inc., "Docker Compose overview." [Online]. Available: <https://docs.docker.com/compose/>. Accessed: Oct. 1, 2026.
- [31] GitHub, "GitHub Actions documentation." [Online]. Available: <https://docs.github.com/en/actions>. Accessed: Oct. 1, 2026.
- [32] GitHub, "Working with the Container registry." [Online]. Available: <https://docs.github.com/en/packages/working-with-a-github-packages-registry/working-with-the-container-registry>. Accessed: Oct. 1, 2026.
- [33] Aqua Security, "Trivy documentation." [Online]. Available: <https://trivy.dev/>. Accessed: Oct. 1, 2026.
- [34] Gitleaks, "gitleaks: detect secrets in git repositories." [Online]. Available: <https://github.com/gitleaks/gitleaks>. Accessed: Oct. 1, 2026.
- [35] Red Hat, "Ansible documentation." [Online]. Available: <https://docs.ansible.com/>. Accessed: Oct. 1, 2026.
- [36] Prometheus Authors, "Prometheus documentation." [Online]. Available: <https://prometheus.io/docs/>. Accessed: Oct. 1, 2026.
- [37] Grafana Labs, "Grafana documentation." [Online]. Available: <https://grafana.com/docs/>. Accessed: Oct. 1, 2026.

- [38] M. Nygard, "Documenting architecture decisions," Cognitect Blog, Nov. 2011. [Online]. Available: <https://www.cognitect.com/blog/2011/11/15/documenting-architecture-decisions>. Accessed: Oct. 1, 2026.
- [39] Redis Ltd., "Redis documentation: Key eviction," [Online]. Available: <https://redis.io/docs/latest/develop/reference/eviction/>. Accessed: Oct. 1, 2026.
- [40] Mozilla, "Progressive web apps," MDN Web Docs. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps. Accessed: Oct. 1, 2026.
- [41] Dart Team, "build_runner," pub.dev. [Online]. Available: https://pub.dev/packages/build_runner. Accessed: Oct. 1, 2026.
- [42] GitHub, "About pull request merges," [Online]. Available: <https://docs.github.com/en/pull-requests>. Accessed: Oct. 1, 2026.
- [43] GitHub, "Expressions — GitHub Actions," [Online]. Available: <https://docs.github.com/en/actions/learn-github-actions/expressions>. Accessed: Oct. 1, 2026.